



SPEARBIT

Kiln Security Review

Auditors

Optimum, Lead Security Researcher

Saw-Mon and Natalie, Lead Security Researcher

Xiaoming90, Security Researcher

0x4non, Associate Security Researcher

Report prepared by: Pablo Misirov

July 27, 2023

Contents

1	About Spearbit	2
2	Introduction	2
3	Risk classification	2
3.1	Impact	2
3.2	Likelihood	2
3.3	Action required for severity levels	2
4	Executive Summary	3
5	Findings	4
5.1	Critical Risk	4
5.1.1	Rental / User info can be overwritten during its rental period	4
5.1.2	LArray's del function does not clear the dynamic array slots correctly.	5
5.2	High Risk	6
5.2.1	Nexus's spawned factories can plug core hachers with custom call data.	6
5.2.2	Allowed depositors who deposit into withdrawal channel 0 do not pay commission fees to the vTreasury	7
5.2.3	vNFT token owner may front-run a call to setUser with a call to exit	7
5.2.4	vNFT token owner can request exit while there is an active renter	8
5.2.5	A user/renter can block the vNFT owner from clearing or setting its address after the expiration time	8
5.2.6	vNFT does not enforce integrator commission fees for generated CL rewards	9
5.2.7	vNFT - The freeze mechanism does not fully mitigate front-running related issues	9
5.2.8	Nexus Admin can DoS all spawned pool's reporting process	10
5.2.9	A malicious pool can DoS the staking process for MultiPool20	10
5.2.10	Updating the routing weights of the MultiPool20 pools can have unexpected consequences for the stakers	11
5.2.11	The vExitQueue's _onTransfer hook does not update the owner field in \$tickets.get()[id]	11
5.2.12	Malicious ticket seller might trick a buyer to buy a depreciated token	12
5.3	Medium Risk	13
5.3.1	MultiPool20 mints 1:1 for the initial stake() and does not use its pools' internal conversion rates.	13
5.3.2	vFactory's admin/operator can collude with allowed depositors to avoid Nexus's protocol fee	13
5.3.3	Any allowed depositor can deposit for any withdrawal channel	14
5.3.4	Liquid20A.requestExit input parameter is not handled correctly	15
5.3.5	Users and owners would need to trust the \$execLayerVault.get()'s maintainer to distribute those fees fairly.	15
5.3.6	vExitQueue does not favour user's potential exit amount when updating the ticket's maxExitable on the claim process.	15
5.3.7	vFactory.addValidators - Possible duplicate keys may lead to wrong allocation of deposits	16
5.3.8	vTreasury.withdraw - In edge cases attackers might be able to lock ERC20 tokens from being withdrawn from the contract	16
5.3.9	MultiPool20._requestExit - Changes in \$poolRoutingList and \$poolMap may cause a temporary denial of service for exit requests	17
5.3.10	Staking to MultiPool will revert if there is a disabled pool	18
5.3.11	The Nexus global fee recipient can DoS the vTreasurys during the fee distribution or token withdrawal process	19
5.4	Low Risk	19
5.4.1	vExitQueue is missing to analyse some cases	19
5.4.2	Anyone can easily send funds to MinimalRecipient	19
5.4.3	MultiPool can receive shares for deactivated pools, pools with zero routing weights and shares transferred from 3rd parties	20
5.4.4	Any allowed depositor can request more validators for any withdrawal channel	21

5.4.5	Anyone can frontrun <code>Hatcher.deleteGlobalFix(...)</code> to force apply a fix to a cub if certain conditions are present	22
5.4.6	Add check to prevent minting zero shares	22
5.4.7	<code>LibErrors.Unauthorized</code> 's second parameter might not be correct	23
5.4.8	A to be updated global oracle member can frontrun its address change to still cast votes for all the oracle aggregators	23
5.4.9	Manually clean the dirty bits of an address	24
5.4.10	<code>vNFT.setUser</code> - approved/owner of a <code>vNFT</code> may set a user for a period ending now or ended in the past	24
5.4.11	Epoch validation check is not accurate	24
5.4.12	Pluggable <code>Hatcher</code> admins and the <code>Nexus</code> would need to have a trustful connection	25
5.4.13	core pluggable hachers <code>nexus</code> parameters are not checked	26
5.4.14	Balanced notion in <code>MultiPool20</code>	26
5.4.15	<code>_transferFrom</code> does not check whether the allowance is <code>type(uint256).max</code>	27
5.4.16	<code>vFactory._approve</code> - admin may set a limit greater than <code>WithdrawalChannel.validators.length</code>	27
5.4.17	<code>vFactory.removeValidators</code> - Potential sync issues may result in the removal of more than one validator	28
5.4.18	<code>MultiPool20</code> 's conversion rate decreases upon a user's exit	28
5.4.19	Some endpoints reads from storage but it are declared as pure	29
5.4.20	<code>FeeDispatcher._setFeeSplit</code> - Recipients array might contain duplications	30
5.4.21	Dust <code>ETH</code> accumulate on <code>MultiPool20</code> contract	30
5.4.22	Use <code>safeTransfer()</code> instead of <code>transfer()</code> for <code>ERC20</code> transfers.	30
5.4.23	<code>MultiPool20._setPoolPercentages</code> - Potential ordering dependence issues	31
5.4.24	<code>MultiPool.changeFee</code> - The new fee should not exceed a maximum value which should be less than 100%	31
5.4.25	<code>Factory</code> 's admin cannot stop the global oracle member from voting for reports	32
5.4.26	<code>replaceCoreHatchers(...)</code> does not update the pluggable hatcher's mapping	32
5.4.27	<code>commitFixes()</code> can be called by anyone and a cub's fix can prevent other fixes to be applied	32
5.4.28	<code>_mint(...)</code> is missing checks after the <code>_onMint(...)</code> hook call	33
5.4.29	<code>_burn(...)</code> does not check whether token still exists after calling <code>_onBurn(...)</code> hook	33
5.4.30	Lack of checks after calling <code>_onTransfer</code> hook	34
5.4.31	Minter counter <code>\$idCounter</code> its not incremented on <code>_mint</code>	34
5.5	Gas Optimization	35
5.5.1	Pre-increment is cheaper than post-increment	35
5.5.2	Expression <code>x = x - y</code> or <code>x = x + y</code> is cheaper than <code>x -= y</code> or <code>x += y</code>	35
5.5.3	Cache redundant math operations, and cache <code>_thisBalance</code>	36
5.5.4	In <code>vOracleAggregator:submitReport</code> state variables <code>\$pool</code> and <code>\$reportVoteCounts</code> can be cached	37
5.5.5	Remove unnecessary <code>else</code> statement for gas optimization	38
5.5.6	<code>toLittleEndian64</code> function could be optimize	38
5.5.7	Optimize <code>authorized</code> function with using <code>yul</code>	39
5.5.8	Optimize <code>min</code> and <code>max</code> functions using a <code>yul</code> implementation	39
5.5.9	On <code>vPool:_transfer</code> balance can be reused to avoid extra storage read	40
5.5.10	On <code>vPool:purchaseValidators</code> computation of the total cost can be cached	40
5.5.11	On <code>vPool:report</code> state variable <code>\$coveredBalanceSum.get()</code> can be cache to avoid extra storage reads	41
5.5.12	Optimize <code>onlyOperatorOrAdmin</code> modifier by consolidating function calls and reducing storage reads	43
5.5.13	Optimize <code>vPool:_burn</code> function by reducing redundant reads	43
5.5.14	Optimize <code>vPool:_mint</code> function by reducing redundant reads	44
5.5.15	Optimize <code>vPool:_consumeApproval</code> function by reducing storage reads	45
5.5.16	Functions <code>toUint256</code> and <code>toBool</code> can be optimized using <code>Yul</code>	46
5.5.17	Improve efficiency in <code>vTreasury:onvPoolSharesReceived</code> by updating <code>poolShares</code>	47
5.5.18	Refactor <code>vFactory:withdraw</code> function to avoid extra storage reads	47
5.5.19	<code>LibSignature.SIGNATURE_LENGTH</code> - 64 is constant therefore can be precalculated.	48

5.5.20	The depositAmount is constant; therefore toLittleEndian64 value can be precalculated and set as a constant.	49
5.5.21	Calculation of sharesAcquired in _stake can be simplified	50
5.5.22	vFactory._removeValidatorsFromWithdrawalChannel - redundant check inside a loop . . .	51
5.5.23	Checking the equality of the length of multiple function parameters can be avoided	52
5.5.24	Cache variable d.withdrawalChannel in vFactory:setOwner	52
5.5.25	Use the cached withdrawalChannelCount variable instead of withdrawalChannels.length	53
5.5.26	_removeValidatorsFromWithdrawalChannel can be optimized	54
5.5.27	On _approve cache withdrawalChannelStruct.limit and avoid return using if else . . .	55
5.5.28	Cache withdrawalChannelStruct.funded in exitTotal	55
5.5.29	Remove redundant checks in vFactory::initialize for address(0)	56
5.5.30	Cache currentBalance - commission and perform calculation only if commission > 0	57
5.5.31	Refactor If statements that use && into nested if statements	57
5.5.32	_applyVote and _hasVoted can be optimized using a bit operations	57
5.5.33	_getVote can be optimized	58
5.5.34	Replace pool.exitQueue() with exitQueue which is already defined	59
5.5.35	Pass owner to the _approve to save from reading it again from storage.	59
5.5.36	Use the cached owner instead of using _ownerOf(tokenId)	60
5.5.37	Implement unchecked blocks in utils.sol/src/NFT.sol as used in OZ NFT implementation	61
5.5.38	Optimize _hatch function, saving a step when hatching a new Cub	61
5.6	Informational	62
5.6.1	MultiPool1155.requestExit - Consider implementing a burn function instead	62
5.6.2	Proxy/cubs behind TUPProxy/Hatcher can have their implementation updated	62
5.6.3	vExitQueue's name() can be changed.	62
5.6.4	Loop can be exited early	62
5.6.5	Suggestions regarding vExitQueue's _consumeTicket and claim	63
5.6.6	All exit queues of the pools connected to the same factory would have the same name	65
5.6.7	All vPools of the same factory would have the same name().	65
5.6.8	vPool.name() might not always return the same name	65
5.6.9	Suggestions for vPool's report(...) endpoints	65
5.6.10	The precision returned by rate() is not used during the calculations internally	69
5.6.11	The logic around decrementing \$poolShares.get() [poolId] and transferring shares to the exit queue can be refactored	69
5.6.12	Anyone can call vFactory.exitTotal(...) to cause custom events to be fired	70
5.6.13	vFactory._approve(...) can be improved regarding some edge cases	70
5.6.14	_removeValidatorsFromWithdrawalChannel can be simplified	71
5.6.15	MinimalRecipient can call the target even when the supplied value to exec is 0	72
5.6.16	Document what it would mean for the owner or user set the token user expiration to 0	72
5.6.17	Further analysis of conversion rates used in MultiPool... _stake(...) is needed	73
5.6.18	Native20 and Native1155 do not need to inherit from Fixable unless also used by Hatcher	73
5.6.19	Token approval race condition	74
5.6.20	A few of TUPProxy endpoints can be declared as view	74
5.6.21	vPool:report could be refactored to reduce complexity	74
5.6.22	LibRLP is not used (and 2 other minor issues)	75
5.6.23	A hardcoded address is used in vTreasury to signal the usage of native token	75
5.6.24	Voting issue when upgrading to global oracle member within same epoch	76
5.6.25	Refactor the deploy function on ProxyFactory.sol to use named return	76
5.6.26	Add check for poolId in exitCommissionShares function on MultiPool contract	77
5.6.27	Define named constants for locked states in FeeDispatcher	77
5.6.28	Lack of setter functions for several variables in vNFT	78
5.6.29	Review function visibility in MultiPool20:_totalUnderlyingSupply and MultiPool20:_totalSupply'	79
5.6.30	Refactor vPool:report to use LibUInt256.max for clarity and cleanliness	79
5.6.31	Clarify use of indicator bit in \$operatorFeeVote and \$globalRecipientFeeVote	80
5.6.32	Missing event emission when setting deposit contract address	80
5.6.33	Rename _onTransfer to _beforeTokenTransfer for clarity	80

5.6.34	Missing event for critical setting change	81
5.6.35	Inconsistent function naming convention	81
5.6.36	Remove unused payable to prevent accidental ETH transfer	81
5.6.37	Validators can change EL fee recipient anytime	82
5.6.38	Remove unused imports for code cleanliness	82
5.6.39	Use <code>address(this).balance</code> instead of <code>selfbalance()</code>	83
5.6.40	Remove unused modifiers	83
5.6.41	There is not an explicit endpoint to exit shares for a <code>IvPool</code>	83
5.6.42	Oracle members need to listen to <code>\$epochsPerFrame</code> parameter changes	84
5.6.43	Rename <code>types</code> library to better indicate its purpose	84
5.6.44	Add more documentation for <code>CAddress.{v,k}</code>	85
5.6.45	<code>TUPProxy</code> can change its admin when frozen	85
5.6.46	<code>Cub</code> allows the <code>beacon</code> to call into it even when the <code>cub</code> is paused	85
5.6.47	Remove unused return keywords	86
5.6.48	There isn't a setter for <code>\$fixProgresses.get()</code> [<code>cub.k()</code>] for the <code>Hatcher</code> admin	86
5.6.49	Pending fixes cannot be unregistered after the <code>Hatcher</code> is frozen.	87
5.6.50	Calculation of <code>\$exitedEth.get()</code> [<code>poolId</code>] when exiting from a <code>MultiPool20</code>	87
5.6.51	<code>MultiPool.changeFee(...)</code> can be simplified	88
5.6.52	Document the choice of conversion rate for calculating <code>pSharesBought</code>	89
5.6.53	Keep the struct fields in the same order for <code>Native1155Configuration</code> and <code>Native20Configuration</code>	90
5.6.54	The balancing algorithm performed in <code>_requestExit</code> depends on the order of pools	91
5.6.55	<code>FeeDispatcher</code> does not have an initialiser but a constructor	91
5.6.56	Some contracts can be marked <code>abstract</code>	92
5.6.57	Add more documentation for <code>\$frameSize</code> in <code>MerkleVault</code>	92
5.6.58	Error and typos in comments / <code>NatSpec</code>	92
5.6.59	Check factory is not <code>address(0)</code> in <code>IntegrationFactory._deploy(...)</code>	93
5.6.60	use <code>abi.encodeCall</code> instead of <code>abi.encodeWithSelector</code> whenever possible	93
5.6.61	<code>IllegalExpirationExtensionAttempt</code> is not included in EIP-4907	94
5.6.62	Expose view functions for some of the key used parameters	94
5.6.63	<code>vExitQueue.claim</code> - Consider adding the recipient to the revert message in case of failure	95
5.6.64	<code>FeeDispatcher.withdrawCommission</code> - Consider adding the recipient to the revert mes- sage in case of failure	95
5.6.65	<code>_requestExit</code> should call <code>_burn</code> instead of duplicating the <code>_burn</code> logic	96
5.6.66	Withdrawal commission process can be DOS	96
5.6.67	Use <code>LibUint256.mulDiv</code> like in above functions for calculate the <code>comission</code>	96
5.6.68	Use <code>BASIS_POINTS_MAX</code> instead of 10000 for clarity and consistency	97
5.6.69	Miscalculation and lack of documentation consistency for unstructured storage slots	97
5.6.70	Use the named constants as indices in <code>replaceCoreHatchers(...)</code>	99
5.6.71	<code>Hatcher.status(...)</code> can be simplified	99
5.6.72	The calculation of <code>fixCount</code> in <code>fixes(...)</code> might underflow and revert	99
5.6.73	Turn <code>Hatcher</code> into an abstract contract	100
5.6.74	<code>Hatcher's InitialProgress</code> can be decreased which might activate stale fixes for to-be- hatched cubs	100
5.6.75	Decrement storage slot hashes by one	100
5.6.76	Rename variable <code>\$idCounter</code> to <code>\$mintCounter</code>	101
5.6.77	Clear approvals after burning token	101
5.6.78	Use <code>val.k()</code> or <code>val.v()</code> instead of <code>val.toUint256()</code> in <code>NFT.sol</code> to be consistent with the <code>Hatcher</code>	102

1 About Spearbit

Spearbit is a decentralized network of expert security engineers offering reviews and other security related services to Web3 projects with the goal of creating a stronger ecosystem. Our network has experience on every part of the blockchain technology stack, including but not limited to protocol design, smart contracts and the Solidity compiler. Spearbit brings in untapped security talent by enabling expert freelance auditors seeking flexibility to work on interesting projects together.

Learn more about us at spearbit.com

2 Introduction

Kiln is a staking platform you can use to stake directly, or whitelabel staking into your product. It enables users to stake crypto assets, manually or programmatically, while maintaining custody of your funds in your existing solution, such as Fireblocks, Copper, or Ledger.

Disclaimer: This security review does not guarantee against a hack. It is a snapshot in time of [vsuite](#), [vsuite-integrations](#) and [utils.sol](#) according to the specific commit. Any modifications to the code will require a new security review.

3 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

3.1 Impact

- High - leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium - global losses <10% or losses to only a subset of users, but still unacceptable.
- Low - losses will be annoying but bearable--applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

3.2 Likelihood

- High - almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium - only conditionally possible or incentivized, but still relatively likely
- Low - requires stars to align, or little-to-no incentive

3.3 Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

4 Executive Summary

Over the course of 20 days in total, Kiln engaged with Spearbit to review [vsuite](#), [vsuite-integrations](#) and [utils.sol](#). In this period of time a total of **172** issues were found.

Note: This document is a continuation to the [staking-contracts](#) report and constitutes the second part of the security review

Summary

Project Name	Kiln
Repository	vsuite
Commit	4ab2a7...23de
Repository	vsuite-integrations
Commit	0b87a2...7c84
Repository	utils.sol
Commit	771718...f6b2
Type of Project	Enterprise Staking, Liquid Staking
Review Timeline	April 17 - May 12
Two week fix period	May 12 - May 26

Post review hashes

vsuite	93e00c...c5a1
vsuite-integrations	a0c5f7...0cc7
utils.sol	08ffea...7cca

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	2	2	0
High Risk	12	8	4
Medium Risk	11	6	5
Low Risk	31	22	9
Gas Optimizations	38	30	8
Informational	78	58	20
Total	172	126	46

5 Findings

5.1 Critical Risk

5.1.1 Rental / User info can be overwritten during its rental period

Severity: Critical Risk

Context:

- [vNFT.sol#L327](#)
- [vNFT.sol#L356](#)
- [vFactory.sol#L595](#)
- [vFactory.sol#L808](#)
- [MinimalRecipient.sol#L48](#)

Description: Rental / User info can be overwritten during its rental period. This can be achieved due to the fact that when the withdrawal recipient of a token's validator has a positive balance during the `setUser` or `clearUsership` call, we would get a call back to the current `_beneficiaryOf(tokenId)` or `userInfo.user` and then after this callback we would actually set the new user info in storage. So a malicious owner or a current user of a token can take advantage of this callback to set the user info multiple times by reentering into these endpoints.

The above re-entrancy trick could have also been used by the owner to first transfer the ownership to a new owner (sell its token) then set itself as the user with expiration as $2^{64} - 1$.

Recommendation: It is recommend to add re-entrancy locks to critical endpoints such as `setUser` and `clearUsership` and also set the storage parameters before the `if` blocks where the factory's withdraw endpoint is called.

Rough sketch:

```
function setUser(
    uint256 tokenId,
    address user,
    uint64 expires
) external nonReentrant {
    ...
    address beneficiary = _beneficiaryOf(tokenId);

    // update userInfo
    userInfo.user = user;
    userInfo.expiration = expires;

    // update freeze timestamp
    mapping(uint256 => uint256) storage freezeTimestamps = $freezeTimestamps.get();
    uint256 freezeTimestamp = block.timestamp + $timeout.get();
    freezeTimestamps[tokenId] = freezeTimestamp;

    if (_getWithdrawableAmount(tokenId) > 0) {
        uint256[] memory tokenIds = new uint256[](1);
        tokenIds[0] = tokenId;
        // slither-disable-next-line reentrancy-events
        _withdraw(tokenIds, beneficiary, false);
    }

    emit UpdateUser(tokenId, user, expires);
    emit Freeze(tokenId, freezeTimestamp);
}
```

and


```

function clearUsership(uint256 tokenId) external nonReentrant {
    vitypes.User4907 storage userInfo = $users.get()[tokenId];
    ...

    // note here we use _beneficiaryOf instead of `userInfo.user`
    // the difference would be the old implementation would revert if
    // `userInfo.user` was `address(0)` if getWithdrawableAmount(tokenId) > 0
    // but with this suggestion in that case funds will be sent to the owner
    address beneficiary = _beneficiaryOf(tokenId);

    // clear userInfo
    userInfo.expiration = 0;
    userInfo.user = address(0);

    if (_getWithdrawableAmount(tokenId) > 0) {
        uint256[] memory tokenIds = new uint256[](1);
        tokenIds[0] = tokenId;
        // slither-disable-next-line reentrancy-events
        _withdraw(tokenIds, beneficiary, true);
    }

    emit UsershipCleared(tokenId);
}

```

Kiln: Fixed in [595916b3fd8116e2c890d8b35079218f12da6fa1](#) by eliminating the ability to call back the contract by implementing a "pull" mechanism to claim the debt instead.

Spearbit: Fixed.

5.1.2 LArray's del function does not clear the dynamic array slots correctly.

Severity: Critical Risk

Context:

- [array.sol#L54](#)

Description: LArray's del assumes that dynamic array elements are positioned consecutively after the array storage pointer slot where it hold the array length. But [solidity](#) actually uses the array storage pointer slot to calculate where in the storage the array elements should start using the keccak256 hash of this pointer slot.

So the current implementation wipes out irrelevant storage slots while keeping the array elements intact.

This deleting functionality has been used for [clearing report slots](#) in [vsuite](#) among other places.

Recommendation: Optimised implementation of the del function

```
// SPDX-License-Identifier: MIT

function del(types.Array position) internal {
    // slither-disable-next-line assembly
    assembly {
        let len := sload(position)

        if len {
            // clear the length slot
            sstore(position, 0)

            // calculate the starting slot of the array elements in storage
            mstore(0, position)
            let startPtr := keccak256(0, 0x20)

            for {} len {} {
                len := sub(len, 1)
                sstore(add(startPtr, len), 0)
            }
        }
    }
}
```

It's recommended to add unit tests to all these utility functions to make sure their supposed effects/actions take place. Also since the in `vsuite` this bug was not caught for example when examining the report clearing process, more test cases also need to be added for `vsuite` in regards to the usage of these utility functions.

Kiln: Fixed in [PR 8](#).

Spearbit: Fixed.

5.2 High Risk

5.2.1 Nexus's spawned factories can plug core hachers with custom call data.

Severity: High Risk

Context:

- [Nexus.sol#L109-L116](#)
- [Nexus.sol#L284-L297](#)

Description: A spawned Nexus factory's admin can call into `plugOnFactory(...)` and plug/hatch cub for core hachers since upon `initialisation` of the Nexus core hachers are also marked to be pluggable hachers.

This for example means that a spawned factory's admin can hatch `plugOnFactory` a:

- treasury
- factory
- pool
- exit queue
- oracle aggregator
- ...

where the call data to the `IPluggableHatcher(pluggableHatcher).plug(cdata)` can be decided by the factory's admin and so it can ejects itself completely from the Nexus system (for example by crafting a call data that the Nexus addresses are replaced by any desired address).

After detecting this issue, the Nexus's admin could call into `unregisterPluggableHatcher(...)` to remove the core hachers from the pluggable ones.

Recommendation: To avoid the issue in the above context, it would be best for the Nexus to not mark the core hachers as pluggable hachers. And so the assignments in the `initialize(...)` endpoint should be removed.

Kiln: Fixed.

Spearbit: Fixed.

5.2.2 Allowed depositors who deposit into withdrawal channel 0 do not pay commission fees to the `vTreasury`

Severity: High Risk

Context:

- [vFactory.sol#L787](#)
- [MinimalRecipient.sol#L42](#)

Description: A factory's allowed depositor if deposits into the `bytes32(0)` withdrawal channel, can withdraw its EL and CL related balances without paying any protocol fee to the Nexus's global fee recipient or the factory's treasury's operator fee recipient. This is due to:

1. The allowed depositor can set its own EL fee recipient when depositing into the factory
2. When an amount is withdrawn from the CL fee recipient no portion of the balance is sent to the `vTreasury`.

Recommendation: If it's important to take commission on the generated rewards and fees, the followings should be implemented:

1. Stricter restriction on the provided EL fee recipient where the factory deploys its own contract for the EL fees as well and associate it with the deposited validator's public key.
2. Introduce a parameter for the operator fee percentage in the `vFactory` contract that is associated for the 0 withdrawal channel and use this parameter in the implementation of `_withdraw(...)` to send portion of the generated reward to the `vTreasury` contract. This might also require to have more storage variables to make sure not apply commission fees for the 32 ETH staked amount.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.2.3 `vNFT` token owner may front-run a call to `setUser` with a call to `exit`

Severity: High Risk

Context:

- [vNFT.sol#L298](#)

Description: The freezing mechanism is partially, as described in [issue 102](#) solving a potential front-running issue in which a `vNFT` token owner front-runs a dex swap transaction with a call to the `exit` function. However, the scenario where the same owner front-runs a call to `setUser` with a call to `exit` is not mitigated by the same freezing mechanism.

Recommendation: Consider applying the recommendation for issue [PR 102](#).

Kiln: Fixed in [d05f25b2d809823d05db13b8a82aa825e0d04c09](#) by using `$exited` to mark the `validatorId` as exited, and requiring `setUser` and `transfer` to only accept non-exited tokens.

Spearbit: Fixed.

5.2.4 vNFT token owner can request exit while there is an active renter

Severity: High Risk

Context:

- [vNFT.sol#L403-L407](#)

Description: The vNFT contract implements `setUser` as part of the IERC4907 rentable NFT ERC. The idea is to allow a rental market for future accrued fees which will effectively act as a hedging instrument for vNFT token owners. The thing is that an owner may rent the validator NFT to someone, get the full payment upfront, and then exit the validator, which will impact future accrued fees.

Recommendation: Make sure that any attempt to call `exit` will revert while there is still an active renter (user).

Kiln: Fixed in [d05f25b2d809823d05db13b8a82aa825e0d04c09](#) by implementing auditor's recommendation.

Spearbit: Fixed.

5.2.5 A user/renter can block the vNFT owner from clearing or setting its address after the expiration time

Severity: High Risk

Context:

- [vNFT.sol#L327](#)
- [vNFT.sol#L356](#)
- [vNFT.sol#L426](#)
- [vFactory.sol#L806-L810](#)
- [MinimalRecipient.sol#L42-L49](#)
- [Factory.sol#L811-L813](#)
- [WithdrawRecipientLike.sol#L14](#)

Description: Once an owner rents its vNFT token to a user if `_getWithdrawableAmount(tokenId) > 0`, the user/renter can block the owner from clearing the usership or setting the user (even after the expiry of the token).

This is due to the fact that when the owner calls `clearUsership(tokenId)` or `setUser(...)` since there is an outstanding withdrawable amount left in this token `f.withdraw(validatorIds, recipient)` will be called with the user as the recipient. This in turn ends up [here](#):

```
(bool success, bytes memory rdata) = IMinimalRecipient(payable(withdrawalRecipient)).exec(
    recipient,
    abi.encodeWithSelector(WithdrawRecipientLike.withdrawCallback.selector, id, publicKey,
        ↪ recipientBalance),
    recipientBalance
);
```

The current `IMinimalRecipient`'s `exec(...)` implementation is as follows:

```
function exec(address target, bytes calldata cdata, uint256 value) external returns (bool, bytes
    ↪ memory) {
    LibSanitize.notZeroAddress(target);
    if (msg.sender != $owner) {
        revert LibErrors.Unauthorized(msg.sender, $owner);
    }
    // slither-disable-next-line low-level-calls
    return target.call{value: value}(cdata);
}
```

Note that in the above snippet `target` would be the renter/user of the vNFT token. So the user can decide to either revert or return successfully. In case of reverting the call, we would [end up](#):

```
if (!success) {
    revert MinimalRecipientExecutionError(rdata); // whole tx will be reverted
}
```

Therefore the user can decide whether the owner of vNFT can clear the user ship or set its user even after the expiration of the token.

Recommendation: Potential mitigation would be to implement a `WithdrawRecipientLike` contract that is used in the `vNFT._withdraw(...)` function that works as a transient middle contract where the withdrawal fees are sent to which then can be sent to the user or the owner of vNFT.

Kiln: Fixed in [PR 2](#).

Spearbit: Fixed.

5.2.6 vNFT does not enforce integrator commission fees for generated CL rewards

Severity: High Risk

Context:

- [vNFT.sol#L77-L89](#)

Description: `$operatorCommission`, `$integratorCommission` and `$integrator` are not used in the accounting logic of vNFT. When an owner/staker `purchase()` validators using the vNFT contract all the CL fees/rewards generated by these validators can be fully withdrawal by the owner/renters of these vNFT tokens. There is no logic included to take a portion of these fees and allocate them for the operator or the integrator.

Recommendation: If allocating commission fees is required for the integrator or the operator, the corresponding logic would need to be added to the vNFT contract. Otherwise it should be documented how and if the integrator/operator would acquire any commission if any from the CL rewards.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.2.7 vNFT - The freeze mechanism does not fully mitigate front-running related issues

Severity: High Risk

Context: [vNFT.sol#L36](#)

Description: The vNFT contract is tokenizing users' deposits into validators, i.e., users calling `purchase` deposit eth through the `vFactory` which is allocated to validators, and in return they are given an NFT in the vNFT contract to represent their ownership. This NFT is transferable and thus, can be traded on decentralized exchanges. Malicious actors can take advantage of the fact that the `tokenId` is not tied to changes in the token storage state. A classic example would be a malicious vNFT seller that can potentially front-run the actual dex swap transaction in the mem-pool, causing the buyer to purchase a depreciated token, but there could be many different attack vectors, some of which will be described later in this issue. The current version of the code implements the freezing mechanism to mitigate this specific issue. In short, the freezing mechanism disallows transfers of vNFT tokens for a certain amount of time after `setUser` and `withdraw` were called and for an infinite amount of time after `exit` was called. The idea behind this design is to revert the immediate swap transaction, giving the other party time to investigate the error and cancel the deal. In practice, however, this solution is far from fully mitigating all front-running related issues for a myriad of reasons, namely:

1. Many users won't be able to understand that the transaction failed due to `IllegalTransferWhileFrozen` since it is not supported in 3rd party decentralized exchanges and thus may cause them to try and buy it again after the freeze period ended, opening the possibility for an attack once the initial freezing period is over.

2. Some NFT decentralized exchanges implement auction methods and contracts that differ from the classic "seller as a maker, buyer as a taker", although less common, it is possible that the buyer would be the maker, signing a buy order off-chain that is supposed to be submitted later by the seller as part of a `swap` transaction. In case the expiration of this buy order is longer than the freeze period, the seller might first call `withdraw` or/and `setUser`, wait until the freezing period is over, and then submit the `swap` transaction, and by doing this, selling a depreciated token to the buyer.

Recommendation: As already mentioned, the issue is caused due to the fact that the `tokenId` is not tied to changes in the token storage state. We propose to implement a mechanism where there are two different identifiers for any token, an internal identifier that should stay constant throughout the entire token life-cycle (it can be implemented as a counter), and an external identifier. The internal identifier will be used to find the corresponding storage data attached to the token. Whenever the token storage data is changed (`setUser`, `withdraw`, `exit` for example), the old token should be burned, and a new token should be minted using a new external identifier. The external identifier might be implemented as `hash(internal_id, version)`, where the version is incremented whenever the token storage data is changed.

Kiln: Fixed in [d05f25b2d809823d05db13b8a82aa825e0d04c09](#) by implementing the auditor's recommendation for `setUser` and `withdraw`, and for `exit` implement a different mechanism that achieves the same result.

Spearbit: Fixed.

5.2.8 Nexus Admin can DoS all spawned pool's reporting process

Severity: High Risk

Context:

- [Nexus.sol#L324-L329](#)
- [vPool.sol#L396](#)
- [vOracleAggregator.sol#L206](#)

Description: The nexus admin can block all spawned pool's reporting process by setting any of the `genesisTimestamp`, `epochsUntilFinal`, `slotsPerEpoch` and `secondsPerSlot` values so high that the oracle members for the pools would not be able to submit reports with valid epochs.

Recommendation: Perhaps the provided epoch related parameters by Nexus should only be able to be used during the bootstrapping of the factory, pools, ... , and then later the factory's admin should be able to override those values.

Kiln: Fixed in the commit below by unlinking Nexus from `vPool`. The consensus layer spec will be provided to `vPool` by Nexus when it is initialised. But afterwards only the `vPool`'s `vFactory` admin can change the specs [3a9e9d8ddfcfb46b96932f387c1cbb9ed64acf42](#).

Spearbit: Fixed.

5.2.9 A malicious pool can DoS the staking process for `MultiPool20`

Severity: High Risk

Context:

- [MultiPool20.sol#L149](#)
- [MultiPool20.sol#L212](#)

Description: An added malicious pool can DoS the staking process for all users by making sure the calculation of `_ethAfterCommission(maliciousPoolId)` reverts when one is trying to calculate `_totalUnderlyingSupply()`. This works even the malicious pool is deactivated or has a routing weight 0. The admin does not have a functionality to remove a bad pool or exclude it from the calculations. It can only be corrected if an upgrade is made to the inherited contracts that are used as implementation of the proxies.

Recommendation: The admin should monitor this type of bad behaviour from the pools and in the case it happens it should have the ability to remove this bad pool from the system.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.2.10 Updating the routing weights of the MultiPool20 pools can have unexpected consequences for the stakers

Severity: High Risk

Context:

- [MultiPool20.sol#L129](#)
- [MultiPool20.sol#L50](#)

Description: The MultiPool20 admin can [change](#) the pool weights which can have unexpected consequences for the stakers. The worst case would be

1. the admin can frontrun the user's `stake(...)` call
2. add a malicious pool
3. reconfigure the pool weights so that all the funds would go to the malicious pool
4. all the user's staking amount will be lost / taken by the malicious pool

Recommendation: Possible solutions are

- not allowing the admin to change the pool weights
- having a time lock policy on how often the pool weights can be reconfigured

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.2.11 The vExitQueue's `_onTransfer` hook does not update the owner field in `$tickets.get()[id]`

Severity: High Risk

Context:

- [vExitQueue.sol#L382-L386](#)
- [vExitQueue.sol#L522-L523](#)
- [vExitQueue.sol#L573](#)

Description: The vExitQueue's `_onTransfer` hook does not update the owner for `$tickets.get()[id]`. So even after transferring the NFT ownership the exited funds will go to the original owner stored in `$tickets.get()[id]`.

Recommendation: Make sure the `_onTransfer` hook also updates the owner in the `$tickets.get()[id]` storage struct to the new owner. Also, it might be better to keep the record of ownership only in one spot.

Kiln: Fixed in the PR below by removing the owner field from the Ticket struct and using the ERC721 owner of the printed ticket [PR 63](#).

Spearbit: Fixed.

5.2.12 Malicious ticket seller might trick a buyer to buy a depreciated token

Severity: High Risk

Context:

- [vExitQueue.sol#L276](#)

Description: The `vExitQueue.onvPoolSharesReceived` function receives pool shares from the sender and mints an ERC721 token, representing ownership over an exit claim. These tokens are transferable and can be traded on decentralized exchanges. However, a malicious ticket seller can potentially front-run the actual dex swap transaction in the mem-pool, by calling `claim` right before, causing the buyer to purchase a depreciated token.

The full scenario:

1. pool shares holder sends its shares to the `vExitQueue` contract and gets a ticket token in return.
2. the ticket owner is now calling `approve` with that token id and an address of a dex (and if needed signs a maker order).
3. A potential buyer now calls the function that facilitates the atomic swap, the transaction is still in the mem-pool.
4. The malicious seller calls `claim` claiming 99% of the ticket value.
5. The atomic swap transaction succeeds, the buyer gets a practically useless token but the seller gets the whole payment.

This issue is relatively common in various protocols, but it is typically considered a medium risk since it is less likely that the NFT will be traded on a decentralized exchange. However, in this case, the tokenization's primary purpose is to enable a secondary market that provides instant liquidity to sellers, making the likelihood of buyers losing their funds high. Therefore, we have classified this issue as high risk.

Recommendation: The issue is caused due to the fact that the `tokenId` is not tied to changes in the token storage state. We propose implementing a mechanism where there are two different identifiers for any token, an internal identifier that should stay constant throughout the entire token life-cycle (it can be implemented as a counter), and an external identifier. Whenever the token storage data is changed (`claim` for example), the old token should be burned, and a new token should be minted using a new external identifier. The external identifier might be implemented as `hash(internal_id, version)`, where the version is incremented whenever the token storage data is changed.

Kiln: Fixed in [b23f94758ae22a476ff1b75939d83ade51bad32e](#) by implementing the auditor's comment with a slight variation where the external id is defined as `ticketIndex << 128 | uint256(t.size)` - a unique identifier, and the internal id is the index of the ticket inside `$tickets`, and the ticket nft is being burnt and re-minted upon claiming.

Spearbit: Fixed.

5.3 Medium Risk

5.3.1 MultiPool20 mints 1:1 for the initial stake() and does not use its pools' internal conversion rates.

Severity: Medium Risk

Context:

- [MultiPool20.sol#L134-L137](#)
- [MultiPool1155.sol#L121-L124](#)

Description: MultiPool20 mints 1:1 for the initial stake() and does not use its pools' internal conversion rates. This is in contrast to MultiPool1155 where the initial stake for each pool uses the pool's internal conversion rate:

```
// MultiPool20
$injectedEth.get()[id] += amount;
_mint(msg.sender, amount); // <--- 1:1 minting
emit Stake(msg.sender, amount, amount, id);
pool.deposit{value: amount}();

// MultiPool1155
$injectedEth.get()[id] += amount;
uint256 mintedPShares = pool.deposit{value: amount}();
_mint(msg.sender, mintedPShares, id); // <--- minting using `pool`'s internal conversion rate
emit Stake(msg.sender, amount, mintedPShares, id);
```

Also note upon next staking, both MultiPool20 and MultiPool1155 use the their corresponding MultiPool's internal conversion rate which when specialised to the initial stake roughly translates into the corresponding pool's internal conversion rate.

Depending on the selected initial pool to deposit into, with the current implementation there could be advantageous or disadvantageous to be the 1st staker.

Recommendation: It would be best to keep the initial stake for MultiPool20 consistent with the other cases:

```
// MultiPool20
$injectedEth.get()[id] += amount;
uint256 mintedPShares = pool.deposit{value: amount}();
_mint(msg.sender, mintedPShares);
emit Stake(msg.sender, amount, mintedPShares, id);
```

Kiln: Fixed in 1bbd98.

Spearbit: Fixed.

5.3.2 vFactory's admin/operator can collude with allowed depositors to avoid Nexus's protocol fee

Severity: Medium Risk

Context:

- [vFactory.sol#L293](#)
- [vFactory.sol#L306](#)
- [vFactory.sol#L357](#)
- [vFactory.sol#L382](#)
- [vFactory.sol#L550](#)
- [vFactory.sol#L582](#)

Description: The vFactory's admin/operator can allow depositors that use different set of withdrawal channels compared to the ones generated for the current implementation of the pool components. Then the admin can add and approve validators for these withdrawn channels.

The allowed depositors can implement their own set of rules on how to distribute the generated rewards and avoid paying protocol fees to the Nexus's global fee recipient.

If Nexus still has some sort of administrator privilege the hatcher that hatched this factory, it can change the implementation and also apply fixes to avoid this behaviour once detected, but if the hatcher is frozen changes are not possible.

Recommendation: If enforcing protocol fee payments for Nexus is important the factory's admin should not be able to set its own allowed depositors. Or potentially Nexus should be able to restrict the allowed withdrawal channels used through an authentication endpoint.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.3.3 Any allowed depositor can deposit for any withdrawal channel

Severity: Medium Risk

Context:

- [vFactory.sol#L382](#)
- `emit PurchasedValidators(purchasedValidatorsIds);`

Description: Any allowed depositor can deposit for any withdrawal channel. Specifically, an allowed depositor can deposit into the withdrawal channel that is uniquely associated to a factory's pool. The depositor would lose its deposited balance though since if exited the deposited amount will be sent to the corresponding withdrawal recipient

1. Doing so, this depositor can set its own execution layer fee recipient and extra data for the number of validators deposited.
2. Depending on how the off-chain oracle members keep track of the pool's accounting for the reporting purposes this might influence the numbers reported by the oracle members back to the pool.
3. Can DoS the `purchaseValidators()` endpoint by depositing enough validators using the pool's withdrawal channel so that the number of funded validators for this withdrawal channel reach its limit. This would only be a really costly grieving attack.

Recommendation: Off-chain oracle members for a pool need to listen to `PurchasedValidators(purchasedValidatorsIds)` when computing their reporting parameters and only do the required calculation for the specific purchased validator ids:

```
emit PurchasedValidators(purchasedValidatorsIds);
```

Also it would be best to implement an enforcement that would only allow assigned depositors to a withdrawal channel to deposit into that withdrawal channel. For example only the pool's associated to their withdrawal recipient can deposit into that specific withdrawal channel.

Related [PR 163](#).

Kiln: Resolved in the commit below by introducing a new mapping storage type that flags which depositors are connected to which withdrawal channels and restricting this endpoint using this new mapping:

- [c6035fae48ba919661ce323ae70a3475709a8782](#)

Event updated to emit withdrawal channel in:

- [7af6abbcd6dfb6fc509a103d790c68a02dfae57d](#)

Spearbit: Fixed.

5.3.4 `Liquid20A.requestExit` input parameter is not handled correctly

Severity: Medium Risk

Context:

- [Liquid20A.sol#L60-L63](#)

Description: `amount` is compared to `_balanceOf(msg.sender)` which has a unit of shares, but also `amount` is used as an underlying amount to calculate `sharesToExit`.

Recommendation: Either we need to

- compare `amount` with `_balanceOfUnderlying(msg.sender)` or
- `amount` is the `sharesToExit`.

To avoid confusion it might be also best to have two endpoints:

1. `requestExitShares`
2. `requestExitUnderlying`

Kiln: Amount is compared to the underlying balance [048576824f7fa3c63f65431becd6202c3090d53e](#).

Spearbit: Fixed.

5.3.5 Users and owners would need to trust the `$execLayerVault.get()`'s maintainer to distribute those fees fairly.

Severity: Medium Risk

Context:

- [vNFT.sol#L382](#)

Description: Once a user/staker purchase `vNFT` tokens and the corresponding validators get deposited the `$execLayerVault` is used as the EL fee recipient. The logic of this address is not concrete and is not implemented trustlessly on-chain.

In the codebase there is one implementation of a vault, `MerkleVault`, where all the accounting and fee allocations are calculated off-chain and then distributed using Merkle trees and the vault contract.

This opens up a centralisation risk that the user has no control over.

Recommendation: If there isn't a plan to include a different logic where all the accounting is performed on-chain, the above fact should be documented and also communicated with the users.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.3.6 `vExitQueue` does not favour user's potential exit amount when updating the ticket's `maxExitable` on the claim process.

Severity: Medium Risk

Context:

- [vExitQueue.sol#L554](#)

Description: `vExitQueue` does not favour user's potential exit amount when updating the ticket's `maxExitable` on the claim process.

The current strategy keeps the internal conversion rate of the ticket the same during the recursion. The difference with the other implementations for example from `LiquidCollective` (LC) strategy is that in LC we would increase the conversion rate in the event the current cash conversion rate is underperforming compared to the ticket so that

potentially during the next ticket/cask matching we can compensate for the underperformance if the next casks would overperform. The LC strategy would have been:

```
ticketArray[ticketId].maxExitable -= uint128(__.maxExitableAmount);
```

Note that this delta is exactly the amount added to `ethToPay`.

Recommendation: It maximising the return user's underlying amount is important it would be best to change the `maxExitable` update strategy to:

```
ticketArray[ticketId].maxExitable -= uint128(__.maxExitableAmount);
```

Kiln: Recommendation has been applied in [8d2f64f39b85cfcabe7aab1d2a736c7afe0e83e1](#).

Spearbit: Fixed.

5.3.7 `vFactory.addValidators` - Possible duplicate keys may lead to wrong allocation of deposits

Severity: Medium Risk

Context:

- [vFactory.sol#L306](#)

Description: `addValidators` does not revert for duplicated keys, i.e. the same key can be used in the same withdrawal channel or in different withdrawal channels more than once, which will lead to multiple deposits on a single key, which is definitely not desired. The issue is amplified because `addValidators` can be called by both the operator and the admin of the `vFactory` contract, which means it might be harder to de-duplicate the keys off-chain due to synchronization issues.

Recommendation: In general, it is highly recommended to implement an on-chain logic to disallow duplicated keys, however, as communicated with the team, this function is gas-sensitive and additional storage operations will be expensive. In this case, we advise at least having a dedicated script that will query the emitted events (with a significant deliberate delay to minimize transaction propagation and inclusion delay issues) and will filter duplications off-chain before calling `addValidators` both for the operator and for the admin, as well as another monitoring script to find duplicated keys and call `removeValidators` hopefully before they are being funded.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.3.8 `vTreasury.withdraw` - In edge cases attackers might be able to lock ERC20 tokens from being withdrawn from the contract

Severity: Medium Risk

Context:

- [vTreasury.sol#L320-L329](#)

Description: The `vTreasury` contract may hold ERC20 tokens that should be withdrawable to the `$operator` address when `withdraw` is called. However, with the current implementation of the `_balance` function, it is possible (in rare scenarios) to "masquerade" the ERC20 token as a `vPool` token and by doing this, to manipulate the `vTreasury` contract to use a lower balance while fulfilling the `withdraw` transaction, effectively locking the ERC20 tokens inside the contract. In order to launch this attack, the ERC20 contract that's in use should have a function callable by anyone that's allowing a random call to any other contract and specifically to `onvPoolSharesReceived(address, address, uint256, bytes)`, the token contract should also implement `totalUnderlyingSupply()` and `totalSupply()` preferably always return 0 for `totalUnderlyingSupply()`. Now to launch the attack, the attacker will use this function to call `vTreasury.onvPoolSharesReceived` inserting his transaction right before a call to `withdraw` in the mem-pool, specifying a tiny amount parameter. By doing this, the call to `withdraw` will use the specified tiny amount instead of the real ERC20 token balance to be sent to the `$operator`. The impact of the attack is medium/high, but the likelihood is low, that's why we decided to label it as medium risk.

Recommendation: Consider changing the implementation of `_balance` to `user.balanceOf` for both `vPool` and `ERC20` cases, and leave the ether handler as it is now.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.3.9 `MultiPool20._requestExit` - Changes in `$poolRoutingList` and `$poolMap` may cause a temporary denial of service for exit requests

Severity: Medium Risk

Context:

- [MultiPool20.sol#L69](#)

Description: When users stake their eth into a `MultiPool` contract, funds are distributed according to the current state of `$poolRoutingList`. Both the pools and their respective percentages can be changed by the admin at any moment by calling `addPool`, `setPoolPercentages` respectively. Changes in these data structures may cause a temporary denial of service for exit requests in certain cases. To better illustrate the issue, let's consider the following example:

1. The `MultiPool` contract is initialized with a single pool denoted as `poolA` that receives 100% of deposits.
2. At some point, Alice deposits 100 eth to the `MultiPool` contract, receiving $100 \cdot 10^{18}$ multi-pool shares that are backed only by `poolA` shares. For simplicity, we will assume that shares' prices were not changed throughout the entire example.
3. Later, the admin is adding 3 more pools, making it 4 pools in total, with 25% percentage for each.
4. Now Alice is willing to cash out her position, she is calling `requestExit` with `amount=100*1018`. Assuming there are no deposits to the newly introduced pools, the transaction will fail since `$poolShares` is 0 for the newly introduced pools and Alice won't be able to request an exit (even not partially) for her position.

It is important to mention that Alice's funds are not locked forever, as they might be claimed in case enough liquidity is deposited into the new pools, or in case the admin has changed the `$poolRoutingList` array back to support Alice's request. The latter might be an issue since in certain scenarios it may cause a temporary denial of service for other users' requests.

Recommendation: We propose an alternative algorithm to handle the allocation of deposits/exits to/from pools, the algorithm is incremental and will be explained with a thorough example, there might be some edge cases that need to be figured.

1. First deposit of 100 eth by a single depositor, 2 pools a,b

Eth before = 0 eth Eth after = 100 eth Pools real balance in eth = [0,0] Pools current percentages = [0%, 0%] Pools desired percentages = [20%, 80%]

$\text{toAdd}(a) = 20\% \cdot 100 - 0\% \cdot 0 = 20 \text{ eth}$ $\text{toAdd}(b) = 80\% \cdot 100 - 0\% \cdot 0 = 80 \text{ eth}$

After the transaction Pools real balance in eth = [20,80] Pools current percentages = [20%, 80%] Pools desired percentages = [20%, 80%]

Now 2 pools are added (c,d) and the percentages are changed to [35%, 5%, 32%, 28%]

2. The single depositor is withdrawing 70 eth

eth before = 100 eth Eth after = $100 - 70 = 30$ eth Pools real balance in eth = [20, 80, 0, 0] Pools current percentages = [20%, 80%] Pools desired percentages = [35%, 5%, 32%, 28%]

$\text{toWithdraw}(a) = 20\% \cdot 100 - 35\% \cdot 30 = 9.5$ $\text{toWithdraw}(b) = 80\% \cdot 100 - 5\% \cdot 30 = 78.5$ $\text{toWithdraw}(c) = 0\% \cdot 100 - 32\% \cdot 30 = -9.6$ can't withdraw $\text{toWithdraw}(d) = 0\% \cdot 100 - 28\% \cdot 30 = -8.4$ can't withdraw

We sum all the negatives divide by the positives count and add to the rest $\text{toWithdraw}(a) = 9.5 - (18/2) = 0.5$ $\text{toWithdraw}(b) = 78.5 - (18/2) = 69.5$

After the transaction Pools real balance in eth = [19.5, 10.5, 0, 0] Pools current percentages = [65%, 35%, 0%, 0%] Pools desired percentages = [35%, 5%, 32%, 28%]

3. Another deposit of 15 eth

Eth before = 30 eth Eth after = 30 + 15 = 45 eth Pools real balance in eth = [19.5, 10.5, 0, 0] Pools current percentages = [65%, 35%, 0%, 0%] Pools desired percentages = [35%, 5%, 32%, 28%]

toAdd(a) = 35% * 45 - 65% * 30 = -3.75 can't add toAdd(b) = 5% * 45 - 35% * 30 = -8.25 can't add toAdd(c) = 32% * 45 - 0% * 0 = 14.4 toAdd(d) = 28% * 45 - 0% * 0 = 12.6

We sum all the negatives divide by the positives count and add to the rest toAdd(c) = 14.4 - 12/6 = 8.4 toAdd(d) = 12.6 - 12/6 = 6.6

After the transaction Pools real balance in eth = [19.5, 10.5, 8.4, 6.6] Pools current percentages = [43.3%, 23.3%, 18.67%, 14.67%] Pools desired percentages = [35%, 5%, 32%, 28%]

4. Another exit request of 20 eth

Eth before = 45 Eth after = 25 Pools real balance in eth = [19.5, 10.5, 8.4, 6.6] Pools current percentages = [43.3%, 23.3%, 18.67%, 14.67%] Pools desired percentages = [35%, 5%, 32%, 28%]

toWithdraw(a) = 43.3% * 45 - 35% * 25 = 10.735 toWithdraw(b) = 23.3% * 45 - 5% * 25 = 9.235 toWithdraw(c) = 18.67% * 45 - 32% * 25 = 0.4015 toWithdraw(d) = 14.67% * 45 - 28% * 25 = -0.3985 can't withdraw

We sum all the negatives divide by the positives count and add to the rest toWithdraw(a) = 10.735 - 0.1328 = 10.6022 toWithdraw(b) = 9.235 - 0.1328 = 9.1022 toWithdraw(c) = 0.4015 - 0.1328 = 0.2687

After the transaction Pools real balance in eth = [8.8978, 1.3978, 7.9985, 6.6] Pools current percentages = [35.7%, 5.6%, 32.1%, 26.5%] Pools desired percentages = [35%, 5%, 32%, 28%]

As we can see, users were able to withdraw large amounts, and after a few steps, the pools are balanced.

Kiln: Thanks for this issue.

After analyzing this algorithm we decided to not implement due to some edge cases and more importantly this would negatively impact the UX, particularly for smaller amounts as this algorithm does not optimize to limit the number of tickets minted, which increases the gas costs of claiming.

We instead chose to allow the most imbalanced pool to go below the expected amount up to a certain threshold that can be set by the admin. As TVL increases bigger exits won't impact the balance as much meaning we can increase the threshold.

Spearbit: Acknowledged.

5.3.10 Staking to MultiPool will revert if there is a disabled pool

Severity: Medium Risk

Context:

- [MultiPool20.sol#L131](#)
- [MultiPool1155.sol#L97](#)

Description: Assume that there are 5 pools. As long as there is a pool that is disabled, no one would be able to stake to the Multipool because the `_checkPoolIsEnabled` function will revert the entire transaction within the `_stake` function.

For the MultiPool20 context, this applies to the pools with positive routing `split` weight.

Recommendation: Consider implementing measures to handle the deactivation of one or more pools so that users can continue to stake assets to other pools that are still active within the MultiPool.

Kiln: For MultiPool20 the commit below ensures that deactivated pools would have 0 weights [f7ecefdb68052636830df741cf7f97fd60f7e495](#).

For MultiPool1155 [see comment](#).

Spearbit: Fixed.

5.3.11 The Nexus global fee recipient can DoS the vTreasury during the fee distribution or token withdrawal process

Severity: Medium Risk

Context:

- [vTreasury.sol#L160-L162](#)
- [vTreasury.sol#L235-L237](#)

Description: As callbacks are made during token transfers to the `to` address, the Nexus global fee recipient can DoS the vTreasury during the fee distribution or token withdrawal process. This can also lead to DoS of report submission by the oracle aggregator to the pool if enough rewards are going to be sent to the treasury to be distributed.

Recommendation: One solution would be to have storage parameter for different token commissions for the Nexus's global fee recipient which the recipient can pull from the treasury using endpoints only callable by this entity.

And in the context of this issue one would just set those parameters and not transfer the tokens to the global recipient, but let recipient to call using a different transaction to pull its commission.

Kiln: Fixed in the commit below by just updating the storage parameters and avoiding the exit and funding flow in vTreasury. [onvPoolSharesReceived 5b0fd23d1d1f3730f4c6c8a896c457132df61f95](#).

Spearbit: Fixed.

5.4 Low Risk

5.4.1 vExitQueue is missing to analyse some cases

Severity: Low Risk

Context:

- [vExitQueue.sol#L133-L137](#)

Description: The `if/else` block in this context does not consider the case where `c.position < t.position < c.position + c.size`.

Recommendation: Make sure the missing set of values are also analyzed.

Kiln: Fixed in [70e349](#).

Spearbit: Fixed.

5.4.2 Anyone can easily send funds to MinimalRecipient

Severity: Low Risk

Context:

- [MinimalRecipient.sol#L52-L55](#)

Description: `IMinimalRecipient` is used as implementation of clone proxies which are used as CL withdrawal recipients of vFactory for validators associated with withdrawal channel 0.

The current implementation of `IMinimalRecipient`, `MinimalRecipient` allows anyone to send funds to it directly using the `receive` or `fallback` endpoint. So one can send funds to `MinimalRecipient` by:

- calling `receive` or `fallback` endpoint (it is instant and can be atomic)
- self-destructing a contract (this attack vector might not be available in the future)

- assign a CL's validator to this address, this would require timing the attack and would be costly

Recommendation: To remove potential unexpected side effects introduced by depositing directly to `Minimal-Recipient` it would be best to remove the `receive` and `fallback` endpoints or have them revert with a custom message for off-chain tools.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.4.3 MultiPool can receive shares for deactivated pools, pools with zero routing weights and shares transferred from 3rd parties

Severity: Low Risk

Context:

- [MultiPool.sol#L168-L173](#)
- [MultiPool.sol#L180-L190](#)
- [MultiPool20.sol#L137](#)
- [MultiPool20.sol#L156](#)
- [MultiPool1155.sol#L114](#)
- [MultiPool1155.sol#L122](#)
- [vPool.sol#L1276](#)
- [vPool.sol#L1210](#)
- [vPool.sol#L478-L483](#)
- [vPool.sol#L1344](#)

Description: MultiPool has a `onvPoolSharesReceived(...)` endpoint that is called by `vPool` proxies when an amount is transferred to the MultiPool by the pool. We need to make sure that this endpoint is only called when a user stakes into the MultiPool (`Native20`, `Native1155` or their child contracts). Currently the implementation only calls `_findPoolIdOrRevert(pool)` which only checks whether the pool at some point had been added to the `$poolMap`. It is missing the following checks:

- whether the pool is currently active
- whether the pool has a positive `$poolRoutingList.toUintA()[poolId]` for MultiPool20
- whether the funds came from a minting process by the MultiPool calling the `vPool`'s deposit endpoint or whether the funds were transferred by a 3rd party

Note the above checks are important as in this callback, we keep track of the `$poolShares.get()[poolId]` internally.

Recommendation: Modify the implementation of the `onvPoolSharesReceived(...)` to check:

- the pool is currently active
- for MultiPool20 the pool has a positive routing weight
- Make sure `onvPoolSharesReceived(operator, from, amount, data)` reads the operator address and matches it with the pool itself and the `from` address to `address(0)`. This would guarantee that the callback was from a minting process which would mean either the original call to the pool was from the MultiPool to deposit funds into the pool or `_mint(sharesToMint, operatorTreasury)` and that would only happen if the pool's factory's treasury is the MultiPool (which currently is not the case unless one is not using the current implementations of `vsuite` or the contracts are fixed or upgraded by their respective hatcher admins).

Kiln: Fixed in [f7ecefdb68052636830df741cf7f97fd60f7e495](#) and [cda494cbb022d7bace11f90245578355936690c0](#)

Spearbit: Fixed.

5.4.4 Any allowed depositor can request more validators for any withdrawal channel

Severity: Low Risk

Context:

- [vFactory.sol#L299](#)
- [vPool.sol#L402](#)
- [Nexus.sol#L284](#)
- [lvFactory.sol#L156](#)

Description: Any allowed depositor can request more validators for any withdrawal channel:

```
function request(bytes32 wc, uint256 amount) external onlyDepositorOperatorOrAdmin {
    ctypes.WithdrawalChannel storage withdrawalChannels = $withdrawalChannels.get()[wc];

    emit ValidatorRequest(wc, amount + LibUint256.max(withdrawalChannels.limit,
        ↪ withdrawalChannels.funded));
}
```

Off-chain agents need to listen to `ValidatorRequest` and setup more validators for the specified `wc`. Note that we have 3 types of allowed depositors:

1. spawned pool's by the factory's admin which have their own unique withdrawal recipient and thus unique withdrawal channel
2. pluggable hatched [cubs](#) by the factory's admin through the `Nexus` interface
3. general allowed depositors [set](#) by the factory's admin or a general hatcher registry.

Allowed depositors from different categories should not be able to trigger changes in on or off-chain parameters related to the other categories. For example a general allowed depositor should not request more validators for a withdrawal channel that is used by a pool.

Recommendation: Either we can introduce a filtering for off-chain agents so that they can validate the above assumption by introducing an extra parameter to the:

```
event ValidatorRequest(bytes32 indexed withdrawalChannel, address caller, uint256 total);
```

Or have an on-chain solution where the request to add more validator for non-zero withdrawal channels can only be made by assigned allowed depositors. For example, only the pool can request validators for its unique withdrawal channel.

Kiln: Resolved in the commit below by introducing a new mapping storage type that flags which depositors are connected to which withdrawal channels and restricting this endpoint using this new mapping [c6035fae48ba919661ce323ae70a3475709a8782](#)

Spearbit: Fixed.

5.4.5 Anyone can frontrun `Hatcher.deleteGlobalFix(...)` to force apply a fix to a cub if certain conditions are present

Severity: Low Risk

Context:

- [Hatcher.sol#L233-L236](#)
- [Cub.sol#L94](#)

Description: Anyone can frontrun the hatcher admin's call to `Hatcher.deleteGlobalFix(...)` to force apply a fix to a cub (or multiple cubs) if the cub in the question:

- is `Fixable`, the actor can call the cub's `fix()` endpoint which would force apply all the pending registered global fixes. `Fixable` is used for `vCoverageRecipient`, `vExecLayerRecipient`, `vExitQueue`, `vFactory`, `vOracleAggregator`, `vPool`, `vTreasury`, `vWithdrawalRecipient`, which are implementations for the relevant cubs.
- Or in general if the cub's implementation has a public or external endpoints which is either `view` or `pure`, then anyone can call that specific endpoint to force apply all the pending global fixes for this cub. The endpoint could also be a `receive` or `empty` fallback.

Recommendation: The above scenario should be documented. If the fixes are required to be applied only when a privileged user makes a non-static call, this should be enforced as currently the Hatcher's admin cannot prevent a global fix to be not applied before calling the `Hatcher.deleteGlobalFix(...)` endpoint.

Kiln: Fixed in [PR 11](#).

Spearbit: Fixed.

5.4.6 Add check to prevent minting zero shares

Severity: Low Risk

Context:

- [MultiPool20.sol#L126-L169](#)

Description: In the `_stake` function of `MultiPool20`, there's a potential risk where users could deposit some assets but receive no shares in return due to potential rounding errors. This issue could be problematic as users would not receive any representation of their deposited assets. This check is commonly implemented in contracts with share minting capability, such as `Solmate`, to ensure that users always receive shares proportional to their deposits. Reference [ERC4626.sol#L48](#)

Recommendation: It is recommended to add a check that prevents the minting of zero shares. This can be done by adding the following lines of code after each share minting operation.

```
+ require(sharesAcquired != 0, "Zero share");
```

Kiln: Fixed in [0713601d454c304edd2968181b525a5a157e9676](#).

Spearbit: Verified.

5.4.7 LibErrors.Unauthorized's second parameter might not be correct

Severity: Low Risk

Context:

- [vNFT.sol#L457-L459](#)
- [vNFT.sol#L343-L345](#)
- [vNFT.sol#L311-L313](#)

Description/Recommendation: LibErrors.Unauthorized's second parameter in `_solveIdsAndFreeze` might not be correct if `ownerOnly == false` and `_beneficiaryOf` is the user. We should instead have:

```
address expectedCaller = ownerOnly ? _ownerOf(tokenIds[idx]) : _beneficiaryOf(tokenIds[idx]);
if (msg.sender != expectedCaller) {
    revert LibErrors.Unauthorized(msg.sender, expectedCaller);
}
```

and for the `clearUsership` (or `setUser` when checking for `_isApprovedOrOwner(msg.sender, tokenId)`) case might be ok to keep the second parameter as is, but potentially there are 3 possible allowed users that can call this endpoint.

Kiln: Fixed in [7f5a1a6edb732b5174d11da8924fc0564f97c97e](#).

Spearbit: Fixed.

5.4.8 A to be updated global oracle member can frontrun its address change to still cast votes for all the oracle aggregators

Severity: Low Risk

Context:

- [vOracleAggregator.sol#L200](#)

Description: The global oracle member can frontrun its `update` call by the nexus admin to all the oracle aggregators to cast a vote before it gets replaced.

Recommendation: This needs monitoring. Also factory admin's should have the option to disallow global oracle members to cast vote for their pool's oracle aggregators. Since after all the required cubs have been hatched, the factory's admin should have total control over all operations.

Kiln: We can also add a toggle in the oracle aggregator to disable the global oracle member from participating:

- to disable, i would suggest that a minimum number of non global member would be 5
- if number of members goes under 5 after toggle is on, then toggle is ignored and global member can participate again.

This preserves the initial idea that the global member is here for bootstrapping purposes and it gives the choice to the operator once the aggregator has enough members.

Spearbit: It would be better than the current setup since it would allow the factory admin to almost eject (/ have more power over) its contract and the related components from the Nexus system.

Fixed in the commit below by allowing the oracle admin to eject the global oracle member from the report process when there are at least 5 non-global oracle members present [4ce50cc4f1cbc65ea9d54827831c320fa1558c60](#) .

5.4.9 Manually clean the dirty bits of an address

Severity: Low Risk

Context:

- [vExitQueue.sol#L243-L247](#)

Description: `mload(add(data, 20))` includes dirty bytes from the `length` portion of the data, we are relying on the compiler to clean it up at some point after this assignment.

```
// 00 ... 00 [LENGTH] [20 BYTE ADDR]
```

most likely cleared [here](#).

Recommendation: It would be best to manually clean the `to` address to avoid potential future pitfalls.

Kiln: Fixed in [00e556ccd8bb277f881dd09bdeb77edaed97d317](#).

Spearbit: Fixed.

5.4.10 `vNFT.setUser` - approved/owner of a `vNFT` may set a user for a period ending now or ended in the past

Severity: Low Risk

Context:

- [vNFT.sol#L298](#)

Description: The `vNFT` contract implements `setUser` as part of the `IERC4907` rentable NFT ERC. However, in the current implementation, it is possible for a `vNFT` owner/approved user to set a user to a period that has already expired or is expiring at the current `block.timestamp`, and by doing this may trick users to pay the full price for a fraudulent rental transaction.

Recommendation: Consider validating at the least that `expires > block.timestamp` or even require a minimum duration til expiration.

Kiln: Fixed in [d05f25b2d809823d05db13b8a82aa825e0d04c09](#) by implementing auditor's recommendation.

Spearbit: Fixed.

5.4.11 Epoch validation check is not accurate

Severity: Low Risk

Context:

- [vPool.sol#L1148](#)
- [vPool.sol#L1164](#)

Description: As part of the epoch [validation](#) we have:

```
!((block.timestamp < finalizedTimestamp) || (epoch > expectedEpoch && !_isFrameFirstEpochId(epoch)))
```

or below in `_onlyValidEpoch`:

```
if (epoch > expectedEpoch && !_isFrameFirstEpochId(epoch)) {  
    revert EpochNotFrameFirst(epoch);  
}
```

These checks are no accurate when one is considering frame boundaries.

Recommendation: The below check might be more accurate:

```
return !(block.timestamp < finalizedTimestamp) && _isFrameFirstEpochId(epoch);
```

Note if the \$epochsPerFrame stays the same for the lifetime of the pool, then both expressions are equivalent as the expectedEpoch would always fall on frame boundaries.

But if for some reason the pool's factory admin decides to update \$epochsPerFrame then due to potential misalignment of the previous and current epoch frame grids the expectedEpoch might not fall on a frame boundary and we might need the epoch to be slightly shifted to the right (bigger value) so that it is more than expectedEpoch but still falls on a new frame boundary grid.

Not that current implementation does not ensure that epoch falls on a frame boundary when epoch == expectedEpoch and block.timestamp >= finalizedTimestamp.

Additional solution would be to have the accepted epoch grid piece-wise adaptive, but that would require to also record at what timestamp the \$epochsPerFrame storage value was updated.

And for `_onlyValidEpoch` better to have:

```
if (!_isFrameFirstEpochId(epoch)) {  
    revert EpochNotFrameFirst(epoch);  
}
```

Kiln: Fixed in [d2112897b00b2efced52de25f26319ffb77b778e](#).

Spearbit: Fixed.

5.4.12 Pluggable Hatcher admins and the Nexus would need to have a trustful connection

Severity: Low Risk

Context:

- [PluggableHatcher.sol#L41-L53](#)

Description: The current design create a fork in the responsibilities between the Nexus admin and the pluggable hatcher admins.

Recommendation: A different approach would be to set the `_nexus` as the admin and expose the required endpoints of the PluggableHatcher through wrapper endpoints in the Nexus contract. This would consolidate all the admin roles into one central point and remove the trust necessary between the Nexus admin and all the different PluggableHatcher admins.

Kiln: This is intended behavior. We need very strong quorums that we're currently setting up as PluggableHatcher.admin as they're in charge of upgrading and fixing all the instances. In the other hand, the Nexus is the entryptpoint of integrators and operators into the system and we can morph it to suite our deployment needs. Also, in this audit, we're planning on making the Nexus non-intrusive in all the different operator setups:

- the global consensus layer specs will be copied to the vPools upon construction.
- the global recipient will not be invoked automatically during oracle reports (manually only to exit vPool shares).
- the global oracle will be ejectable upon certain conditions (an OracleAggregator would need 5 regular members before it can eject the global oracle member).

Knowing this, it's ok imo that the PluggableHatcher admin and Nexus admin are two different quorums.

Spearbit: Acknowledged.

5.4.13 core pluggable hachers nexus parameters are not checked

Severity: Low Risk

Context:

- [Nexus.sol#L84](#)

Description: These core pluggable hachers would need to have set their \$nexus storage variable.

Recommendation: It would be great to check that those \$nexus addresses match with the current contract address during the initialisation of the Nexus contract.

Kiln: Fixed in [c41973dd902be80b85db2721ab88b4c259293f94](#).

Spearbit: Fixed.

5.4.14 Balanced notion in MultiPool20

Severity: Low Risk

Context:

- [MultiPool20.sol#L87-L89](#)

Description: When requesting an exit in a MultiPool20 and when there are more than 1 pool present and the amount requested to be exited is less than 1% of the TVL_stakedEthValue(id) is compared with the expectedValue for the pool using the provided routing weights. On the other hand one could have used [_ethAfterCommission\(id\)](#) which does not take the integration commission owed amount into consideration.

We have

$$\dot{B}_j = B_j - o_j$$

$$B_u^M = \sum_j \dot{B}_j$$

parameter	description
j	poolId
B_j	_stakedEthValue(poolId)
o_j	_integratorCommissionOwed(poolId)
$\dot{B}_j$	_ethAfterCommission(poolId)
B_u^M	totalUnderlyingSupply
w_j	splits[poolId]

Not considering the division errors (when multiplying the routing weights) if we use $\dot{B}_j$ to calculate the imbalance, having no imbalance would allow for pools to have integration commission owed amount (o_j) to be non-zero.

But with the current algorithm a balanced multi pool is only achieved when no pool has an owed amount:

$$\begin{aligned} B_u^M &= \sum_j (B_j - o_j) \sim \sum_j (w_j B_u^M - o_j) = B_u^M \sum_j w_j - \sum_j o_j = B_u^M - \sum_j o_j \\ &\Rightarrow \sum_j o_j = 0 \Rightarrow \forall j o_j = 0 \end{aligned}$$

A balanced multi pool where `_ethAfterCommission == expectedValue` can be a unbalanced one with the current implementation this is because a balanced pool with according to the alternative implementation can have some owed amount which can allow this algorithm to have the pool picked for exiting the user stakes an turn it into an unbalanced one in there alternative sense.

Recommendation: If in a balanced system one can allow non-zero owed amount for the integrator, then `_ethAfterCommission(id)` should be used when selecting an imbalanced pool.

Kiln: Recommendation has been applied in [13504fb4520d634e7a0ee7ee2cc87a7e36ea6ce4](#).

Spearbit: Fixed.

5.4.15 `_transferFrom` does not check whether the allowance is `type(uint256).max`

Severity: Low Risk

Context:

- [Liquid20A.sol#L80](#)
- [MultiPool20.sol#L256](#)

Description: In the context above when during the token transfer in `_transferFrom` we do not check whether the `currentAllowance` is `type(uint256).max` which in most implementation of (IERC20 like) token transfer correspond to infinite allowance. So in such a case the allowance of the spender is still deducted by the amount to be transferred.

Recommendation: If it is desired to keep the implementation similar to other standard token implementations, it would be best to compare allowance with `type(uint256).max` and if equal skip reducing the allowance.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.4.16 `vFactory._approve` - admin may set a limit greater than `WithdrawalChannel.validators.length`

Severity: Low Risk

Context:

- [vFactory.sol#L673](#)

Description: The `vFactory` admin may specify a new limit that is greater than the validators count, which may cause the `deposit` function to go OOB for certain transactions in the line of:

```
__v = withdrawalChannelStruct.validators[withdrawalChannelStruct.funded + idx];
```

And in addition, may lead to a wrong value returned by `withdrawalChannel.availableValidators` and emitted in the `ValidatorRequest` event in `request`. **Recommendation:** Consider adding a validation check to the `_approve` function that makes sure `limit <= withdrawalChannelStruct.validators.length`.

Kiln: Fixed in [9a041b12364ef17091102335417c0f5ed26da0fb](#) by implementing the auditor's recommendation.

Spearbit: Fixed.

5.4.17 `vFactory.removeValidators` - Potential sync issues may result in the removal of more than one validator

Severity: Low Risk

Context:

- [vFactory.sol#L333](#)

Description: `removeValidators` receives an ordered array of indexes and removes the corresponding validators from the contract storage. Since it is based on indexes and not on the underlying addresses, it is possible that two different validators will be removed instead of one in case the function was called twice. The function is callable by both the operator and the admin of the contract which may increase the likelihood of a duplicated removal due to potential sync issues between the different actors.

Recommendation: In general, it is highly recommended to change the `removeValidators` implementation to be based on an array of validators keys' instead of indexes and to add a mapping to map between a key and its corresponding index. However, as communicated with the team, this function is gas-sensitive and additional storage operations will be expensive. In this case, we advise at least having a dedicated script that will query the `RemovedValidator` past events (with a significant deliberate delay to minimize transaction propagation and inclusion delay issues) and will filter duplicated removal requests off-chain before calling `removeValidators` both for the operator and for the admin, as well as another monitoring script to ensure that all desired keys are indeed part of the contract storage, and if not, add the missing ones.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.4.18 `MultiPool20`'s conversion rate decreases upon a user's exit

Severity: Low Risk

Context:

- [MultiPool20.sol#L120](#)

Description: The effect of exiting shares on the multi pool's conversion rate is:

$$\frac{B_u^M}{S_m} \xrightarrow{\sim} \frac{B_u^M S_m - s - \sum_i f_i s_i \frac{B'_i / S'_i}{B_u^M / S_M} (w_i > 0)}{S_m - s}$$

So the multi pool's conversion rate decreases which benefits those who exit earlier (for example if there were multiple exits in the same block). This might cause stakers to race and try to frontrun one another to get the best conversion rate.

Recommendation: More documentation would need to be added around the parameter updates when a user requests an exit.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.4.19 Some endpoints reads from storage but it are declared as pure

Severity: Low Risk

Context:

- [FeeDispatcher.sol#L81-L83](#)
- [vPool.sol#L335-L337](#)
- [vPool.sol#L339-L342](#)
- [Nexus.sol#L165-L167](#)
- [Nexus.sol#L185-L187](#)
- [vFactory.sol#L265](#)
- [vOracleAggregator.sol#L127](#)
- [Hatcher.sol#L153](#)
- [ViewPureChecker.cpp](#)

Description: solc's [ViewPureChecker](#) is not bullet proof when it comes to storage pointers and there might be some misses. As is the case for [getCurrentSplit\(\)](#) (and other endpoints in this context) which reads from storage but is defined as pure.

related:

- [PR 11573](#)

Slightly related this endpoint is not defined in `IFeeDispatcher`

Recommendation: Make sure for the endpoints in this context the mutability is changed to `view`.

Kiln: Spearbit:

Fixed	context	commit
	FeeDispatcher.sol#L81-L83	585c1dd73b65e1fc4036b7a011b309c0df718314
	vPool.sol#L335-L337	43470125a138647308a1e0cd9d189c0f0233b47b only comments added
	vPool.sol#L339-L342	43470125a138647308a1e0cd9d189c0f0233b47b only comments added
	Nexus.sol#L165-L167	0d47e7a217c6a609bd75ab07a2a44b58510a4d2e only comments added
	Nexus.sol#L185-L187	0d47e7a217c6a609bd75ab07a2a44b58510a4d2e only comments added
	vFactory.sol#L265	605011108848e33c97ef1ec7af02e683ca4e142c only comments added
	vOracleAggregator.sol#L127	130b025965c529fe36bc863944565d7bbb231734 only comments added
	Hatcher.sol#L153	130b025965c529fe36bc863944565d7bbb231734 only comments added

5.4.20 `FeeDispatcher._setFeeSplit` - Recipients array might contain duplications

Severity: Low Risk

Context:

- [FeeDispatcher.sol#L89](#)

Description: Although it is an admin-only function, there is still the possibility of duplicates in the `recipients` array which will result in an unexpected larger share assigned to the duplicated address.

Recommendation: Consider validating that the array is sorted, and by this ensure that there are no duplications in the `recipients` array.

Kiln: Kiln acknowledges this issue and provides the following context on this decision:

- This would be bad admin behavior
- If recipients are set incorrectly nothing prevents the admin from changing it again

Spearbit: Acknowledged.

5.4.21 Dust ETH accumulate on `MultiPool20` contract

Severity: Low Risk

Context:

- [MultiPool20.sol#L130](#)

Description: Some dust amount of ETH will accumulate on the `MultiPool20` contract for every transaction depending on the `totalAmount` and the splitting configuration.

Assume that two pools are configured to split the amount equally (50% = 5000). If someone deposits an odd number of ETH (e.g. 11 wei), then based on the computation:

$$(11 * 5000) / 10000 = 5$$

There will be 1 wei of dust ETH remaining on the contract at the end of the transaction.

Recommendation: Consider sweeping the dust ETH into the last "pool" to prevent dust ETH from accumulating on the contract.

Kiln: Additional logic would cost more in gas than the dust left in the contract. The dust will be picked up when the commission is withdrawn since it also accumulated on the contract.

Spearbit: Acknowledged.

5.4.22 Use `safeTransfer()` instead of `transfer()` for ERC20 transfers.

Severity: Low Risk

Context:

- [vTreasury.sol#L290](#)

Description: Some ERC20 implementations do not implement a return value such as BNB. This will cause the token to always revert when trying to `_transfer`.

Recommendation: It is recommended to always use `safeTransfer` when transferring arbitrary ERC20s. More about on [OpenZeppelin SafeERC20:safeTransfer](#)

Kiln: Fixed in [a9a6a45b83ba41d98a0bb139eef51fabeb45a607](#) by implementing auditor's recommendation.

Spearbit: Fixed.

5.4.23 MultiPool20._setPoolPercentages - Potential ordering dependence issues

Severity: Low Risk

Context:

- [MultiPool20.sol#L171](#)

Description: The `_setPoolPercentages` function receives an array of percentages and applies it to the contract storage as is so that pools should match with the corresponding percentages based on an index matching, i.e. pool in index 0 will use the percentages array in index 0, etc. This design leaves the possibility for unexpected ordering dependence issues that will end up in a mismatch between the admin's intentions and the underlying state of the contract after the call to `setPoolPercentages`. To better illustrate the issue let's consider a simple example: For simplicity, let's assume both `$poolRoutingList` and `$poolMap` are empty.

1. Admin is calling `addPool` for pool A in tx1.
2. Right after that but before tx1 was included in a block, the admin is calling `addPool` for pool B in tx2.
3. Admin is calling `setPoolPercentages` allocating the 0 index for pool A and the 1st index for pool B in tx3.
4. The final order execution in the blockchain is tx2,tx1,tx3 which means pool B will get the percentage meant for pool A and vice versa.

Recommendation: There are multiple ways to solve this issue, as it is not complex, one of them might be to maintain another mapping that maps from the pool's address to the pool's index and then change the `_setPoolPercentages` implementation to be based on the pool's address and not the index.

Kiln: Admins should be aware of this possibility and either do those actions atomically or ensure the transactions were included before sending the next one

Spearbit: Acknowledged.

5.4.24 MultiPool.changeFee - The new fee should not exceed a maximum value which should be less than 100%

Severity: Low Risk

Context:

- [MultiPool.sol#L111](#)

Description: In the current version of the code, `LibSanitize.notInvalidBps` make sure that the newly introduced fee (denoted as `newFeeBps`) can not exceed 100% which is necessary but in practice to have a better limit on the admin power over the system.

Recommendation: It is highly recommended to make this change in fee stricter by using a maximum fee value that is less than 100% and is closer to a reasonable fee rate.

Kiln: Fixed in [049232eebb40d06c5517e78de6ffa84f0d80d834](#) by implementing the auditor's recommendation.

Spearbit: Fixed.

5.4.25 Factory's admin cannot stop the global oracle member from voting for reports

Severity: Low Risk

Context:

- [vOracleAggregator.sol#L267-L269](#)
- [vOracleAggregator.sol#L200](#)

Description: The Nexus global oracle can submit report for all spawned pools through the pool's oracle aggregator contract. The pool's factory's admin cannot remove the global oracle's power from voting for reports.

Recommendation: It might be best to allow the factory admins to have the decision to allow or disallow the global oracle. If this freedom is not granted to the factory admins, it should at least be communicated to them.

Kiln: The commit below allows to eject Global oracle member if there are enough local oracle members [4ce50cc4f1cbc65ea9d54827831c320fa1558c60](#).

Spearbit: Fixed.

5.4.26 `replaceCoreHatchers(...)` does not update the pluggable hatcher's mapping

Severity: Low Risk

Context:

- [Nexus.sol#L109-L116](#)
- [Nexus.sol#L195-L214](#)

Description: When the Nexus gets initialised the core hatchers are also marked as pluggable hatchers. But when these core hatchers are replaced, the pluggable hatcher's mapping for the old and new core hatcher addresses are not updated.

Recommendation: It would be best to document whether the core hatchers should also be marked as pluggable hatchers and the codebase then should respect this invariant.

Kiln: Fixed by removing the pluggable hatcher functionality from Nexus [b697c209452c41263a0a217a73198c1634ca5561](#).

Spearbit: Fixed.

5.4.27 `commitFixes()` can be called by anyone and a cub's fix can prevent other fixes to be applied

Severity: Low Risk

Context:

- [Hatcher.sol#L183-L188](#)

Description: Anyone can call the `commitFixes()` endpoint and populate the `$fixProgresses` storage and cause an event to be fired. For example, a cub's delegated-called `fix` can call into the hatcher `commitFixes()` and cause all the other fixes to be not applied to the cub.

Recommendation: One way to prevent this edge case would be to have a call back into the `fix` where the `fix` can query some on-chain data to make sure the fixed has been applied and then the `fix` would call into the `Hatcher` to mark itself as applied to that specific cub.

Kiln: Considering that fixes as as critical as contract upgrades, the code that would be applied as part of a fix will get the same level of review as a global implementation upgrade. To keep the fix process as simple as possible, I would prefer keeping it this way.

Spearbit: Acknowledged.

5.4.28 `_mint(...)` is missing checks after the `_onMint(...)` hook call

Severity: Low Risk

Context:

- [NFT.sol#L259](#)

Description: `_mint(...)` is missing checks after the `_onMint(...)` hook call.

Recommendation: The implementation of this `_mint` function is different from [OpenZeppelin's](#) regarding how the checks are performed after the `_onMint(...)` hook:

```
require(to != address(0), "ERC721: mint to the zero address");
require(!_exists(tokenId), "ERC721: token already minted");

_beforeTokenTransfer(address(0), to, tokenId, 1);

// Check that tokenId was not minted by `_beforeTokenTransfer` hook
require(!_exists(tokenId), "ERC721: token already minted");
```

After the `_onMint(...)` we should still check that the `tokenId` does not exist, otherwise it is possible to reenter and for example increase \$balances of multiple addresses more than just 1.

Kiln: Fixed in [PR 10](#).

Spearbit: Fixed.

5.4.29 `_burn(...)` does not check whether token still exists after calling `_onBurn(...)` hook

Severity: Low Risk

Context:

- [NFT.sol#L272](#)

Description: `_burn(...)` does not check whether token still exists after calling `_onBurn(...)` hook. In general `_onBurn` could reenter to for example increment `$burnCounter` more than once. hook.

Recommendation: Implementation of the checks before and after the `_onBurn` hook are different from OpenZeppelin's [implementation](#):

```
address owner = ERC721.ownerOf(tokenId); // <--- not needed for Kiln since `_onBurn` does not need the
↳ `owner`

_beforeTokenTransfer(owner, address(0), tokenId, 1);

// Update ownership in case tokenId was transferred by `_beforeTokenTransfer` hook
owner = ERC721.ownerOf(tokenId);
```

They are not directly comparable as the hooks take different type of arguments and the general assumptions are different. But One can avoid this issue by checking the token still exists after the `_onBurn`

Kiln: Fixed in [PR 10](#).

Spearbit: Fixed.

5.4.30 Lack of checks after calling `_onTransfer` hook

Severity: Low Risk

Context:

- [NFT.sol#L294](#)

Description: When `_onTransfer` hook is called in `_transfer(...)`, the implementation does not check whether the ownership has been moved during this hook call.

Recommendation: We need to make sure the `_onTransfer(...)` hook does not reenter to change the ownership of `tokenId`. Note OpenZeppelin has a post-hook [check](#):

```
require(ERC721.ownerOf(tokenId) == from, "ERC721: transfer from incorrect owner");
require(to != address(0), "ERC721: transfer to the zero address");

_beforeTokenTransfer(from, to, tokenId, 1);

// Check that tokenId was not transferred by `_beforeTokenTransfer` hook
require(ERC721.ownerOf(tokenId) == from, "ERC721: transfer from incorrect owner");
```

Kiln: Fixed in [PR 10](#).

Spearbit: Verified.

5.4.31 Minter counter `$idCounter` its not incremented on `_mint`

Severity: Low Risk

Context:

- [NFT.sol#L250-L265](#)

Description: The minter counter `$idCounter` its not being updated after a token is minted. This could trigger a DOS in the function `totalSupply()` on [NFT.sol#L95-L97](#) after burning one token.

Recommendation:

```
diff --git a/src/NFT.sol b/src/NFT.sol
index 6397510..0770db8 100644
--- a/src/NFT.sol
+++ b/src/NFT.sol
@@ -260,6 +260,10 @@ abstract contract NFT is INFT {

    $balances.get()[to.toUint256()] += 1;
    $owners.get()[tokenId] = to.toUint256();
+   unchecked {
+       // Will not overflow unless all 2**256 token ids are minted, which is virtually impossible.
+       $idCounter.set($idCounter.get() + 1);
+   }

    emit Transfer(address(0), to, tokenId);
}
```

Kiln: Fixed in [PR 10](#).

Spearbit: Fixed.

5.5 Gas Optimization

5.5.1 Pre-increment is cheaper than post-increment

Severity: Gas Optimization

Context:

- [Native1155.sol#L55](#)
- [Native1155.sol#L110](#)
- [Native20.sol#L58](#)
- [MultiPool1155.sol#L82](#)
- [MultiPool1155.sol#L127](#)
- [MultiPool20.sol#L182](#)
- [MultiPool20.sol#L213](#)
- [MultiPool.sol#L87](#)
- [MultiPool.sol#L222](#)
- [FeeDispatcher.sol#L76](#)
- [FeeDispatcher.sol#L96](#)

Description: Using pre-increment `++i` instead of post-increment `i++` can save 5 gas per loop, also using unchecked for incrementing `i` can save 30-40 gas per loop.

Recommendation: Use this pattern;

```
for (uint256 i; i < len;) {  
    // do some work  
    unchecked {  
        ++i;  
    }  
}
```

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.5.2 Expression `x = x - y` or `x = x + y` is cheaper than `x -= y` or `x += y`

Severity: Gas Optimization

Context:

- [vTreasury.sol#L163](#)
- [vTreasury.sol#L174](#)

Description: Expression `x = x - y` or `x = x + y` is more gas efficient than `x -= y` or `x += y`, this is true if the `--via-ir` flag is not turned on, otherwise the expression should be equivalent. More about [intermediate representation](#)

Tested using;

```
forge snapshot --diff base --fuzz-seed 1234 --mp test/vTreasury.t.sol  
# make the changes  
# check the new snapshot against previous one  
forge snapshot --diff base --fuzz-seed 1234 --mp test/vTreasury.t.sol  
Running 25 tests for test/vTreasury.t.sol:vTreasuryTest  
Overall gas change: -1794 (-0.017%)
```

Recommendation:

```
diff --git a/src/vTreasury.sol b/src/vTreasury.sol
index 5893fe2..c4f68a2 100644
--- a/src/vTreasury.sol
+++ b/src/vTreasury.sol
@@ -160,7 +160,7 @@ contract vTreasury is Initializable, Implementation, Fixable, IvTreasury {
    if (feeAmount > 0 && !pool.transferShares(exitQueue, feeAmount,
        ↪ abi.encodePacked(_globalRecipient())) {
        revert TransferError(address(pool), exitQueue, abi.encodePacked(_globalRecipient()));
    }
-    currentSharesCount -= feeAmount;
+    currentSharesCount = currentSharesCount - feeAmount;
}
// if the auto cover value for the pool is not null, we send the amount from the operator
↪ commission
// to the coverage recipient
@@ -171,7 +171,7 @@ contract vTreasury is Initializable, Implementation, Fixable, IvTreasury {
    if (autoCoverAmount > 0 && !pool.transferShares(coverageRecipient, autoCoverAmount, "")) {
        revert TransferError(address(pool), coverageRecipient, "");
    }
-    currentSharesCount -= autoCoverAmount;
+    currentSharesCount = currentSharesCount - autoCoverAmount;
}
if (currentSharesCount > 0 && !pool.transferShares(pool.exitQueue(), currentSharesCount,
    ↪ abi.encodePacked($operator.get())) {
    revert TransferError(address(pool), pool.exitQueue(), abi.encodePacked($operator.get()));
}
```

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.5.3 Cache redundant math operations, and cache `_thisBalance`

Severity: Gas Optimization

Context:

- [vCoverageRecipient.sol#L147-L151](#)
- [vWithdrawalRecipient.sol#L62-L64](#)
- [vCoverageRecipient.sol#L161-L162](#)

Description: Some math operations performed are redundantly, and there are opportunities to cache values to avoid multiple calls to the same function.

Recommendation:

- Avoid extra math operation in [vCoverageRecipient.sol#L147-L151](#)
- Return of `_thisBalance()` can be cached in [vWithdrawalRecipient.sol#L62-L64](#)
- Avoid extra math operation in [vCoverageRecipient.sol#L161-L162](#)

Kiln: [PR 62](#).

Spearbit: Fixed.

5.5.4 In `vOracleAggregator:submitReport` state variables `$pool` and `$reportVoteCounts` can be cached

Severity: Gas Optimization

Context:

- [vOracleAggregator.sol#L206](#)
- [vOracleAggregator.sol#L261](#)
- [vOracleAggregator.sol#L239](#)
- [vOracleAggregator.sol#L243](#)

Description: In the `submitReport` function, `$pool` and `$reportVoteCounts` are accessed multiple times. Each access could be relatively expensive in terms of gas cost.

Recommendation:

```
diff --git a/src/vOracleAggregator.sol b/src/vOracleAggregator.sol
index b8f3aec..91c8b16 100644
--- a/src/vOracleAggregator.sol
+++ b/src/vOracleAggregator.sol
@@ -203,7 +203,8 @@ contract vOracleAggregator is Fixable, Initializable, Implementation, IvOracleAg
     revert EpochTooOld(report.epoch, currentHighestReportedEpoch);
 }

-    IvPool($pool.get()).onlyValidEpoch(report.epoch);
+    IvPool _pool = IvPool($pool.get());
+    _pool.onlyValidEpoch(report.epoch);
     if (report.epoch > currentHighestReportedEpoch) {
         // if epoch is higher than highest and valid, we clear the reporting data of possibly old
         // epochs and we update the expectedEpoch
         _clear();
@@ -231,16 +232,18 @@ contract vOracleAggregator is Fixable, Initializable, Implementation, IvOracleAg
     bytes32 reportHash = _reportHash(report);

     int256 variantIdx = _variantIndex(reportHash);
-    uint256 variantVotes;
     uint256 voteCount = (isMember ? 1 : 0) + (isGlobalMember ? 1 : 0);
+    uint256 variantVotes;
+    uint256[] storage reportVoteCounts = $reportVoteCounts.toUintA();
     if (variantIdx == -1) {
         // if the proposed variant is not found, we add it and set the vote count to 1 (or 2 if
         // isMember && member == globalMember)
         $reportVariants.toBytes32A().push(reportHash);
-        $reportVoteCounts.toUintA().push(voteCount);
+        reportVoteCounts.push(voteCount);
+        variantVotes = voteCount;
     } else {
         // otherwise we simply increment the vote count
-        variantVotes = ($reportVoteCounts.toUintA()[uint256(variantIdx)] += voteCount);
+        variantVotes = reportVoteCounts[uint256(variantIdx)] + voteCount;
+        reportVoteCounts[uint256(variantIdx)] = variantVotes;
     }

     // both of these conditions can happen at the same time
@@ -258,7 +261,7 @@ contract vOracleAggregator is Fixable, Initializable, Implementation, IvOracleAg
     // quorum is met ! we can push to vPool, after clearing the report
     emit SubmittedReport(report, reportHash, variantVotes,
         // $reportVariants.toBytes32A().length);
         _clear();
-    IvPool($pool.get()).report(report);
+    _pool.report(report);
```

```
}  
}
```

Kiln: [PR 65](#).

Spearbit: Fixed.

5.5.5 Remove unnecessary else statement for gas optimization

Severity: Gas Optimization

Context:

- [vFactory.sol#L811-L815](#)

Description: The `else` statement is unnecessary. If the condition in the `if` statement is `true`, it will `revert`, and the code will not continue.

Recommendation:

```
diff --git a/src/vFactory.sol b/src/vFactory.sol  
index f943f94..ef210ae 100644  
--- a/src/vFactory.sol  
+++ b/src/vFactory.sol  
@@ -810,8 +810,7 @@ contract vFactory is Depositor, Administrable, Initializable, Implementation, Fi  
    );  
    if (!success) {  
        revert MinimalRecipientExecutionError(rdata);  
-    } else {  
-        emit Withdraw(id, recipient, recipientBalance);  
    }  
+    emit Withdraw(id, recipient, recipientBalance);  
    }  
}
```

Kiln: Fixed in [PR 60](#).

Spearbit: Fixed.

5.5.6 `toLittleEndian64` function could be optimize

Severity: Gas Optimization

Context:

- [LibUint256.sol#L16-L30](#)

Description: The `toLittleEndian64` function could be simplified, you could use the proposed method used in the official as reference [DepositContract implementation](#). Or a yul more optimized version.

Recommendation: This issue has already been discussed on [review-kiln/issues/39](#). Please also take in mind that you might not need to use this function at all [PR 114](#).

Kiln: Fixed in [PR 7](#).

Spearbit: Fixed.

5.5.7 Optimize `authorized` function with using yul

Severity: Gas Optimization

Context:

- [LibErrors.sol#L21-L25](#)

Description: The `authorized` function checks if the caller is the expected address. However, it could be optimized for better gas efficiency by using yul.

Recommendation: Implement the `authorized` function using inline assembly as shown below:

```
function authorized(address _caller, address _expected) internal pure {
    assembly {
        // if dirty upper bits present, they should also match
        if xor(_caller, _expected) {
            // bytes4(keccak256("Unauthorized(address account, address expected)")) == 0x295a81c1
            mstore(0, 0x295a81c1)
            mstore(0x20, _caller)
            mstore(0x40, _expected)
            revert(0x1c, 0x44)
        }
    }
}
```

Kiln: Fixed [PR 7](#).

Spearbit: Fixed.

5.5.8 Optimize `min` and `max` functions using a yul implementation

Severity: Gas Optimization

Context:

- [LibUint256.sol#L33-L35](#)
- [LibUint256.sol#L38-L40](#)

Description: The current implementation of `min` and `max` functions uses ternary conditional logic to determine the smallest or largest number. While this approach is straightforward, it may not be the most gas-efficient solution.

Recommendation: It is suggested to use a YUL implementation for the `min` and `max` functions, as it can be more gas-efficient. You could based on the Solady library implementation.

- Refactor `min` function using this implementation, [Solady:FixedPointMathLib.sol#L638-L643](#)
- Refactor `max` function using this implementation, [Solady:FixedPointMathLib.sol#L654-L659](#)

Kiln: Fixed in [PR 7](#).

Spearbit: Fixed.

5.5.9 On `vPool:_transfer` balance can be reused to avoid extra storage read

Severity: Gas Optimization

Context:

- [vPool.sol#L1238](#)

Description: In the `_transfer` function, the balance of the from account is fetched once and used in a conditional check. When deducting the amount from the balance, it is fetched again, which is more expensive in terms of gas cost due to the storage read operation.

Recommendation:

```
diff --git a/src/vPool.sol b/src/vPool.sol
index e0cc0ef..b27b62e 100644
--- a/src/vPool.sol
+++ b/src/vPool.sol
@@ -1235,7 +1235,7 @@ contract vPool is Initializable, Implementation, Fixable, IvPool {
    emit Transfer(from, to, amount);

    unchecked {
-       $balances.get()[from.k()] -= amount;
+       $balances.get()[from.k()] = balance - amount;
    }
    $balances.get()[to.k()] += amount;
```

Kiln: Fixed in [PR 63](#).

Spearbit: Fixed.

5.5.10 On `vPool:purchaseValidators` computation of the total cost can be cached

Severity: Gas Optimization

Context:

- [vPool.sol#L502-L504](#)

Description: `LibConstant.DEPOSIT_SIZE * purchasableValidatorCount` is calculated twice. This value can be cached to avoid extra gas costs.

Recommendation:

```
diff --git a/src/vPool.sol b/src/vPool.sol
index e0cc0ef..36270da 100644
--- a/src/vPool.sol
+++ b/src/vPool.sol
@@ -499,9 +499,10 @@ contract vPool is Initializable, Implementation, Fixable, IvPool {
    IvFactory vfactory = IvFactory($factory.get());
    uint256 purchasableValidatorCount =
        ↳ LibUint256.min(vfactory.availableValidators(withdrawalChannel), maxPurchaseAmount);

-    _setCommittedEthers(committedEthers - uint128(LibConstant.DEPOSIT_SIZE *
↳ purchasableValidatorCount));
+    uint256 _totalValue = LibConstant.DEPOSIT_SIZE * purchasableValidatorCount;
+    _setCommittedEthers(committedEthers - uint128(_totalValue));

-    uint256[] memory purchasedValidatorsIds = vfactory.deposit{value: LibConstant.DEPOSIT_SIZE *
↳ purchasableValidatorCount}(
+    uint256[] memory purchasedValidatorsIds = vfactory.deposit{value: _totalValue}(
        withdrawalChannel, purchasableValidatorCount, $execLayerRecipient.get(), address(this),
        ↳ $validatorGlobalExtraData.get()
    );
```

Kiln: Fixed in [PR 63](#).

Spearbit: Fixed.

5.5.11 On `vPool:report` state variable `$coveredBalanceSum.get()` can be cache to avoid extra storage reads

Severity: Gas Optimization

Context:

- [vPool.sol#L853-L1010](#)

Description: In the `report` function, the value of `$coveredBalanceSum.get()` are used multiple times. This can be optimized by caching the values to avoid extra storage reads.

Recommendation:

```

diff --git a/src/vPool.sol b/src/vPool.sol
index e0cc0ef..953cb50 100644
--- a/src/vPool.sol
+++ b/src/vPool.sol
@@ -850,13 +850,14 @@ contract vPool is Initializable, Implementation, Fixable, IvPool {
    // - The covered amount is also requested to be under the upper bound margin but there is
    ↪ an addition "boost" margin //
    // that helps pull coverage funds faster. You can see that the margin for this specific
    ↪ operation is increased. //
    //
    ↪ //
+   uint256 _coveredBalanceSum = $coveredBalanceSum.get();
    {
        // ----- We increase the allowed margin for coverage
        ↪ //
        uint256 maxPullableCoverage = __.increaseCredit + __.traces.coverageIncreaseLimit;
        // ----- We only pull funds if there is a difference with the reported slashed balance
        ↪ and the amount covered by //
        // the vPool
        ↪ //
-       if (maxPullableCoverage > 0 && $coveredBalanceSum.get() < rppt.slashedSum) {
-       uint256 maxCoverableAmount = LibUint256.min(rppt.slashedSum -
    ↪ $coveredBalanceSum.get(), maxPullableCoverage);
+       if (maxPullableCoverage > 0 && _coveredBalanceSum < rppt.slashedSum) {
+       uint256 maxCoverableAmount = LibUint256.min(rppt.slashedSum - _coveredBalanceSum,
    ↪ maxPullableCoverage);
        __.traces.pulledCoverageFunds = uint128(_pullCoverage(maxCoverableAmount));
        __.traces.delta += int128(__.traces.pulledCoverageFunds);
    }
@@ -1001,12 +1002,13 @@ contract vPool is Initializable, Implementation, Fixable, IvPool {
    //
    ↪ //
    // ----- The tracked covered amount is updated
    ↪ //
    if (__.traces.pulledCoverageFunds > 0) {
-       $coveredBalanceSum.set($coveredBalanceSum.get() + __.traces.pulledCoverageFunds);
+       _coveredBalanceSum = _coveredBalanceSum + __.traces.pulledCoverageFunds;
+       $coveredBalanceSum.set(_coveredBalanceSum);
    }
    //
    ↪ //
    // ----- And if we covered more than the slashing balance sum, we revert
    ↪ //
-   if ($coveredBalanceSum.get() > rppt.slashedSum) {
-   revert CoverageHigherThanLoss($coveredBalanceSum.get(), rppt.slashedSum);
+   if (_coveredBalanceSum > rppt.slashedSum) {
+   revert CoverageHigherThanLoss(_coveredBalanceSum, rppt.slashedSum);
    }
    //
    ↪ //
    // ----- Now that funds and internal buffers are updated, we retrieve the final total
    ↪ underlying supply and total supply //

```

Kiln: Fixed in [PR 63](#).

Spearbit: Fixed.

5.5.12 Optimize `onlyOperatorOrAdmin` modifier by consolidating function calls and reducing storage reads

Severity: Gas Optimization

Context:

- [vPool.sol#L202-L211](#)

Description: The current implementation of the `onlyOperatorOrAdmin` modifier executes two function calls to the factory for each operation. This can lead to increased gas costs and could be optimized by consolidating these two calls into one or by checking for the actor that is most likely to trigger this modifier first. Additionally, caching `$factory.get()` could reduce storage reads.

Recommendation:

```
diff --git a/src/vPool.sol b/src/vPool.sol
index e0cc0ef..7f9f160 100644
--- a/src/vPool.sol
+++ b/src/vPool.sol
@@ -201,10 +201,13 @@ contract vPool is Initializable, Implementation, Fixable, IvPool {
    /// @notice Only allows the admin or the operator to perform the call
    modifier onlyOperatorOrAdmin() {
        {
            address operator = IvFactory($factory.get()).operator();
            address admin = IvFactory($factory.get()).admin();
            if (msg.sender != operator && msg.sender != admin) {
                revert LibErrors.Unauthorized(msg.sender, address(0));
            }
            IvFactory _factory = IvFactory($factory.get());
            address _role = _factory.operator();
            if (msg.sender != _role) {
                _role = _factory.admin();
                if (msg.sender != _role) {
                    revert LibErrors.Unauthorized(msg.sender, address(0));
                }
            }
        }
    }
}
```

Kiln: Fixed in [PR 63](#).

Spearbit: Fixed.

5.5.13 Optimize `vPool::_burn` function by reducing redundant reads

Severity: Gas Optimization

Context:

- [vPool.sol#L1216-L1221](#)

Description: The current implementation of `_burn` function can be optimized by reducing redundant reads of `$totalSupply.get()`. By caching the value of `$totalSupply.get()` - amount into a local variable, we can reduce the gas costs associated with this operation.

Recommendation:

```

diff --git a/src/vPool.sol b/src/vPool.sol
index e0cc0ef..bed7ed8 100644
--- a/src/vPool.sol
+++ b/src/vPool.sol
@@ -1215,9 +1215,11 @@ contract vPool is Initializable, Implementation, Fixable, IvPool {
    /// @param owner The address burning the shares
    function _burn(uint256 amount, address owner) internal {
        $balances.get()[owner.k()] -= amount;
-       $totalSupply.set($totalSupply.get() - amount);
+
+       uint256 newTotalSupply = $totalSupply.get() - amount;
+       $totalSupply.set(newTotalSupply);
        emit Transfer(owner, address(0), amount);
-       emit Burn(owner, amount, $totalSupply.get());
+       emit Burn(owner, amount, newTotalSupply);
    }

    /// @notice Internal utility to transfer shares of the vPool

```

Kiln: Fixed in [PR 63](#).

Spearbit: Fixed.

5.5.14 Optimize vPool::_mint function by reducing redundant reads

Severity: Gas Optimization

Context:

- [vPool.sol#L1202-L1211](#)

Description: The current implementation of _mint function is performing redundant reads of the \$totalSupply.get(). This can be optimized by passing totalSupply as a parameter, as it has already been read at the _mint call sites. Caching \$totalSupply.get() + amount can also improve the gas efficiency.

Recommendation: Refactor the _mint function with the proposed implementation.

```

diff --git a/src/vPool.sol b/src/vPool.sol
index e0cc0ef..1842002 100644
--- a/src/vPool.sol
+++ b/src/vPool.sol
@@ -476,10 +476,10 @@ contract vPool is Initializable, Implementation, Fixable, IvPool {
    // The exchange rate cannot change without active validators and active validators cannot exist
    // before at least DEPOSIT_SIZE has been deposited
    if (currentTotalUnderlyingBalance < LibConstant.DEPOSIT_SIZE) {
-       _mint(amountToMint, msg.sender);
+       _mint(amountToMint, currentTotalSupply, msg.sender);
    } else {
        amountToMint = LibUint256.mulDiv(msg.value, currentTotalSupply,
            ↪ currentTotalUnderlyingBalance);
-       _mint(amountToMint, msg.sender);
+       _mint(amountToMint, currentTotalSupply, msg.sender);
    }

    return amountToMint;
@@ -1201,11 +1201,15 @@ contract vPool is Initializable, Implementation, Fixable, IvPool {

    /// @notice Internal utility to mint new shares of the vPool
    /// @param amount The amount of shares to mint
+   /// @param totalSupply The current totalSupply

```



```

    /// @param owner The address receiving the shares
-   function _mint(uint256 amount, address owner) internal {
+   function _mint(uint256 amount, uint256 totalSupply, address owner) internal {
        $balances.get()[owner.k()] += amount;
        $totalSupply.set($totalSupply.get() + amount);
        emit Mint(owner, amount, $totalSupply.get());
        totalSupply += amount;
+
+       $totalSupply.set(totalSupply);
+       emit Mint(owner, amount, totalSupply);
+
        emit Transfer(address(0), owner, amount);
        _acceptanceChecks(address(this), address(0), owner, amount, "");
    }
@@ -1341,7 +1345,7 @@ contract vPool is Initializable, Implementation, Fixable, IvPool {
    currentTotalSupply,
    currentTotalUnderlyingSupply
    );
-    _mint(sharesToMint, operatorTreasury);
+    _mint(sharesToMint, currentTotalSupply, operatorTreasury);
}
}

```

Kiln: Fixed in [PR 63](#).

Spearbit: Fixed.

5.5.15 Optimize vPool:_consumeApproval function by reducing storage reads

Severity: Gas Optimization

Context:

- [vPool.sol#L1259-L1261](#)

Description: The current implementation of the _consumeApproval function can be optimized for better performance and lower gas costs by reducing redundant storage reads. This can be achieved by reusing the local variable approval instead of reading from storage again. Note that when approval == type(uint256).max no Approval event is emitted. This behaviour matches with OpenZeppelin's ERC20 implementation.

Recommendation:

```

diff --git a/src/vPool.sol b/src/vPool.sol
index e0cc0ef..f681fc3 100644
--- a/src/vPool.sol
+++ b/src/vPool.sol
@@ -1254,11 +1254,14 @@ contract vPool is Initializable, Implementation, Fixable, IvPool {
    if (approval < amount) {
        revert AllowanceTooLow(owner, operator, approval, amount);
    }
+
    if (approval != type(uint256).max) {
        unchecked {
-            $approvals.get()[owner][operator] -= amount;
+            approval = approval - amount;
        }
-        emit Approval(owner, operator, approval - amount);
+
+        $approvals.get()[owner][operator] = approval;
+        emit Approval(owner, operator, approval);
    }
}

```

Kiln: Fixed in [PR 63](#).

Spearbit: Fixed.

5.5.16 Functions `toUint256` and `toBool` can be optimized using Yul

Severity: Gas Optimization

Context:

- [uint256.sol#L53](#)
- [bytes32.sol#L54](#)
- [address.sol#L55](#)
- [bool.sol#L53](#)

Description: The current implementation of the `toUint256` and `toBool` functions can be optimized for performance and gas cost by using Yul.

In the case of `toBool`, the current version only checks if the first bit is 1, and the proposed optimized one will check for value greater than zero.

In `toUint256`, the proposed implementation will clean any upper dirty bits in the conversion process.

Recommendation:

- Refactor `toBool` in [uint256.sol#L53](#)
- Refactor `toBool` in [bytes32.sol#L54](#)
- Refactor `toBool` in [address.sol#L55](#)
- Refactor `toUint256` in [bool.sol#L53](#)

Please note that if, in the future, dirty bits are introduced in the storage of these bool type values, even if the lower bit would be 0, `toBool(uint256)` would return `true`. Therefore, the implementation must be carefully managed to prevent any potential issues.

Suggested implementation:

```
// you will have to change signature from (uint256 val) to (bytes32 val) and (address val) for  
↳ bytes32.sol and address.sol  
function toBool(uint256 val) internal pure returns (bool converted) {  
    assembly {  
        converted := gt(val, 0)  
    }  
}  
  
function toUint256(bool val) internal pure returns (uint256 converted) {  
    assembly {  
        converted := iszero(iszero(val))  
    }  
}
```

Kiln: Fixed in [PR 8](#).

Spearbit: Fixed.

5.5.17 Improve efficiency in `vTreasury:onvPoolSharesReceived` by updating `poolShares`

Severity: Gas Optimization

Context:

- `vTreasury.sol`#L132
- `vTreasury.sol`#L143

Description: The current implementation in `vTreasury.sol` calculates the updated `poolShares` value as `$poolShares.get()[msg.sender.k()] += amount;`. However, the current shares count is already calculated and stored in the `currentSharesCount` variable.

Recommendation:

```
diff --git a/src/vTreasury.sol b/src/vTreasury.sol
index 5893fe2..9da72b4 100644
--- a/src/vTreasury.sol
+++ b/src/vTreasury.sol
@@ -140,7 +140,7 @@ contract vTreasury is Initializable, Implementation, Fixable, IvTreasury {
    if (LibUint256.mulDiv(currentSharesCount, totalUnderlyingSupply, totalSupply) >=
        EXIT_THRESHOLD) {
        _exitAndFundCoverageFund(pool, currentSharesCount);
    } else {
-        $poolShares.get()[msg.sender.k()] += amount;
+        $poolShares.get()[msg.sender.k()] = currentSharesCount;
    }

    return IvPoolSharesReceiver.onvPoolSharesReceived.selector;
```

Kiln: Fixed in [PR 61](#).

Spearbit: Fixed.

5.5.18 Refactor `vFactory:withdraw` function to avoid extra storage reads

Severity: Gas Optimization

Context:

- `vFactory.sol`#L595-L629

Description: In the `vFactory:withdraw` function, there are additional storage reads that could be optimized. Specifically, the `ctypes.Deposit` data structure is read from storage multiple times, and the withdrawal channel is guaranteed to be 0.

Recommendation: Consider passing `ctypes.Deposit` memory `d = ds[id]` as a parameter to avoid reading it from storage again. Additionally, the withdrawal channel could be hardcoded to 0 in `_withdraw(...)` because you have already ensured it on `vFactory.sol`#L615-L617

```
diff --git a/src/vFactory.sol b/src/vFactory.sol
index f943f94..a76dc77 100644
--- a/src/vFactory.sol
+++ b/src/vFactory.sol
@@ -612,15 +612,11 @@ contract vFactory is Depositor, Administrable, Initializable, Implementation, Fi
    revert InvalidValidatorId(id);
}

-    if (d.withdrawalChannel != bytes32(0)) {
-        revert InvalidWithdrawalChannel(d.withdrawalChannel);
-    }
-
    if (d.owner != msg.sender) {
        revert LibErrors.Unauthorized(msg.sender, d.owner);
    }
```

```

    }

-     _withdraw(id, recipient);
+     _withdraw(id, d, recipient);

    unchecked {
        ++idx;
@@ -784,9 +780,12 @@ contract vFactory is Depositor, Administrable, Initializable, Implementation, Fi
    /// @param id The validator id
    /// @param recipient The address receiving the funds
    // slither-disable-next-line reentrancy-events,calls-loop
-    function _withdraw(uint256 id, address recipient) internal {
-        ctypes.Deposit memory d = $deposits.get()[id];
-        ctypes.Validator storage v = $withdrawalChannels.get()[d.withdrawalChannel].validators[d.index
-  ↪ - 1];
+    function _withdraw(uint256 id, ctypes.Deposit memory d, address recipient) internal {
+        if (d.withdrawalChannel != bytes32(0)) {
+            revert InvalidWithdrawalChannel(d.withdrawalChannel);
+        }
+
+        ctypes.Validator storage v = $withdrawalChannels.get()[0].validators[d.index - 1];

        bytes memory publicKey = LibPublicKey.toBytes(v.publicKey);

@@ -810,8 +809,7 @@ contract vFactory is Depositor, Administrable, Initializable, Implementation, Fi
    );
    if (!success) {
        revert MinimalRecipientExecutionError(rdata);
-    } else {
-        emit Withdraw(id, recipient, recipientBalance);
-    }
+    emit Withdraw(id, recipient, recipientBalance);
}
}
}

```

Kiln: Fixed in [PR 60](#).

Spearbit: Fixed.

5.5.19 LibSignature.SIGNATURE_LENGTH - 64 is constant therefore can be precalculated.

Severity: Gas Optimization

Context:

- [Depositor.sol#L75](#)

Description: Since LibSignature.SIGNATURE_LENGTH - 64 equals 32, and this value is constant, it could be replaced with the constant 32 to improve readability and avoid unnecessary computations.

Recommendation: Just replace LibSignature.SIGNATURE_LENGTH - 64 for 32 or setup a constant in [LibSignature.sol](#) to avoid magic numbers.

```

diff --git a/src/Depositor.sol b/src/Depositor.sol
index 0957c9d..d0c6e23 100644
--- a/src/Depositor.sol
+++ b/src/Depositor.sol
@@ -72,7 +72,8 @@ contract Depositor is IDepositor {
    bytes32 signatureRoot = sha256(
        bytes.concat(
            sha256(LibBytes.slice(_signature, 0, 64)),
-           sha256(bytes.concat(LibBytes.slice(_signature, 64, LibSignature.SIGNATURE_LENGTH -
↪ 64), bytes32(0)))
+           // 32 = LibSignature.SIGNATURE_LENGTH - 64
+           sha256(bytes.concat(LibBytes.slice(_signature, 64, 32), bytes32(0)))
        )
    );

```

Kiln: Fixed in [PR 7](#).

Spearbit: Fixed.

5.5.20 The depositAmount is constant; therefore toLittleEndian64 value can be precalculated and set as a constant.

Severity: Gas Optimization

Context:

- [Depositor.sol#L69](#)
- [Depositor.sol#L82](#)

Description: The depositAmount is constant; therefore, the value of toLittleEndian64(depositAmount) can be precalculated and set as a constant. This issue has also been reported on [here](#):

- [PR 47](#)

Recommendation:

```

diff --git a/src/Depositor.sol b/src/Depositor.sol
index 0957c9d..1abb594 100644
--- a/src/Depositor.sol
+++ b/src/Depositor.sol
@@ -33,6 +33,8 @@ contract Depositor is IDepositor {
    types.Address internal constant $depositContract =
        types.Address.wrap(0x51b708339b28db69fafc07d2fc5a46c3487f5c5cd1fcb575eb01044dd8dd4de6);

+    uint256 internal constant DEPOSIT_SIZE_AMOUNT_LITTLEENDIAN64 =
+        0x0040597307000000000000000000000000000000000000000000000000000000;
+
    /// @inheritdoc IDepositor
    function depositContract() external view returns (address) {
        return $depositContract.get();
@@ -66,8 +68,8 @@ contract Depositor is IDepositor {
        revert InvalidDepositSize();
    }

-    uint256 depositAmount = LibConstant.DEPOSIT_SIZE / 1 gwei;
-
    bytes32 pubkeyRoot = sha256(bytes.concat(_publicKey, bytes16(0)));
    bytes32 signatureRoot = sha256(
        bytes.concat(
@@ -79,7 +79,7 @@ contract Depositor is IDepositor {
        bytes32 depositDataRoot = sha256(
            bytes.concat(
                sha256(bytes.concat(pubkeyRoot, withdrawalCredentials)),
-                sha256(bytes.concat(bytes32(LibUint256.toLittleEndian64(depositAmount)),
+                signatureRoot))
+                sha256(bytes.concat(bytes32(DEPOSIT_SIZE_AMOUNT_LITTLEENDIAN64), signatureRoot))
            )
        );
};

```

Kiln: Fixed in [PR 9](#).

Spearbit: Fixed.

5.5.21 Calculation of sharesAcquired in _stake can be simplified

Severity: Gas Optimization

Context:

- [MultiPool20.sol#L149](#)
- [MultiPool20.sol#L157](#)

Description: Calculation of sharesAcquired in _stake can be simplified.

Recommendation: By swapping two lines, we can avoid two unnecessary subtractions and also simplify formulas:

```

diff --git a/src/MultiPool20.sol b/src/MultiPool20.sol
index 65de85b..e39cdd5 100644
--- a/src/MultiPool20.sol
+++ b/src/MultiPool20.sol
@@ -144,17 +144,17 @@ abstract contract MultiPool20 is MultiPool, IMultiPool20 {
        uint256 pSharesBought = LibUint256.mulDiv(
            ethForCommission, $poolShares.get()[id] - _poolSharesOfIntegrator(id),
            ↪ _ethAfterCommission(id)
        );
-        $commissionPaid.get()[id] += ethForCommission;
        uint256 sharesAcquired =
-        LibUint256.mulDiv(ethForCommission, _totalSupply(),
-        ↪ _totalUnderlyingSupply() - ethForCommission);
+        LibUint256.mulDiv(ethForCommission, _totalSupply(),
+        ↪ _totalUnderlyingSupply());
+        $commissionPaid.get()[id] += ethForCommission;
        sharesBoughtTotal += sharesAcquired;
        _mint(msg.sender, sharesAcquired);
        emit CommissionSharesSold(pSharesBought, id, ethForCommission);
    }
    if (amount > 0) {
-        $injectedEth.get()[id] += amount;
        pool.deposit{value: amount}();
-        uint256 sharesAcquired = LibUint256.mulDiv(amount, _totalSupply(),
-        ↪ _totalUnderlyingSupply() - amount);
+        uint256 sharesAcquired = LibUint256.mulDiv(amount, _totalSupply(),
+        ↪ _totalUnderlyingSupply());
+        $injectedEth.get()[id] += amount;
        sharesBoughtTotal += sharesAcquired;
        _mint(msg.sender, sharesAcquired);
    }
}

```

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.5.22 vFactory._removeValidatorsFromWithdrawalChannel - redundant check inside a loop

Severity: Gas Optimization

Context:

- [vFactory.sol#L751-L753](#)

Description:

```

if (indexToRemove < smallestIndex) {
    smallestIndex = indexToRemove;
}

```

The above if statement is redundant, as it will always evaluate to false since it's being validated earlier in the loop and would revert for UnsortedIndexArray otherwise.

Recommendation: Consider removing this code block entirely.

Kiln: Fixed in [b2915fd69f91c95e31c8725a1ac2333a246245ab](#) by implementing the auditor's recommendation.

Spearbit: Fixed.

5.5.23 Checking the equality of the length of multiple function parameters can be avoided

Severity: Gas Optimization

Context:

- [ExitQueueClaimHelper.sol#L24](#)
- [ExitQueueClaimHelper.sol#L48](#)
- [FeeDispatcher.sol#L89](#)
- [MultiPool1155.sol#L75](#)
- [MultiPool1155.sol#L163](#)
- [Native20.sol#L52-L54](#)
- [Native1155.sol#L103](#)
- [vFactory.sol#L357](#)

Description: In this context, we have functions of the form:

```
function foo(T1[] a1, T2[] a2, ..., Tn[] an, U1 b1, ..., Um bm) visibility mutability returns(...) {  
    // checks that a1.length == a2.length == ... == an.length  
}
```

Recommendation: The length checks performed are redundant if we define input structs of the following form:

```
struct T {  
    T1 a1;  
    T2 a2;  
    ...  
    Tn an;  
}  
function foo(T[] a, U1 b1, ..., Um bm) visibility mutability returns(...) {  
    // no length checks needed as its abstracted in the struct definition  
}
```

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.5.24 Cache variable `d.withdrawalChannel` in `vFactory:setOwner`

Severity: Gas Optimization

Context:

- [vFactory.sol#L511-L512](#)

Description: In the `setOwner` function of the `vFactory` contract, the `d.withdrawalChannel` variable is used multiple times and can be cache.

Recommendation:


```

diff --git a/src/vFactory.sol b/src/vFactory.sol
index f943f94..63c91c3 100644
--- a/src/vFactory.sol
+++ b/src/vFactory.sol
@@ -508,8 +508,9 @@ contract vFactory is Depositor, Administrable, Initializable, Implementation, Fi
    emit SetValidatorOwner(id, newOwner);

    unchecked {
-       --bs[d.withdrawalChannel][msg.sender];
-       ++bs[d.withdrawalChannel][newOwner];
+       bytes32 channel = d.withdrawalChannel;
+       --bs[channel][msg.sender];
+       ++bs[channel][newOwner];
+       ++idx;
    }
}

```

Kiln: Fixed [PR 60](#).

Spearbit: Fixed.

5.5.25 Use the cached withdrawalChannelCount variable instead of withdrawalChannels.length

Severity: Gas Optimization

Context:

- [vFactory.sol#L362](#)

Description: Use the cached withdrawalChannelCount variable instead of withdrawalChannels.length.

Recommendation:

```

diff --git a/src/vFactory.sol b/src/vFactory.sol
index f943f94..f275684 100644
--- a/src/vFactory.sol
+++ b/src/vFactory.sol
@@ -359,7 +359,7 @@ contract vFactory is Depositor, Administrable, Initializable, Implementation, Fi
    if (withdrawalChannelCount != limits.length || limits.length != snapshots.length) {
        revert InvalidArrayLengths();
    }
-   if (withdrawalChannels.length == 0) {
+   if (withdrawalChannelCount == 0) {
        revert EmptyArray();
    }
}

```

Kiln: Fixed [PR 60](#).

Spearbit: Fixed.

5.5.26 `_removeValidatorsFromWithdrawalChannel` can be optimized

Severity: Gas Optimization

Context:

- [vFactory.sol#L721-L767](#)

Description: `_removeValidatorsFromWithdrawalChannel` can be optimized.

Related issue [PR 30](#).

Recommendation: Here is a more optimised version (rough) of this endpoint:

```
/// @notice Internal utility to remove keys from a withdrawal channel
/// @param wc The withdrawal channel to remove keys from
/// @param indexes The list of key indexes to remove from the withdrawal channel
/// @param index The index of the withdrawal channel in the initial array, only used for events
function _removeValidatorsFromWithdrawalChannel(bytes32 wc, uint256[] calldata indexes, uint256
→ index) internal {
    ctypes.WithdrawalChannel storage withdrawalChannelStruct = $withdrawalChannels.get()[wc];
    uint256 indexesCount = indexes.length;
    uint256 totalValidatorCount = withdrawalChannelStruct.validators.length;
    uint256 smallestIndex = indexes[indexesCount - 1];

    if (smallestIndex < withdrawalChannelStruct.funded) {
        revert FundedValidatorRemovalAttempt(index, smallestIndex);
    }

    uint256 biggestIndex = indexes[0];
    if (!(biggestIndex < totalValidatorCount)) {
        revert ValidatorIndexOutOfBounds(index, biggestIndex);
    }

    for (uint256 idx = 0; idx < indexesCount;) {
        uint256 indexToRemove = indexes[idx];
        if (idx > 0 && !(indexToRemove < indexes[idx - 1])) {
            revert UnsortedIndexArray(index);
        }

        ctypes.Validator memory removedValidator =
        → withdrawalChannelStruct.validators[indexToRemove];
        emit RemovedValidator(wc, LibPublicKey.toBytes(removedValidator.publicKey), indexToRemove);
        unchecked { ++idx; }
        uint256 lastValidatorIndex = totalValidatorCount - idx;
        if (indexToRemove != lastValidatorIndex) {
            withdrawalChannelStruct.validators[indexToRemove] =
            → withdrawalChannelStruct.validators[lastValidatorIndex];
        }
        withdrawalChannelStruct.validators.pop();
    }

    if (withdrawalChannelStruct.limit > smallestIndex) {
        withdrawalChannelStruct.limit = uint32(smallestIndex);
        emit UpdatedLimit(wc, smallestIndex);
    }
    withdrawalChannelStruct.lastEdit = block.number;
}
```

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.5.27 On `_approve` **cache** `withdrawalChannelStruct.limit` and avoid return using `if else`

Severity: Gas Optimization

Context:

- [vFactory.sol#L676-L680](#)

Description: `withdrawalChannelStruct.limit` in a local variable, also the `if` could be refactored to avoid the `return` statement.

Recommendation:

```
diff --git a/src/vFactory.sol b/src/vFactory.sol
index f943f94..7cf43f2 100644
--- a/src/vFactory.sol
+++ b/src/vFactory.sol
@@ -673,11 +673,10 @@ contract vFactory is Depositor, Administrable, Initializable, Implementation, Fi
    function _approve(bytes32 wc, uint256 limit, uint256 snapshot) internal {
        ctypes.WithdrawalChannel storage withdrawalChannelStruct = $withdrawalChannels.get()[wc];

-        if (limit == withdrawalChannelStruct.limit) {
+        uint32 currentLimit = withdrawalChannelStruct.limit;
+        if (limit == currentLimit) {
            emit UpdatedLimit(wc, limit);
-            return;
-        }
-        if (limit < withdrawalChannelStruct.limit) {
+        } else if (limit < currentLimit) {
            uint32 newLimit = limit < withdrawalChannelStruct.funded ? withdrawalChannelStruct.funded
            ↪ : uint32(limit);
            withdrawalChannelStruct.limit = newLimit;
            emit UpdatedLimit(wc, newLimit);
```

Kiln: Fixed [PR 60](#).

Spearbit: Fixed.

5.5.28 **Cache** `withdrawalChannelStruct.funded` in `exitTotal`

Severity: Gas Optimization

Context:

- [vFactory.sol#L586-L589](#)

Description: In the `exitTotal` function of the `vFactory` contract, the `withdrawalChannelStruct.funded` variable is used multiple times. Each access to this variable consumes gas, which could be saved by caching the variable.

Recommendation: It is recommended to cache the `withdrawalChannelStruct.funded` variable in a local variable within the `exitTotal` function.

```
diff --git a/src/vFactory.sol b/src/vFactory.sol
index f943f94..6d14a4d 100644
--- a/src/vFactory.sol
+++ b/src/vFactory.sol
@@ -583,9 +583,10 @@ contract vFactory is Depositor, Administrable, Initializable, Implementation, Fi
    LibSanitize.notNullValue(totalExited);
    bytes32 wc = LibAddress.toWithdrawalCredentials(msg.sender);
    ctypes.WithdrawalChannel storage withdrawalChannelStruct = $withdrawalChannels.get()[wc];
-   if (totalExited > withdrawalChannelStruct.funded) {
-       emit ExitRequestAboveFunded(withdrawalChannelStruct.funded, totalExited);
-       totalExited = withdrawalChannelStruct.funded;
+   uint32 _funded = withdrawalChannelStruct.funded;
+   if (totalExited > _funded) {
+       emit ExitRequestAboveFunded(_funded, totalExited);
+       totalExited = _funded;
    }
    emit SetExitTotal(wc, totalExited);
    return totalExited;
```

Kiln: Fixed [PR 60](#).

Spearbit: Fixed.

5.5.29 Remove redundant checks in `vFactory::initialize` for `address(0)`

Severity: Gas Optimization

Context:

- [vFactory.sol#L107-L108](#)

Description: In the `initialize` method, checks are performed on the `operator_` and `treasury_` to ensure that the address is not `address(0)`. However, these checks appear to be redundant as same checks are already performed when calling the `_setOperator` and `_setTreasury` functions.

Recommendation: Remove the redundant checks for `address(0)` in the `initialize` method.

```
diff --git a/src/vFactory.sol b/src/vFactory.sol
index f943f94..5dd8b43 100644
--- a/src/vFactory.sol
+++ b/src/vFactory.sol
@@ -104,8 +104,6 @@ contract vFactory is Depositor, Administrable, Initializable, Implementation, Fi
    address minimalRecipientImplementation,
    address hatcherRegistry
) external init(0) {
-   LibSanitize.notZeroAddress(operator_);
-   LibSanitize.notZeroAddress(treasury_);
    LibSanitize.notZeroAddress(minimalRecipientImplementation);
    LibSanitize.notZeroAddress(hatcherRegistry);
```

Kiln: Fixed in [PR 60](#).

Spearbit: Fixed.

5.5.30 Cache `currentBalance - commission` and perform calculation only if `commission > 0`

Severity: Gas Optimization

Context:

- [vTreasury.sol#L242-L244](#)

Description: Avoid redundant mathematical operations by caching and or performing them only when necessary.

Recommendation: Cache the value of `currentBalance - commission` and perform this calculation only when `commission > 0`.

Kiln: Fixed in [PR 61](#).

Spearbit: Fixed.

5.5.31 Refactor `If` statements that use `&&` into nested `if` statements

Severity: Gas Optimization

Context:

- [vTreasury.sol#L81](#)
- [vTreasury.sol#L160](#)
- [vTreasury.sol#L171](#)
- [vTreasury.sol#L176](#)

Description: Nested `if` statements are more gas efficient because it allows the EVM to short circuit the execution, exiting as soon as a condition is not met, without needing to evaluate the remaining conditions.

Recommendation: Break `If` statements that use `&&` into nested `if` statements.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.5.32 `_applyVote` and `_hasVoted` can be optimized using a bit operations

Severity: Gas Optimization

Context:

- [vTreasury.sol#L299-L301](#)
- [vTreasury.sol#L306-L308](#)

Description: `_applyVote` and `_hasVoted` can be optimized using a bit operations.

Recommendation:

```
diff --git a/src/vTreasury.sol b/src/vTreasury.sol
index 5893fe2..6e1703f 100644
--- a/src/vTreasury.sol
+++ b/src/vTreasury.sol
@@ -34,6 +34,9 @@ contract vTreasury is Initializable, Implementation, Fixable, IvTreasury {

    using CAddress for address;

+    /// @notice Last bit of a uint256 used in `_applyVote` and `_hasVoted`
+    uint256 internal constant UINT256_LAST_BIT =
+    0x8000000000000000000000000000000000000000000000000000000000000000;
+
    /// @notice The address to use to withdraw ethers
    address public constant ETHER = LibConstant.ETHER;
```

```

@@ -295,16 +298,20 @@ contract vTreasury is Initializable, Implementation, Fixable, IvTreasury {

    /// @notice Internal utility to apply the active bit to the vote value
    /// @param newFee The fee value in bps
    - /// @return The vote value
    - function _applyVote(uint256 newFee) internal pure returns (uint256) {
    -     return (1 << 255) + newFee;
    + /// @return appliedVote The vote value
    + function _applyVote(uint256 newFee) internal pure returns (uint256 appliedVote) {
    +     assembly {
    +         appliedVote := or(newFee, UINT256_LAST_BIT)
    +     }
    }

    /// @notice Internal utility to check on a vote value if a vote has been made
    /// @param vote The vote value
    - /// @return True if contains a vote
    - function _hasVoted(uint256 vote) internal pure returns (bool) {
    -     return (vote >> 255) & 1 == 1;
    + /// @return voted True if contains a vote
    + function _hasVoted(uint256 vote) internal pure returns (bool voted) {
    +     assembly {
    +         voted := and(vote, UINT256_LAST_BIT)
    +     }
    }
}

```

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.5.33 `_getVote` can be optimized

Severity: Gas Optimization

Context:

- [vTreasury.sol#L310-L315](#)

Description: `_getVote` is used for values that excluding the indicator bit is supposed to be `notInvalidBps` (less than or equal to `100_00` or `0x2710`). So it's just enough to mask 2 bytes and also since the returned value is compared against a `uint256` we don't need to clean it up to `uint128`

Recommendation:

```

diff --git a/src/vTreasury.sol b/src/vTreasury.sol
index 5893fe2..4c1aea6 100644
--- a/src/vTreasury.sol
+++ b/src/vTreasury.sol
@@ -309,9 +309,11 @@ contract vTreasury is Initializable, Implementation, Fixable, IvTreasury {

    /// @notice Internal utility to retrieve the bps value from the vote value
    /// @param vote The vote value
-    /// @return The voted fee value
-    function _getVote(uint256 vote) internal pure returns (uint128) {
-        return uint128(vote & uint256(type(uint128).max));
+    /// @return votedFee The voted fee value
+    function _getVote(uint256 vote) internal pure returns (uint256 votedFee) {
+        assembly {
+            votedFee := and(vote, 0xFFFF)
+        }
    }

    /// @notice Internal utility to retrieve the balance in the specified token

```

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.5.34 Replace pool.exitQueue() with exitQueue which is already defined

Severity: Gas Optimization

Context:

- [vTreasury.sol#L176-L177](#)

Description: Avoid extra call to pool.exitQueue() reusing the already defined exitQueue

Recommendation: Replace pool.exitQueue() with exitQueue which is already defined

Kiln: Fixed in [PR 61](#).

Spearbit: Fixed.

5.5.35 Pass owner to the _approve to save from reading it again from storage.

Severity: Gas Optimization

Context:

- [NFT.sol#L140](#)

Description: Pass owner to the _approve to save from reading it again from storage.

Recommendation:

```

diff --git a/src/NFT.sol b/src/NFT.sol
index 6397510..1ae3b53 100644
--- a/src/NFT.sol
+++ b/src/NFT.sol
@@ -137,7 +137,7 @@ abstract contract NFT is INFT {
     revert LibErrors.Unauthorized(msg.sender, _ownerOf(tokenId));
 }

-    _approve(to, tokenId);
+    _approve(to, tokenId, owner);
 }

    /// @inheritdoc IERC721
@@ -307,8 +307,16 @@ abstract contract NFT is INFT {
    /// @param to The address receiving the approval
    /// @param tokenId The token id
    function _approve(address to, uint256 tokenId) internal virtual {
+    _approve(to, tokenId, _ownerOf(tokenId));
+    }
+
+    /// @notice Internal utility to approve an account for a token id
+    /// @param to The address receiving the approval
+    /// @param tokenId The token id
+    /// @param _owner The address owning the token
+    function _approve(address to, uint256 tokenId, address _owner) internal virtual {
        $tokenApprovals.get()[tokenId] = to.toUint256();
        emit Approval(_ownerOf(tokenId), to, tokenId);
+    emit Approval(_owner, to, tokenId);
    }

    /// @notice Internal utility to approve an account for all tokens of another account

```

Kiln: Fixed in [PR 10](#).

Spearbit: Fixed.

5.5.36 Use the cached owner instead of using `_ownerOf(tokenId)`

Severity: Gas Optimization

Context:

- [NFT.sol#L137](#)

Description: Use the cached owner instead of using `_ownerOf(tokenId)`

Recommendation:

```

diff --git a/src/NFT.sol b/src/NFT.sol
index 6397510..b87ecb5 100644
--- a/src/NFT.sol
+++ b/src/NFT.sol
@@ -134,7 +134,7 @@ abstract contract NFT is INFT {
 }

    if (msg.sender != owner && !isApprovedForAll(owner, msg.sender)) {
-    revert LibErrors.Unauthorized(msg.sender, _ownerOf(tokenId));
+    revert LibErrors.Unauthorized(msg.sender, owner);
    }

    _approve(to, tokenId);

```


Kiln: Fixed in [PR 10](#).

Spearbit: Fixed.

5.5.37 Implement unchecked blocks in `utils.sol/src/NFT.sol` as used in OZ NFT implementation

Severity: Gas Optimization

Context:

- [NFT.sol#L261](#)
- [NFT.sol#L275-L277](#)
- [NFT.sol#L299-L300](#)

Description: `utils.sol/src/NFT.sol` is based on the OpenZeppelin NFT implementation, therefore you could use unchecked block on safe math operations.

Recommendation:

- Based on [OpenZeppelin/ERC721.sol#L273-L279](#), add unchecked on [NFT.sol#L261](#)
- Based on [OpenZeppelin/ERC721.sol#L310-L314](#), add unchecked on [NFT.sol#L275-L277](#)
- Based on [OpenZeppelin/ERC721.sol#L345-L353](#), add unchecked on [NFT.sol#L299-L300](#)

Kiln: Fixed in [PR 10](#).

Spearbit: Fixed.

5.5.38 Optimize `_hatch` function, saving a step when hatching a new Cub

Severity: Gas Optimization

Context:

- [Hatcher.sol#L342-L356](#)

Description: In the `Hatcher`, the current implementation gets the precalculated address of the cub via `_nextHatch()` before creating the contract. This can be optimized by creating the contract first, saving a step.

Recommendation: Modify the `_hatch()` function for gas optimization.

Kiln: Fixed in [PR 11](#).

Spearbit: Fixed.

5.6 Informational

5.6.1 MultiPool1155.requestExit - Consider implementing a burn function instead

Severity: Informational

Context: [MultiPool1155.sol#L61-L64](#)

Description: Currently, the requestExit function decreases the total supply without emitting any event, which will make it harder to track off-chain, moreover, the function actually implements a burn function implicitly. **Recommendation:** Consider implementing a burn function with a Transfer event instead.

Kiln: Fixed in [1a00e3](#) by implementing the auditor's recommendation.

Spearbit: Fixed.

5.6.2 Proxy/cubs behind TUPProxy/Hatcher can have their implementation updated

Severity: Informational

Context: General

Description: Proxy/cubs behind TUPProxy/Hatcher can have their implementation updated (or fixes applied to them) if their corresponding contracts are not frozen. This applies to vFactory, vTreasury, vPool, vCoverageRecipient, vExecLayerRecipient, vExitQueue, vOracleAggregator, vWithdrawalRecipient, MultiPool inherited child contracts, vNFT, MerkleVault.

Recommendation: The above should be commented and communicated to the revenant parties (users and admins) so that they would be aware of the potential risks.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.3 vExitQueue's name() can be changed.

Severity: Informational

Context:

- [vExitQueue.sol#L113-L116](#)

Description: vExitQueue's name() can be changed. name() is not immutable since the factory's admin can update the metadata.

Recommendation: The above issue should be documented for on/off-chain agents.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.4 Loop can be exited early

Severity: Informational

Context:

- [vExitQueue.sol#L306](#)

Description/Recommendation: In the loop in this context we can break once we find a matching used owner.

Kiln: Fixed in [746c52](#)

Spearbit: Fixed.

5.6.5 Suggestions regarding vExitQueue's _consumeTicket and claim

Severity: Informational

Context:

- [vExitQueue.sol#L509-L516](#)
- [vExitQueue.sol#L519-L521](#)
- [vExitQueue.sol#L516](#)
- [vExitQueue.sol#L528](#)
- [vExitQueue.sol#L543](#)
- [vExitQueue.sol#L552-L565](#)
- [vExitQueue.sol#L552](#)
- [vExitQueue.sol#L538-L540](#)
- [vExitQueue.sol#L550-L561](#)
- [vExitQueue.sol#L557](#)
- [vExitQueue.sol#L542-L568](#)

Description/Recommendation:

- [vExitQueue.sol#L509-L516](#)

some of the recommendations regarding the _claimRedeemRequests process in LiquidCollective (LC) from the following PR can be applied here for _consumeTicket:

- [PR 197](#)
- [vExitQueue.sol#L519-L521](#)

this check can be hoisted out of _consumeTicket

- [vExitQueue.sol#L516](#)

owner can be hoisted out since it all be the same at all recursion depths.

- [vExitQueue.sol#L528](#)

skipUnknownCaskId is true for the inner recursions.

- [vExitQueue.sol#L543](#)

simpler:

```
uint256 overlappingSize = (__c.position + __c.size) - __t.position;
```

also it would be best to cache __c.position + __c.size

- [vExitQueue.sol#L552-L565](#)

updating the storage ticketArray[ticketId] can be hoisted out of recursion.

- [vExitQueue.sol#L552](#)

Better to write:

```
ticketArray[ticketId].position = __c.position + __c.size;
```

and cache the end position value.

- [vExitQueue.sol#L538-L540](#)

this check is only needed for the initial call of the recursion at the original depth.

- [vExitQueue.sol#L550-L561](#)

ethToPay is return stack variable that initially has been set to 0 and before reaching either of if/else cases there are no assignments so we can just assign the value to it (if it was stored in memory and reused during recursion then += would have made sense).

```
ethToPay = __.maxExitableAmount;
```

and since it is assigned the same variable in both. the if/else block we can perform this assignment after the block.

- [vExitQueue.sol#L557](#)

simpler:

```
__.maxExitableAmount = LibUint256.min(__.t.maxExitable, __.caskAvailableAmount);
```

- [vExitQueue.sol#L542-L568](#)

if/else block can be removed if we calculate the overlappingSize as:

```
uint256 overlappingSize = LibUint256.min(__.t.size, (__.c.position + __.c.size) - __.t.position);

// amount using cask conversion rate
__.caskAvailableAmount = LibUint256.mulDiv(overlappingSize, __.c.value, __.c.size);

// amount using the internal ticket conversion rate
uint256 relativeTicketMaxExitableAmount = LibUint256.mulDiv(overlappingSize, __.t.maxExitable,
↳ __.t.size);

// amount using the minimum rate
__.maxExitableAmount = LibUint256.min(__.caskAvailableAmount, relativeTicketMaxExitableAmount);

// cask surplus amount
__.unclaimedAmount = __.caskAvailableAmount > __.maxExitableAmount ? __.caskAvailableAmount -
↳ __.maxExitableAmount : 0;

ethToPay = __.maxExitableAmount;

ticketArray[ticketId].position += uint128(overlappingSize);
ticketArray[ticketId].size -= uint128(overlappingSize);

// different from the Liquid-Collective strategy
ticketArray[ticketId].maxExitable -= uint128(relativeTicketMaxExitableAmount);

// note the simplification below
emit FilledTicket(ticketId, caskId, uint128(overlappingSize), ethToPay, __.unclaimedAmount);
```

Kiln: Fixed [PR 63](#).

Spearbit: Fixed.

5.6.6 All exit queues of the pools connected to the same factory would have the same name

Severity: Informational

Context:

- [fvExitQueue.sol#L113-L116](#)

Description: All exit queues of the pools connected to the same factory would have the same name.

Recommendation: If it is desired for these exit queues to have distinguished names, it might be best to add unique suffixes to them.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.7 All vPools of the same factory would have the same name().

Severity: Informational

Context:

- [vPool.sol#L283-L289](#)

Description: All vPools of the same factory would have the same name().

Recommendation: If it is desired for vPools to have distinguished names, it might be best to add a unique suffix.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.8 vPool.name() might not always return the same name

Severity: Informational

Context:

- [vPool.sol#L283-L289](#)

Description: vPool.name() might not always return the same name. name() is not immutable since the factory's admin can [update](#) the metadata. Also implementation is different from vExitQueue.

Recommendation: It might be great to document the above in the contract and also for the off-chain agents.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.9 Suggestions for vPool's report(...) endpoints

Severity: Informational

Context:

- [vPool.sol#L752-L760](#)
- [vPool.sol#L751-L770](#)
- [vPool.sol#L883-L895](#)
- [vPool.sol#L944](#)
- [vPool.sol#L971](#)
- [vPool.sol#L1003-L1010](#)
- [vPool.sol#L1031-L1034](#)

- vPool.sol#L1016-L1035
- vPool.sol#L1002-L1010
- vPool.sol#L1058-L1060
- vPool.sol#L906
- vPool.sol#L930-L931

Description/Recommendation:

- vPool.sol#L752-L760

calculation of `__.traces.consensusLayerDelta` can be hoisted out of the if/else block:

```
__.traces.consensusLayerDelta = int128(historicalVolume) - int128(__.preHistoricalVolume);

if(__.traces.consensusLayerDelta < 0) { ... }
else if (__.traces.consensusLayerDelta < int128(__.traces.increaseLimit)) { ... }
else { ... }
```

Also `__.traces.consensusLayerDelta` is not a good variable name, as this reflects the net change of balances in both CL and EL summed up.

`__.traces.consensusLayerDelta` is only used in this if/else block and would be copied to `__.traces.revenue` both would have the same value here and so we can remove `__.traces.consensusLayerDelta` completely (or keep it for event emission):

```
__.traces.revenue = int128(historicalVolume) - int128(__.preHistoricalVolume);
__.traces.consensusLayerDelta = __.traces.revenue;

if(__.traces.revenue < 0) { ... }
else if (__.traces.revenue < int128(__.traces.increaseLimit)) { ... }
else { ... }
```

- vPool.sol#L751-L770

cleaner:

```

___.traces.revenue = int128(historicalVolume) - int128(___.preHistoricalVolume);

//
// ----- Based only on reported information, we check how the margin should be updated,
//
if (___traces.revenue < 0) {
    // if the balance has decreased, we increase
    // our margin and capacity of eth to pull into the system
    ___.increaseCredit += uint256(-int256(___traces.revenue));
} else if (___traces.revenue < int128(___increaseCredit)) {
    // if the balance has increased, while staying under the upper bound limit, we update the
    // margin by reducing its capacity
    ___.increaseCredit -= uint256(int256(___traces.revenue));
} else {
    // otherwise, it means that the balance increased outside of the allowed upper bound
    revert UpperBoundCrossed(historicalVolume - ___.preHistoricalVolume, ___traces.increaseLimit);
}

___traces.consensusLayerDelta = ___traces.revenue;
___traces.delta = ___traces.revenue;

```

or even simpler if `___increaseCredit` was `int128` we could do:

```

___traces.revenue = int128(historicalVolume) - int128(___preHistoricalVolume);

if (___traces.revenue > ___increaseCredit) {
    revert UpperBoundCrossed(___traces.revenue, ___increaseCredit);
}

___traces.consensusLayerDelta = ___traces.revenue;
___traces.delta = ___traces.revenue;
___increaseCredit -= ___traces.revenue; // will be non-negative due to the check above

```

For the current and any of the suggested implementations, we should check the effect of signed to/from unsigned conversions.

- [vPool.sol#L883-L895](#)

By definition commitment is not greater than deposited and in the outer if block we know `deposited > 0` so if:

- `commitment == 0`, we would have `_setDepositedEthers(0)`
- `deposited > commitment`, we would have `_setDepositedEthers(0)`
- `deposited == commitment > 0`, we would have `_setDepositedEthers(0)`

So either way in the if block we end up `_setDepositedEthers(0)`. So if `$ethers.get().deposited` is positive we set it to 0 in this if block. Therefore after this if block `$ethers.get().deposited` is guaranteed to be 0. That means the other updates we performed to `$ethers.get().deposited` when we injected ether to this pool were not necessary (during different pulling and covering mechanisms or when the new ether skimmed was added, we could have kept this in memory and then distribute here). Here is a simplified version of this block without considering the last fact:

```

uint256 deposited = $ethers.get().deposited;

if (deposited > 0) {
    uint256 commitment = LibUint256.min(deposited, rprt.maxCommittable > 0 ?
    ↪ uint256(rprt.maxCommittable) : 0);
    commitment -= commitment % LibConstant.DEPOSIT_SIZE;

    if (commitment > 0) {
        _setCommittedEthers(uint128($ethers.get().committed + commitment));
    }

    __.traces.exitBoostEthers = uint128(deposited - commitment);
    _setDepositedEthers(0);
}

// $ethers.get().deposited would be 0 here

```

Note that `$ethers.get().deposited` gets distributed between `commitment` and `__.traces.exitBoostEthers`

- [vPool.sol#L944](#)

`__.traces.exitBurnedShares > 0` implies `__.traces.exitFedEthers > 0` so the conditional can be simplified to

```

if (__.traces.exitBurnedShares > 0) ...

```

- [vPool.sol#L971](#)

`$ethers.get().deposited` should be 0 at this point.

note:

- [discussion](#)
- [vPool.sol#L1003-L1010](#)

`$coveredBalanceSum.get() + __.traces.pulledCoverageFunds` can be cached and reused in the 2nd if block;

- [vPool.sol#L1031-L1034](#)

`__.traces.decreaseLimit` is only used here. So its definition can be moved here and only be calculated when needed:

```

__.traces.decreaseLimit = uint128(_maxAllowedBalanceDecrease(__.traces.preUnderlyingSupply));

```

- [vPool.sol#L1016-L1035](#)

the checks for `__.traces.delta` can be moved to this [scope](#) above. This should help make the process cleaner and also revert early if needed without doing further calculation.

- [vPool.sol#L1002-L1010](#)

the checks and updates for `__.traces.pulledCoverageFunds` can be moved to this [scope](#) above. This should help make the process cleaner and also revert early if needed without doing further calculation.

- [vPool.sol#L1058-L1060](#)

The only place we set the requested exits for this pool. This would guarantee that the `$requestedExits.get()` will be lower than or equal to the `$withdrawalRecipient.get()`'s channel funded amount in the factory.

- [vPool.sol#L906](#)

`__.traces.newExitedEthers + __.traces.exitBoostEthers` term is important and is related to how deposited ether has been updated above (1, 2). This needs to be noted to prevent future accounting errors.


```

...

_setDepositedEthers($ethers.get().deposited - uint128(__.traces.newExitedEthers));

...

__.traces.exitBoostEthers = uint128(deposited - commitment);
_setDepositedEthers(0);

...

```

- [vPool.sol#L930-L931](#)

if there are no stack-too-deep errors it would be great to define a new stack variable to highlight that `extraDemand - LibUint256.min(extraDemand, __.traces.exitingProjection)` is the extra demand amount that will not be covered with the projected exiting amount.

Kiln: Fixed [PR 185](#).

Spearbit: Fixed.

5.6.10 The precision returned by `rate()` is not used during the calculations internally

Severity: Informational

Context:

- [MultiPool20.sol#L62](#)
- [vPool.sol#L312](#)

Description: The precision returned by `rate()` is not used during the calculations internally. Wherever `_totalUnderlyingSupply()` and `_totalSupply()` are used to calculate other parameters, for example in conversions from shares to underlying balances or vice versa the precision used is not `1e18`.

Recommendation: Perhaps this should be documented and communicated to the users/devs.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.11 The logic around decrementing `$poolShares.get()[poolId]` and transferring shares to the exit queue can be refactored

Severity: Informational

Context:

- [MultiPool1155.sol#L65-L70](#)
- [MultiPool20.sol#L115-L119](#)
- [MultiPool.sol#L158-L163](#)

Description/Recommendation: In `MultiPool`'s `exitCommissionShares` we have:

```

$poolShares.get()[poolId] -= shares;
IvPool pool = _getPool(poolId);
bool result = pool.transferShares(pool.exitQueue(), shares, abi.encodePacked(msg.sender));
if (!result) {
    revert PoolTransferFailed(poolId);
}

```

We can refactor this logic since it has been used also among the child contracts:

- [MultiPool1155.requestExit\(...\)](#)
- [MultiPool20._sendToExitQueue\(...\)](#)

There is slight differences on where `IvPool pool = _getPool(poolId)` is defined. But we can also pass the `pool` to this internal function which can be inherited by the child contracts.

Note that this is the only logic that decrements `$poolShares.get()[poolId]` just before transferring the shares to `pool.exitQueue()`

Kiln: Recommendation has been applied in [0ddccf](#).

Spearbit: Fixed.

5.6.12 Anyone can call `vFactory.exitTotal(...)` to cause custom events to be fired

Severity: Informational

Context:

- [vFactory.sol#L581-L592](#)

Description: Anyone can call `vFactory.exitTotal(...)`'s endpoint and cause some custom events to be emitted by this contract. This might introduce some unexpected behaviour for off-chain agents that are listening to these events.

Recommendation: If it is important to only fire these events from allowed actors, `vFactory` would need to keep track of the withdrawal channels that are associated with its spawned `vPools` and cross-reference the `msg.sender` with that information.

Kiln: Yes, but very easy to filter + ignore and gas cost is a good enough DoS protection. A comment has been added [fa6b05](#).

Spearbit: Fixed.

5.6.13 `vFactory._approve(...)` can be improved regarding some edge cases

Severity: Informational

Context:

- [vFactory.sol#L672-L691](#)

Description/Recommendation:

Current [implementation](#) of `LiquidCollective` (LC) for the corresponding endpoint is slightly different and can be applied here in `_approve(...)` for setting the limit. Differences are that:

- In LC we skip when the snapshot limit is stale
- In LC if the limit is low we revert instead of setting it to `funded`
- In LC it is checked that the limit is not too high surpassing the current set of keys.

related:

- [discussion](#)

For longer discussion around setting the `limit` check this LC PR:

- [PR 140](#)

Note that in LC skipping stale limit updates were chosen due to some DoS scenarios by the operator.

Kiln: Agreed for the skip + not too high.

For the case where we would reduce (and ignore snapshot), it would be better to allow an argument lower than the `funded` (and we would just set the limit to `funded` in that case) because:

- we might want to just set 0 to disable no matter the current state
- a funding can happen between the moment the proposal is crafted and broadcasted

Spearbit: In the commit [0087cb](#) the following are enforced:

- we skip when the snapshot limit is stale
- if the limit is low we revert instead of setting it to funded
- it is checked that the limit is not too high surpassing the current set of keys.

5.6.14 `_removeValidatorsFromWithdrawalChannel` can be simplified

Severity: Informational

Context:

- [vFactory.sol#L734-L736](#)
- [vFactory.sol#L738-L740](#)

Description/Recommendation: `_removeValidatorsFromWithdrawalChannel` has a loop that checks the following:

```
if (indexToRemove < withdrawalChannelStruct.funded) {
    revert FundedValidatorRemovalAttempt(index, indexToRemove);
}

if (indexToRemove >= totalValidatorCount) {
    revert ValidatorIndexOutOfBounds(index, indexToRemove);
}
```

as the indexes is checked to be a strictly decreasing array of ids:

```
if (idx > 0 && indexToRemove >= indexes[idx - 1]) {
    revert UnsortedIndexArray(index);
}
```

one would only require to compare `indexes[0]` with `totalValidatorCount` and `indexes[indexesCount - 1]` with `withdrawalChannelStruct.funded`.

Also the implementation can be adopted from the following issue to further simplify the logic:

- [PR 30](#)

Kiln: Fixed in:

- [379379c279eca9b4c614aae3c22bbb47213dd777](#)
- [fbdcbcaa7263097b1b6b763eda93f071ded391ad9](#) fixes an error that was introduced in the commit above.

Spearbit: Fixed.

5.6.15 MinimalRecipient can call the target even when the supplied value to exec is 0

Severity: Informational

Context:

- [MinimalRecipient.sol#L48](#)

Description: MinimalRecipient can call the target even when the supplied value to exec is 0. Also the value provided is not checked whether it exceeds the balance to potentially revert with a custom error.

Recommendation: The above should be documented and it should be decided if the current implementation matches with what the internal spec is asking for.

Kiln: Upper bound check has been added in [382f50](#).

Spearbit: Fixed.

5.6.16 Document what it would mean for the owner or user set the token user expiration to 0

Severity: Informational

Context:

- [vNFT.sol#L331](#)
- [vNFT.sol#L340](#)

Description: If the owner or the user of a token would set the token's user expiration to 0, the owner would not be able to call `clearUsership` to clear the user info. This is due to the following [check](#):

```
if (userInfo.expiration == 0 || block.timestamp <= userInfo.expiration) {  
    revert ClearUsershipError();  
}
```

but the owner can call `setUser(...)` to reset or clear the user info.

Another important fact about `clearUsership` is that when there is a positive CL balance for this token and the user is `address(0)`, one cannot call this endpoint either.

In general when one can call `clearUsership` to clear the user info, the same actor can also call `setUser` to clear the user info. Also an approved user can call `setUser` to clear the user info even in some edge cases where one cannot call `clearUsership`.

When one has the choice of using either of these 2 endpoints to clear the user info, calling `clearUsership` would be cheaper.

Recommendation: It would be best to define and enforce stricter condition/capabilities for these two endpoints and perhaps disallow calling `setUser(...)` with an expiration which is either 0 or even in the past.

Related [PR 111](#).

Kiln: Fixed.

Spearbit: Fixed.

5.6.17 Further analysis of conversion rates used in `MultiPool... _stake(...)` is needed

Severity: Informational

Context:

- [MultiPool20.sol#L144-L146](#)
- [MultiPool20.sol#L148-L150](#)
- [MultiPool20.sol#L157](#)
- [MultiPool1155.sol#L105](#)
- [MultiPool1155.sol#L107](#)
- [MultiPool1155.sol#L115](#)

Description: When staking for both `MultiPool20` and `MultiPool1155` the conversion rates used for calculating `pSharesBought` and `sharesAcquired` can be unified and simplified. For example the conversion rate used to calculate `pSharesBought` (not considering the rounding errors) would be the internal conversion rate of the pool in this context. Using the internal conversion rate of the pool would only require fetching two values from the pool, 1 multiplication and 1 division.

Similar simplifications can be applied to the two instances of `sharesAcquired`.

Recommendation: Regardless of the simplification applied, it would be best to formulate all these values and have a design document that would define how each parameter should be calculated/updated and why. Then one should match the `MultiPool`'s implementation to those specifications.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.18 `Native20` and `Native1155` do not need to inherit from `Fixable` unless also used by `Hatcher`

Severity: Informational

Context:

- [Native20.sol#L27](#)
- [Native1155.sol#L29](#)
- [Fixable.sol#L19](#)
- [TUPProxy.sol#L108](#)
- [IntegrationFactory.sol#L33](#)
- [ProxyFactory.sol#L34](#)
- [IntegrationFactory.sol#L267](#)

Description: Both `Native20` and `Native1155` inherit from `Fixable` which is used for hatched cubs so that when there is global pending fix for a cub anyone can call the inherited `fix()` endpoint to apply those changes/fixes. But `Native20` and `Native1155` sit behind a `TUPProxy` if deployed using [IntegrationFactory](#).

Recommendation: If there is no plan to use the `Native20` and `Native1155` as implementations for a `Hatcher` one can remove the `Fixable` base contract from these 2 contracts.

Kiln: Recommendation applied in [867c968c6d5bca54deac68ccac290c6e305338df](#).

Spearbit: Fixed.

5.6.19 Token approval race condition

Severity: Informational

Context:

- [Liquid20A.sol](#)
- [Liquid20C.so](#)

Description: The token is not guarded against approval front-running attacks. Changing an allowance using the "classic" approval method brings the risk that someone may use both the old and the new allowance by unfortunate transaction ordering, resulting in a double spend.

Recommendation: Consider implementing `increaseAllowance` and `decreaseAllowance` methods to mitigate this race condition similar to what has been done in OpenZeppelin's [ERC20](#) library.

Kiln: Fixed in [67160c4ecc828192a3e1082fd20933b4457963a8](#).

Spearbit: Fixed.

5.6.20 A few of TUPProxy endpoints can be declared as `view`

Severity: Informational

Context:

- [TUPProxy.sol#L125](#)
- [TUPProxy.sol#L151](#)
- [TUPProxy.sol#L157](#)

Description: The endpoints in the above context do not have an explicit visibility although they only retrieve storage values and should not be used to manipulate the state.

- [paused\(\)](#)
- [frozen\(\)](#)
- [freezeTime\(\)](#)

Recommendation: It would be best to mark these endpoints as `view`.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.21 `vPool:report` could be refactor to reduce complexity

Severity: Informational

Context:

- [vPool.sol#L582-L1070](#)

Description: The current `report` function is highly complex, handling a multitude of operations in a single function. Due to this complexity, to avoid a "Stack Too Deep" error there is an internal struct to handle variables, [ReportInternalVariables](#). Furthermore, the function also uses internal blocks, adding another layer of complexity.

While this approach works, it results in a function that is harder to understand, maintain, and test.

Recommendation: Following the principles of Clean Code and the "Divide and Conquer" strategy, it is suggested that we could refactor this function to improve its readability and testability.

In the diffs provided, a draft implementation that demonstrates how the function might be split up. Please consider this as a high-level proof of concept. This proposed change should, significantly enhance the simplicity and readability of the function, while also eliminating the need for the memory struct and the internal blocks.

However, it's important to note that this proposed refactoring is still a draft. As such, it may contain bugs and should be thoroughly reviewed and verified before any potential adoption. A thorough set of unit tests is also strongly recommended to ensure all aspects of the refactored function(s) are behaving as expected.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.22 LibRLP is not used (and 2 other minor issues)

Severity: Informational

Context:

- [LibRLP.sol#L15](#)
- [LibRLP.sol#L47](#)
- [LibRLP.sol#L19](#)
- [LibRLP.sol#L48](#)

Description: LibRLP is not used in the `utils.sol` repo codebase. Possibly this library was going to be used for the Hatcher contract to calculate the next hatch address if `create` opcode was going to be used instead of `create2` which requires a salt.

- A typo in comment which should be 4 bytes:

```
- // We assume nobody can have a nonce large enough to require more than 32 bytes.  
+ // We assume nobody can have a nonce large enough to require more than 4 bytes.
```

Also note that the `geth` [implementation\(2, 3\)](#) of the EVM allows nonce to be `uint64`.

- The `uint256 nonce` provided to `computeAddress` is silently truncated to `uint32` if it is more than `type(uint32).max`.

Recommendation: If LibRLP is not going to be used, it would be best to be removed from the codebase. Otherwise it is recommended to fix the other two issues mentioned above:

1. The code comment typo needs to be fixed to 4 bytes or 8 bytes depending on how the next issue is also addressed.
2. Revert if nonce is more than the allowed range. Also if it is desirable to match `geth` implementation the allowed range should be extended to `uint64`.

Kiln: Fixed in [PR 7](#).

Spearbit: Fixed.

5.6.23 A hardcoded address is used in `vTreasury` to signal the usage of native token

Severity: Informational

Context:

- [vTreasury.sol#L38](#)
- [vTreasury.sol#L284](#)
- [vTreasury.sol#L324](#)
- [lvTreasury.sol#L107](#)
- [ibConstant.sol#L17](#)

Description: A special hardcoded address, ETHER (`0xEeeeeEeeeEeEeeEeEeEEEEeeeeEeeeeeeEEeE`) is used to query the `_balance(token)` or `_transfer(token, ...)` in `vTreasury`. This special address is used to signal that the token is indeed the native token of the chain.

Recommendation: The above fact needs to be documented for the relevant endpoints of `vTreasury`.

Kiln: Fixed in [PR 61](#).

Spearbit: Fixed.

5.6.24 Voting issue when upgrading to global oracle member within same epoch

Severity: Informational

Context:

- [vOracleAggregator.sol#L222-L224](#)

Description: If a regular oracle member who has already casted its vote for this epoch becomes a global oracle member, it cannot register its extra global vote for the current epoch. Its full voting power can only be counted on the next valid reporting epoch.

Recommendation: Consider updating the mechanism to account for this edge case.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.25 Refactor the `deploy` function on `ProxyFactory.sol` to use named return

Severity: Informational

Context:

- [ProxyFactory.sol#L29-L37](#)

Description: The `deploy` function currently declares and initializes a new variable `newProxy` within its body, and then returns this variable at the end of the function.

Recommendation: To improve simplicity it is suggested to refactor the function to use named return.

```
diff --git a/src/ProxyFactory.sol b/src/ProxyFactory.sol
index f0be751..e6a64bd 100644
--- a/src/ProxyFactory.sol
+++ b/src/ProxyFactory.sol
@@ -29,10 +29,9 @@ contract ProxyFactory is IProxyFactory, Administrable {
    function deploy(bytes32 channel, address owner, address implementation, bytes calldata cdata)
        external
        onlyAdmin
-        returns (address)
+        returns (address newProxy)
    {
-        address newProxy = address(new TUPProxy(implementation, owner, cdata));
+        newProxy = address(new TUPProxy(implementation, owner, cdata));
        emit DeployedProxy(channel, newProxy, implementation, cdata, owner);
-        return newProxy;
    }
}
```

Kiln: Fixed in [PR 13](#).

Spearbit: Fixed.

5.6.26 Add check for `poolId` in `exitCommissionShares` function on `MultiPool` contract

Severity: Informational

Context:

- [MultiPool.sol#L154-L156](#)

Description: In the `exitCommissionShares` function, there is no explicit check for the existence of `poolId`. If `poolId` does not exist, the compiler might make this revert, which could lead to an unclear error message and difficulty in troubleshooting.

Recommendation: It is recommended to add a check for the existence of `poolId` at the beginning of the function. If `poolId` does not exist, the function should throw a custom error, example `InvalidPoolId(uint256 poolId)`. This will improve the clarity of error messages and make troubleshooting easier.

Kiln: Fixed in [9b732773fcd909e6b889510516c82c98aa5e22ad](#)

Spearbit: Fixed.

5.6.27 Define named constants for locked states in `FeeDispatcher`

Severity: Informational

Context:

- [FeeDispatcher.sol#L38-L57](#)

Description: In the `FeeDispatcher`, the lock states for reentrancy protection are represented by raw numbers 1 and 2. This could potentially lead to confusion and errors as the meaning of these numbers may not be immediately clear.

Recommendation: To improve code readability and maintainability, it is suggested to define named constants for these locked states.

```

diff --git a/src/FeeDispatcher.sol b/src/FeeDispatcher.sol
index ff88ab6..e06502b 100644
--- a/src/FeeDispatcher.sol
+++ b/src/FeeDispatcher.sol
@@ -36,24 +36,27 @@ contract FeeDispatcher is IFeeDispatcher {
    types.Array internal constant $fee$plits =
        ↳ types.Array.wrap(0x31a3fa329157566a07927d0c2ba92ff801e4db8af2ec73f92eaf3e7f78d587a9);

    // @notice The lock to prevent reentrancy, 1 = unlocked, 2 = locked
+   uint256 internal constant LOCKED = 2;
+   uint256 internal constant UNLOCKED = 1;
+
    // @dev keccak256(bytes("feeDispatcher.1.locked"))
    types.Uint256 internal constant $locked =
        ↳ types.Uint256.wrap(0x8472de2bbf04bc62a7ee894bd625126d381bf5e8b726e5cd498c3a9dad76d85c);

    constructor() {
-       $locked.set(1);
+       $locked.set(UNLOCKED);
    }

    /// @notice Modifier to prevent reentrancy
    modifier nonReentrant() virtual {
-       if ($locked.get() == 2) {
+       if ($locked.get() == LOCKED) {
            revert Reentrancy();
        }

-       $locked.set(2);
+       $locked.set(LOCKED);

        _;

-       $locked.set(1);
+       $locked.set(UNLOCKED);
    }

    /// @inheritdoc IFeeDispatcher

```

Kiln: Fixed in [c92ac8e578a8e71c69a8a482bf543a05c8ba5aa5](#).

Spearbit: Verified.

5.6.28 Lack of setter functions for several variables in `vNFT`

Severity: Informational

Context:

- [vNFT.sol#L67](#)
- [vNFT.sol#L71](#)
- [vNFT.sol#L93](#)
- [vNFT.sol#L101](#)

Description: In the `vNFT` initialization, several variables such as `$soulboundMode`, `$factory`, `$execLayerVault`, and `$extraData` are set. However, there are no setter functions available for these variables that can be used by the admin. These variables can only be set during the proxy initialization.

Recommendation: Consider adding setter functions for these variables that can be used by the admin. The setter functions should have appropriate access controls to ensure that only authorized addresses can change

these values. It is also recommended to add sanity checks on the inputs to these setters.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.29 Review function visibility in `MultiPool20:_totalUnderlyingSupply` and `MultiPool20:_totalSupply`

Severity: Informational

Context:

- [MultiPool20.sol#L220](#)
- [MultiPool20.sol#L208](#)

Description: The functions `_totalUnderlyingSupply` and `_totalSupply` are declared as `public view` but they are not defined in the interface. This could be due to a typo or misuse of function visibility.

Recommendation: Consider changing the visibility of these functions from `public` to `internal`, if they are intended to be used only within the contract or derived contracts. However, if the exposure of these functions is intended, it would be advisable to remove the underscore prefix, as it typically denotes private or internal functions.

Kiln: These two functions have been made `internal`: [b2633052cae465344e1ab7b1eedd83ff6d7f0641](#)

Spearbit: Fixed.

5.6.30 Refactor `vPool:report` to use `LibUint256.max` for clarity and cleanliness

Severity: Informational

Context:

- [vPool.sol#L1046-L1050](#)

Description: The current implementation uses an if-statement to check if `currentRequestedExits` is less than `rppt.stoppedCount`, and if so, sets `currentRequestedExits` to `rppt.stoppedCount`. This could be refactored to use `LibUint256.max`, which would make the code cleaner and more readable.

Recommendation:

```
diff --git a/src/vPool.sol b/src/vPool.sol
index e0cc0ef..b8c913a 100644
--- a/src/vPool.sol
+++ b/src/vPool.sol
@@ -1043,11 +1043,8 @@ contract vPool is Initializable, Implementation, Fixable, IvPool {
    //
    ↪                               //
    // ----- If exit demand is not null, we compute the amount of validator we should exit and
    ↪ forward the info to the factory    //
    {
-       uint256 currentRequestedExits = $requestedExits.get();
-       uint256 initialRequestedExits = currentRequestedExits;
-       if (currentRequestedExits < rppt.stoppedCount) {
-           currentRequestedExits = rppt.stoppedCount;
-       }
+       uint256 initialRequestedExits = $requestedExits.get();
+       uint256 currentRequestedExits = LibUint256.max(initialRequestedExits, rppt.stoppedCount);
        if (__exitDemand > __traces.exitingProjection) {
            uint256 newExitRequests =
                LibUint256.ceil(LibUint256.min(__exitDemand - __traces.exitingProjection,
                    ↪ rppt.maxExitable), LibConstant.DEPOSIT_SIZE);
```

Kiln: Fixed in [PR 63](#).

Spearbit: Fixed.

5.6.31 Clarify use of indicator bit in `$operatorFeeVote` and `$globalRecipientFeeVote`

Severity: Informational

Context:

- [vTreasury.sol#L52-L59](#)

Description: The usage of the last bit in `$operatorFeeVote` and `$globalRecipientFeeVote` is not clearly documented. The last bit indicates whether the operator or global recipient has voted or not, and the rest of the parameters are the voted fees.

Recommendation: Add comments in the code to document the use of the last bit as an indicator for voting. This will improve code readability and understanding for other developers reviewing or maintaining the code in the future.

Kiln: Fixed in [PR 61](#).

Spearbit: Fixed.

5.6.32 Missing event emission when setting deposit contract address

Severity: Informational

Context:

- [Depositor.sol#L44-L47](#)

Description: The function `_setDepositContract` sets a new deposit contract address but does not emit an event to record this change. This lack of event emission could make tracking changes in the deposit contract address more challenging and less transparent.

Recommendation: Consider emitting an event when the deposit contract address is updated. The event could have a signature like `event DepositContractUpdated(address indexed oldDepositContract, address indexed newDepositContract);`, which will include the old and new deposit contract addresses.

Kiln: Fixed in [PR 9](#).

Spearbit: Fixed.

5.6.33 Rename `_onTransfer` to `_beforeTokenTransfer` for clarity

Severity: Informational

Context:

- [NFT.sol#L76-L81](#)

Description: The internal function `_onTransfer` is used as a hook that occurs during each transfer. However, the name might be misleading as it suggests that the function is called after the transfer has taken place, while it is actually called before the transfer. Also is not being triggered on `mint` or `burn`

Recommendation: For clarity, consistency, and given that `NFT.sol` is based on the [OpenZeppelin's ERC721 implementation](#), consider renaming `_onTransfer` to `_beforeTokenTransfer`. This would more accurately reflect when the function is being called during the token transfer process, thereby improving the readability of the code. Please also note in the comments that this hook does not get triggered on `mint` or `burn`. This clarification is important because in OpenZeppelin's ERC721 implementation, this hook does get triggered on `mint` or `burn`.

Kiln: Fixed in [PR 10](#).

Spearbit: Fixed.

5.6.34 Missing event for critical setting change

Severity: Informational

Context:

- [IntegrationFactory.sol#L221](#)

Description: The following critical setting changes are not included in the event emitted within the function:

- [IntegrationFactory.sol#L221](#) - `updateRequired` is not included in the event emitted

Recommendation: Ensure that all critical setting changes are included in the event so that they will be logged and available off-chain.

Kiln: Fixed in [c2c22b443f4c1941b47f690a08e46c7985531f4f](#).

Spearbit: Verified.

5.6.35 Inconsistent function naming convention

Severity: Informational

Context:

- [IntegrationFactory.sol#L94](#)
- [IntegrationFactory.sol#L128](#)

Description: Some function names do not follow a consistent naming convention. This does not conform to good practices and might affect code readability.

Recommendation: Consider changing the following function names. Consider changing the following function names:

- [IntegrationFactory.sol#L94](#)

```
- function deployNativeVPoolIntegration(address proxyOwner, Native20Configuration calldata
  ↳ config)
+ function deployNative20vPoolIntegration(address proxyOwner, Native20Configuration calldata
  ↳ config)
```

- [IntegrationFactory.sol#L128](#)

```
+ function deployNativeVPoolIntegration(address proxyOwner, bytes calldata cdata)
- function deployNative20vPoolIntegration(address proxyOwner, bytes calldata cdata)
```

Kiln: Fixed in [f52d65af7b469f000b2cc1f016a2139df391d010](#) .

Spearbit: Verified.

5.6.36 Remove unused payable to prevent accidental ETH transfer

Severity: Informational

Context:

- [vPool.sol#L557](#)

Description: Unused payable should be removed to prevent ETH from accidentally being transferred to the function and for better code cleanliness and readability.

Recommendation: Consider removing the unused payable.

Kiln: Fixed in [b1f289680347957307e723ad4f1e89de014f31ea](#) .

Spearbit: Verified.

5.6.37 Validators can change EL fee recipient anytime

Severity: Informational

Context:

- [vPool.sol#L802](#)

Description: A malicious operator could change the EL fee recipient to point to an address controlled by them, and bypass the Kiln's execution layer recipient contract entirely. As a result, the staker and vPool will not receive any EL rewards.

Recommendation: Consider having some off-chain agreement or taking some on-chain collateral to disincentivize these behaviors. Off-chain monitoring can be set up to monitor the fee recipient change activities on the consensus layer.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.38 Remove unused imports for code cleanliness

Severity: Informational

Context:

- [NFT.sol#L5](#)
- [NFT.sol#L7](#)
- [NFT.sol#L10](#)
- [Hatcher.sol#L13](#)
- [Hatcher.sol#L15](#)
- [Hatcher.sol#L17](#)
- [Hatcher.sol#L23](#)

Description: Unused imports can clutter the codebase and should be removed for better code cleanliness and readability.

Recommendation:

- `./types/bool.sol` is not used in `NFT.sol`
- `openzeppelin-contracts/utils/Strings.sol` is not used in `NFT.sol`
- `./types/bytes32.sol` is not used in `NFT.sol`
- `openzeppelin-contracts/utils/Create2.sol` is not used in `Hatcher.sol`
- `../src/interfaces/IFixer.sol` is not used in `Hatcher.sol`
- `../src/interfaces/ICub.sol` is not used in `Hatcher.sol` - `../src/libs/LibUint256.sol` is not used in `Hatcher.sol`

Kiln: Fixed in [PR 11](#).

Spearbit: Fixed.

5.6.39 Use `address(this).balance` instead of `selfbalance()`

Severity: Informational

Context:

- [vWithdrawalRecipient.sol#L74-L81](#)
- [vTreasury.sol#L331-L338](#)
- [vPool.sol#L1193-L1200](#)
- [vExecLayerRecipient.sol#L80-L87](#)

Description: Currently, the function `_thisBalance()` is used to retrieve the contract's balance by using the `self-balance()` yul instruction. However, the Solidity compiler transpiles `address(this).balance` into `selfbalance()`, so the `_thisBalance()` function is redundant.

Recommendation: Consider removing the `_thisBalance()` function and directly using `address(this).balance` instead.

- Removing the `_thisBalance()` function in `vPool.sol`
- Removing the `_thisBalance()` function in `vExecLayerRecipient.sol`
- Removing the `_thisBalance()` function in `vTreasury.sol`
- Removing the `_thisBalance()` function in `vWithdrawalRecipient.sol`

Kiln: Fixed in [PR 61](#).

Spearbit: Fixed.

5.6.40 Remove unused modifiers

Severity: Informational

Context:

- [vPool.sol#L201-L211](#)
- [vPool.sol#L179-L188](#)

Description/Recommendation: If there is no plan to use the modifiers in this context, it would be best to remove them from the codebase.

Kiln: Fixed in [bed5e12c46376ebd416e8a07dc9600573b13295e](#).

Spearbit: Verified.

5.6.41 There is not an explicit endpoint to exit shares for a `IvPool`

Severity: Informational

Context:

- [vPool.sol#L455-L462](#)
- [vPool.sol#L409-L413](#)

Description: There is no explicit endpoint to exit. One would need to call `transferShares` or `transferSharesFrom` and set either the `to` or `data` to the exit queue address.

Recommendation: It would make it easier for the UX and potentially avoid setting the `to` address incorrectly if we had:

```
function exitShares(uint256 amount, bytes calldata data) external returns (bool) {
    address to = $exitQueue.get(); // added cost of reading from storage

    LibSanitize.notZeroAddress(to);
    LibSanitize.notNullValue(amount);
    return _transfer(msg.sender, msg.sender, to, amount, data);
}

// similar endpoint for exitSharesFrom(...)
```

One call still call the `transferShares(...)` endpoint directly to avoid the extra storage read cost.

If extra endpoints are not going to be added. This intended behaviour should be documented.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.42 Oracle members need to listen to `$epochsPerFrame` parameter changes

Severity: Informational

Context:

- [vPool.sol#L390-L399](#)

Description: Oracle members can use `isValidEpoch(uint256 epoch)` endpoint to make sure they epoch they are reporting for are valid epochs. They might need to run off-chain simulation where they can manipulate the timestamp to ensure when they submit their report the epoch data is finalised (according to the timestamp requirements).

If the pool's factory admin updates the `$epochsPerFrame` parameter, the oracle members might be forced to re-submit their reports for a slightly different epoch when in cases where quorum might be almost reached.

There are edge cases where oracle members have casted their vote for some report variants for an epoch e_1 , then the pool's factory admin updated `$epochsPerFrame` which might cause no oracle members to be able to vote due to the condition of finally of the epoch and also requirement to be on a frame boundary. This would halt the voting mechanism for a certain period of time till the oracle members can vote for a new finalised on-frame-boundary epoch e_2 .

Recommendation: The above should be documented for the oracle members and perhaps their off-chain tooling would also need to listen to changes in the `$epochsPerFrame` parameter.

Kiln: It's accepted behavior, using the expiry of the frame as the general way to reset the voting data is preferred.

Spearbit: Acknowledged.

5.6.43 Rename `types` library to better indicate its purpose

Severity: Informational

Context:

- [types.sol#L15](#)

Description/Recommendation: It might be best to rename `types` library, since most of the types used point to different types of storage slot pointers.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.44 Add more documentation for `CAddress.{v,k}`

Severity: Informational

Context:

- [address.sol#L58-L66](#)

Description: `k` represents that the returned value is used for the key input of a mapping type and `v` is used to set the value of a mapping type for specific key.

Recommendation: These should be documented.

Kiln: Fixed in [PR 8](#).

Spearbit: Fixed.

5.6.45 `TUPProxy` can change its admin when frozen

Severity: Informational

Context:

- [TUPProxy.sol#L71-L73](#)

Description: `TUPProxy` can change its admin when frozen since the `changeAdmin(...)` has not been overridden, but cannot upgrade its implementation.

Recommendation: If changing admin is not required anymore after being frozen perhaps `changeAdmin(...)` can be overridden to only allow change of admin when not frozen.

Kiln: Fixed in [PR 11](#).

Spearbit: Verified.

5.6.46 `Cub` allows the beacon to call into it even when the cub is paused

Severity: Informational

Context:

- [Cub.sol#L96](#)
- [Cub.sol#L144](#)

Description: `Cub` allows the beacon to call into it even when the cub is paused:

```
if (isPaused && msg.sender != beacon && msg.sender != address(0)) {
    revert CalledWhenPaused(msg.sender);
}
```

The condition `msg.sender != beacon` as the current implementation of the beacon, `Hatcher` and `Pluggable-Hatcher` do not make a call into the cub except at defined endpoints.

Unless beacon is also behind a proxy which allows a delegate call go through.

Recommendation: If there is no plan to allow an arbitrary fallback call from the beacon to the cub when in paused state, the condition in the question can be removed:

```
if (isPaused && msg.sender != address(0)) {
    revert CalledWhenPaused(msg.sender);
}
```

Kiln: Fixed in [PR 11](#).

Spearbit: Verified

5.6.47 Remove unused return keywords

Severity: Informational

Context:

- [Administrable.sol#L84](#)
- [Administrable.sol#L97](#)
- [address.sol#L26-L31](#)

Description/Recommendation: Remove unused return keywords.

```
diff --git a/src/Administrable.sol b/src/Administrable.sol
index 1035fe6..59e2296 100644
--- a/src/Administrable.sol
+++ b/src/Administrable.sol
@@ -81,7 +81,7 @@ contract Administrable is IAdministrable {
    function _setAdmin(address newAdmin) internal {
        LibSanitize.notZeroAddress(newAdmin);
        emit SetAdmin(newAdmin);
-       return $admin.set(newAdmin);
+       $admin.set(newAdmin);
    }

    /// @notice Retrieve the pending admin address
@@ -94,6 +94,6 @@ contract Administrable is IAdministrable {
    /// @param newPendingAdmin The new pending admin address
    function _setPendingAdmin(address newPendingAdmin) internal {
        emit SetPendingAdmin(newPendingAdmin);
-       return $pendingAdmin.set(newPendingAdmin);
+       $pendingAdmin.set(newPendingAdmin);
    }
}
```

Kiln: Fixed in [PR 9](#).

Spearbit: Fixed.

5.6.48 There isn't a setter for `$fixProgresses.get()[cub.k()]` for the Hatcher admin

Severity: Informational

Context:

- [Hatcher.sol#L65-L66](#)

Description: There is not an exposed endpoint for the admin to set `$fixProgresses.get()[cub.k()]` for a cub. Let's say the admin would want to register a global fix, but would want certain cubs to be excluded.

Recommendation: If this a desired action, the specific endpoint can be added to the Hatcher contract.

Kiln: In this case we would do the filtering in an "hard-coded" manner inside the fix if we ever need to do any filtering but the above approach might be more interesting. [PR 11](#).

Spearbit: Fixed.

5.6.49 Pending fixes cannot be unregistered after the Hatcher is frozen.

Severity: Informational

Context:

- [Hatcher.sol#L103-L109](#)

Description: Pending global fixes cannot be unregistered after the Hatcher is frozen. These fixes still will be applied for the cubs who have not progressed that far.

Recommendation: It would be great to document this for the Hatcher admin.

Kiln: The general idea of freezing is that the Hatcher keeps its exact runtime state for ever (except pausing). So it's also up to the admin to properly manage the freezing. The goal would be of course to cancel the freezing process (if not too late) if we ever need to apply a fix until the fix is properly applied. [PR 11](#).

Spearbit: Fixed.

5.6.50 Calculation of `$exitedEth.get()[poolId]` when exiting from a MultiPool20

Severity: Informational

Context:

- [MultiPool20.sol#L120](#)

Description: In MultiPool20 when exiting, `$exitedEth.get()[poolId]` is [calculated](#) as:

```
$exitedEth.get()[poolId] += stakedValueBefore - _stakedEthValue(poolId);
```

`stakedValueBefore - _stakedEthValue(poolId)` is sometimes 1 wei off from what gets recorded to the exited staker's `pool.exitQueue()`'s [maxExitable](#) amount (sometimes even more due to fluctuations caused by callbacks when safe minting an exit ticket, more on that below):

$$\lfloor a \rfloor - \lfloor b \rfloor - \lfloor a - b \rfloor \in 0, 1$$

also it would be cheaper to calculate the `maxExitable` based on the pool's current internal rate and the size of the ticket shares compared to calculating both `stakedValueBefore` and `_stakedEthValue(poolId)`.

It's good to note that if one used `maxExitable` (and with the current implementation roughly) then

$$B_i^{exit} = \sum_e T_m^e$$

parameter	description
I	<code>poolId</code>
B_i^{exit}	<code>\$exitedEth.get()[poolId]</code>
e	iteration idea over exit requests
T_m^e	ticket.maxExitable

That means B_i^{exit} does not equal to the sum of amounts returned to the stakers, but the sum of max possible amounts that could be taken out of the exit tickets (which would depends on the cask rates that these tickets get matched with).

`$exitedEth.get()[poolId] += stakedValueBefore - _stakedEthValue(poolId)`, on the other hand also reflects balance fluctuations caused by transferring shares in a pool or printing tickets in the exit queue which could

have some external callbacks that would affect the pool's conversion rate. This can further deviate the current calculation from (what I expect the value needed to be calculated) `maxExitable`.

`vExitQueue` does have a callback when minting tickets (`_safeMint(owner, ticketsLength, "")`). Here `owner` would be the caller to the multi pool's exit endpoint.

`maxExitable` is up to rounding errors the `ethAmount` (underlying value calculated from the requested shares to be exited using the multi pool's conversion rate).

$$M_i = \left\lfloor \frac{\left\lfloor \frac{b_{exit} S_i}{B_i} \right\rfloor B_i}{S_i} \right\rfloor \sim b_{exit} = \left\lfloor \frac{s_{exit} B_u^M}{S_M} \right\rfloor$$

parameter	description
i	<code>poolId</code>
s_{exit}	<code>amount</code> shares to exit from the multi pool
B_u^M	multi pool's <code>_totalUnderlyingSupply()</code>
S_M	multi pool's <code>totalSupply</code>
b_{exit}	<code>ethValue</code> , underlying balance to exit using the multi pool's conversion rate
S_i	<code>pool.totalSupply()</code>
B_i	<code>pool.totalUnderlyingSupply()</code>
M_i	<code>maxExitable</code> amount for the exit queue's printed ticket

Recommendation: `stakedValueBefore - _stakedEthValue(poolId)` does reflect a component of the change in the multi pool's underlying balance (the portion not related to the commission fees) and also captures any potential fluctuation caused by callbacks when printing exit tickets. Potential further analysis and documentation is recommended.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.51 `MultiPool.changeFee(...)` can be simplified

Severity: Informational

Context:

- [MultiPool.sol#L111-L127](#)

Description: The `MultiPool.changeFee(...)`'s logic can be simplified.

Recommendation: Here is a more simplified implementation

```
function changeFee(uint256 poolId, uint256 newFeeBps) external onlyAdmin {
    uint256 earnedBeforeFeeUpdate = _integratorCommissionEarned(poolId);
    _setFee(newFeeBps, poolId);
    uint256 earnedAfterFeeUpdate = _integratorCommissionEarned(poolId);

    uint256 paid = $commissionPaid.get()[poolId];
    // can underflow both here and in original implementation
    $commissionPaid.get()[poolId] = (paid + earnedAfterFeeUpdate) - earnedBeforeFeeUpdate;
}
```

Basically we would have $\Delta C_i = \Delta I_i$ or better to say I_i

- C_i

= $I_i - C_i$ and so the invariant preserved is the commission amount owed.

parameter	description
I_i	earnedBeforeFeeUpdate
I'_i	earnedAfterFeeUpdate
C_i	paid
C'_i	(paid + earnedAfterFeeUpdate) - earnedBeforeFeeUpdate

Some of the calculation for earnedBeforeFeeUpdate and earnedAfterFeeUpdate can be shared to optimise further.

Also it should be decided what should happen when:

$(\text{paid} + \text{earnedAfterFeeUpdate}) - \text{earnedBeforeFeeUpdate} < 0$

The options are:

- revert with a custom error or
- use a different formula

Kiln: Fixed in [399c7d8d404158aaf6c6963365a6dc97c7e06781](#).

Spearbit: Fixed.

5.6.52 Document the choice of conversion rate for calculating pSharesBought

Severity: Informational

Context:

- [MultiPool20.sol#L144-L145](#)

Description: MultiPool20._stake(...) [calculates](#) the pSharesBought as below:

```
uint256 pSharesBought = LibUint256.mulDiv(
    ethForCommission, $poolShares.get()[id] - _poolSharesOfIntegrator(id), _ethAfterCommission(id)
);
```

It uses the P_i (pool with poolId id) 's internal rate restricted to the portion allocated to this multi pool:

$$\left\lfloor b_{comm} \frac{s_i - s_i^{owed}}{b_i - b_i^{owed}} \right\rfloor$$

parameter	description
b_{comm}	ethForCommission
s_i	$\$poolShares.get()[id]$
s_i^{owed}	$_poolSharesOfIntegrator(id)$
b_i	$_stakedEthValue(poolId)$

parameter	description
b_i^{owed}	comOwed

not including the portions for the owed amount.

Note that forgetting the rounding errors introduced from multiple divisions, this rate should roughly be the pool's internal conversion rate.

There are some edge cases where `pool.totalUnderlyingSupply()` or `pool.totalSupply()` is 0.

Recommendation: Document why $\frac{s_i - s_i^{owed}}{b_i - b_i^{owed}}$ was chosen as the conversion rate.

Kiln: Acknowledged.

Spearbit: Acknowledged.

5.6.53 Keep the struct fields in the same order for `Native1155Configuration` and `Native20Configuration`

Severity: Informational

Context:

- [INative20.sol#L23-L32](#)
- [INative1155.sol#L23-L32](#)

Description/Recommendation: Keep the order of fields consistent with between `Native20Configuration` and `Native1155Configuration`:

```
struct Native20Configuration {
    string name;
    string symbol;
    address admin;
    address[] commissionRecipients;
    uint256[] commissionDistribution;
    address[] pools;
    uint256[] poolFees;
    uint256[] poolPercentages;
}

struct Native1155Configuration {
    string name;
    string symbol;
    address admin;
    address[] commissionRecipients;
    uint256[] commissionDistribution;
    address[] pools;
    uint256[] poolFees;
    string baseUri;
}
```

Kiln: Fixed in [90eb52e042b0883d3c58d541fa2df3e0b7c57532](#).

Spearbit: Verified.

5.6.54 The balancing algorithm performed in `_requestExit` depends on the order of pools

Severity: Informational

Context:

- [MultiPool20.sol#L85-L96](#)

Description: The balancing algorithm performed in `_requestExit` depends on the order of pools. One might not always pick the pool with the worst imbalance.

The current algorithm when there are more than one pool and the amount to be exited is less than 1% of the TVL picks the first pool in the pool list with a usable imbalance to fulfil the exit request.

Recommendation: The above should be documented. One can pick a different algorithm above such as picking the pool with the worst imbalance but those strategies might be more gas intensive for the user.

Kiln: The picking algorithm has been changed in the commit below to pick the worst imbalance if certain conditions have been met [ea6c469d25336d0c8f1e93ac7d2047d600707cd1](#).

Spearbit: Fixed.

5.6.55 `FeeDispatcher` does not have an initialiser but a constructor

Severity: Informational

Context:

- [FeeDispatcher.sol#L42-L44](#)

Description: `FeeDispatcher` is used in the following inheritance hierarchy

Where

- `MultiPool...` are abstract contracts
- `Native...` have initialisers and so live behind proxy contracts
- `Liquid...` add some extra endpoints to `Native...`

The inherited `FeeDispatcher` in an implementation contract would have constructor to set the `locked` to 1 and it does not have an initialiser. Moreover, anyone can call the `withdrawCommission()` which might not perform any fund distribution in the implementation contract.

The proxy contract which uses this implementation starts with `locked` as 0 since there are no initialisers and after the first call to `withdrawCommission()` its value remain at 1.

Recommendation: It might make sense to have the implementation contract in the `locked == 2` state in the constructor and have initialisers that set the `locked` state to 1.

Kiln: Recommendation has been added in [d8426edf92f1a9291e0d6836f894bc80904a5ea3](#).

Spearbit: Fixed.

5.6.56 Some contracts can be marked `abstract`

Severity: Informational

Context:

- [FeeDispatcher.sol#L26-L41](#)
- [Administrable.sol#L21](#)
- [Initializable.sol#L20](#)

Description:

- best to mark [FeeDispatcher](#) contract as `abstract` since `$feeRecipients` and `$feeSplits` cannot be set in this contract.
- [Administrable](#)
- [Initializable](#)

Recommendation: Contracts in this context can be marked as `abstract`.

Kiln: Only [FeeDispatcher](#) is marked `abstract` [e15ca8d646987b3202e955bf8a8a2adb571a9c61](#).

Spearbit: Fixed.

5.6.57 Add more documentation for `$frameSize` in `MerkleVault`

Severity: Informational

Context:

- [MerkleVault.sol#L58](#)

Description: `$frameSize` is defined and has a setter and getter function but it's not used in the logic of `MerkleVault` and from the documentation it is not clear how it is used and why it is needed in this contract.

Recommendation: It would be great to document `$frameSize`'s role in `MerkleVault` for example if it's merely used for some off-chain tooling.

Kiln: Similar to what the Oracle is doing on the `vPool`, the accounting engine will craft a report at the state of the first slot of the first epoch of each frame. Then we will craft a tree, publish it to ipfs and publish its root to the contract.

Spearbit: Acknowledged.

5.6.58 Error and typos in comments / `NatSpec`

Severity: Informational

Context:

- [IFixer.sol#L16-L17](#)
- [MerkleVault.sol#L56](#)
- [Freezable.sol#L25](#)
- [IMerkleVault.sol#L72](#)
- [vFactory.sol#L719](#)
- [vFactory.sol#L720](#)
- [vPool.sol#L613](#)
- [MultiPool20.sol#L68](#)
- [MultiPool.sol#L295](#)

Description:

Misplaced comment for `IFixer`

type `coomon`

`MerkleVault.$frameSize`

The `Freezable`'s comment describes 'Fixable'.

The hashing function used is not `keccak256(account + amount)`

Comment for `vFactory.sol#L719` needs to mention that the indexes are supposed to be in a decreasing order and in the range `[funded, totalValidators)`

Clarify comment, `index` is only used in case of `revert` in `vFactory.sol#L720`

In `vPool.sol#L613` Change comment otherwise we take `max(amount, 32 eth) - 32 eth` into account for otherwise we take `amount - min(amount, 32 eth)` into account

`MultiPool20.sol#L68` amount of shares, not ETH

`MultiPool1.sol#L295` needs to be amount of shares

Recommendation: The above errors in the comments should be fixed.

Kiln: Fixed `16f59122e4fc744aa5b9364074d154cdb85917a4`.

Spearbit: Fixed.

5.6.59 Check factory is not address(0) in IntegrationFactory._deploy(...)

Severity: Informational

Context:

- `IntegrationFactory.sol#L266`

Description: If `_factory(channel)` is `address(0)` the deploy call returns `nothing`, but the `solc` compiler checks for this and reverts.

Recommendation: It would be great to be more explicit and

- add a test case around this scenario.
- and/or check `_factory(channel)` and revert with a custom error if it is not `initiated`.

Kiln: Fixed in `d8a9d7f19e639d331a85245755ef09dd4793227d`.

Spearbit: Fixed.

5.6.60 use abi.encodeCall instead of abi.encodeWithSelector whenever possible

Severity: Informational

Context:

- `IntegrationFactory.sol#L217`
- `IntegrationFactory.sol#L271`
- `IntegrationFactory.sol#L275`
- `Nexus.sol#L222`
- `Nexus.sol#L227`
- `Nexus.sol#L255-L270`
- `Nexus.sol#L344`

- [vFactory.sol#L808](#)
- [Cub.sol#L130](#)

Description/Recommendation: To ensure type and typo safety whenever possible use `abi.encodeCall` instead of `abi.encodeWithSelector`

Kiln: Fixed in

- [040ced8e8a268e95c108f0967e40f2b16f58b0fb](#)
- [3810f3e95a764d9c7ba4dcaf0214fb07e4f48c4a](#)
- [6d584ec39eddf3a654a2f0aa62a8cb581f41f096](#)
- [270c35d4b78fa0ddb75addedd6df3b71875ed537](#)

Spearbit: Fixed.

5.6.61 `IllegalExpirationExtensionAttempt` is not included in EIP-4907

Severity: Informational

Context:

- [IERC4907.sol#L23](#)

Description: `IllegalExpirationExtensionAttempt` is not included in [EIP-4907](#).

Recommendation: It might be best to keep that `IERC4907` interface as close as possible to the EIP-4907 and move this custom error to `vNFT` contract.

Kiln: Fixed in [5baa4ec2683123855d690e82f90fecb519b8fa5d](#).

Spearbit: Verified.

5.6.62 Expose view functions for some of the key used parameters

Severity: Informational

Context:

- [vNFT.sol#L97](#)
- [IntegrationFactory.sol#L52](#)
- [PluggableHatcher.sol#L29](#)

Description: Currently, the `$purchasePaused`, `$outdated` and `$nexus` statuses are stored internally and is not exposed via a public or external view function. This lack of an exposed endpoint prevents other contracts or users from checking if the purchase is paused, which could be crucial information for interaction with the contract.

Recommendation: Create a public or external view function that returns the status of `purchasePaused`, `outdated` and `nexus`.

- Add view function for `purchasePaused` in [src/vNFT.sol](#)

```
function purchasePaused() external view returns (bool) {
    return $purchasePaused.get();
}
```

- Add view function for `outdated` [src/IntegrationFactory.sol](#)

```
function outdated(bytes32 channel) external view returns (bool) {
    return $outdated.get()[channel.k()];
}
```

- Add view function for `nexus` in [src/PluggableHatcher.sol](#)

```
function nexus() external view returns (address) {
    return $nexus.get();
}
```

Kiln: Fixed in

- [6d1c3f94ae122c43bbd57c804a1fd1fee33b72e8](#)
- [3458ac9a9e7c7e3404548e5ebff9ca797be983e1](#)
- [5afaa340878b66d776d75da5e31681c2589cb237](#)

Spearbit: Fixed.

5.6.63 `vExitQueue.claim` - Consider adding the recipient to the revert message in case of failure

Severity: Informational

Context:

- [vExitQueue.sol#L337](#)

Description: The purpose of the `claim` function is to facilitate the claiming of ETH on behalf of a fee recipient. It is worth noting that if any of the calls to recipients fail, the entire transaction will revert.

Although it is impossible to conduct a denial-of-service (DoS) attack in this scenario, since the transaction sender can remove a problematic fee recipient tickets, it may still be challenging to determine the specific recipient request that caused the transaction to fail.

Recommendation: Consider adding the recipient to the revert message.

Kiln: Fixed in [3c002831f7132b4f0c59859360e6549eebe5345d](#) by implementing the auditor's recommendation.

Spearbit: Fixed.

5.6.64 `FeeDispatcher.withdrawCommission` - Consider adding the recipient to the revert message in case of failure

Severity: Informational

Context:

- [FeeDispatcher.sol#L72](#)

Description: The purpose of the `withdrawCommission` function is to facilitate the claiming of ETH on behalf of a fee recipient. It is worth noting that if any of the calls to recipients fail, the entire transaction will revert.

Although it is impossible to conduct a denial-of-service (DoS) attack in this scenario, since the transaction sender can remove a problematic fee recipient, it may still be challenging to determine the specific recipient request that caused the transaction to fail.

Recommendation: Consider adding the recipient to the revert message.

Kiln: Kiln acknowledges this issue and provides the following context on this decision:

- This would be bad admin behavior
- Finding the reverting recipient should be fairly straightforward using a tx simulation like tenderly

Spearbit: Acknowledged.

5.6.65 `_requestExit` should call `_burn` instead of duplicating the `_burn` logic

Severity: Informational

Context:

- [MultiPool20.sol#L76-L80](#)

Description: `_requestExit` decreases the caller balance as well as `totalSupply` but a `Transfer` event to the zero address is not emitted.

Recommendation: Consider calling `_burn` instead and if you do that, you can also remove the would-be redundant check of sufficient balance at the beginning of the function.

Kiln: Fixed in [454b3a4620ef443e45fcac82a769eb4120d41a7f](#) by implementing the auditor's recommendation.

Spearbit: Fixed.

5.6.66 Withdrawal commission process can be DOS

Severity: Informational

Context:

- [FeeDispatcher.sol#L69](#)

Description: When the `FeeDispatcher.withdrawCommission` function is executed, the entire withdrawal commission process will be DOS if a fee recipient reverts (intentionally or unintentionally).

Note that the admin can mitigate this issue by calling `changeSplit` function inherited from `MutliPool` contract to remove the bad recipient. Thus, marking this issue as Informational.

Recommendation: Considering adopting a [withdrawal pattern](#) instead.

Kiln: It is unlikely an admin will refuse to receive their commission, and would be bad admin behavior. As you pointed out the admin can fix the situation should it arise. Using a withdrawal pattern introduces a lot of complexity to fix an unlikely and low impact issue.

Spearbit: Acknowledged.

5.6.67 Use `LibUint256.mulDiv` like in above functions for calculate the `comission`

Severity: Informational

Context:

- [vTreasury.sol#L234](#)

Description: For consistency in the code use `LibUint256.mulDiv` like you do in the other functions.

Recommendation: Use `LibUint256.mulDiv(currentBalance, currentFee, LibConstant.BASIS_POINTS_MAX)` instead of `(currentBalance * currentFee) / LibConstant.BASIS_POINTS_MAX`.

Kiln: Fixed in [PR 61](#).

Spearbit: Fixed.

5.6.68 Use BASIS_POINTS_MAX instead of 10000 for clarity and consistency

Severity: Informational

Context:

- [vTreasury.sol#L162](#)
- [vTreasury.sol#L172](#)

Description: The value 10000 is used directly in calculations. This value represents the maximum BPS, but it is not explicitly defined as a constant, which could lead to potential confusion or inconsistencies if the value needs to be changed in the future.

Recommendation: It is recommended to replace the hardcoded value 10000 with a constant named `LibConstant.BASIS_POINTS_MAX`. This will improve code readability, maintainability, and consistency

Kiln: Fixed in [PR 61](#).

Spearbit: Fixed.

5.6.69 Miscalculation and lack of documentation consistency for unstructured storage slots

Severity: Informational

Context:

- [vWithdrawalRecipient.sol#L32](#)
- [vCoverageRecipient.sol#L43-L45](#)
- [vPool.sol#L96](#)
- [vOracleAggregator.sol#L59](#)

Description: In the context above either there is typo in the `@dev NatSpec` comment or the actual hash of the unstructured storage slot does not match with the comment.

Also, the NatSpec comments have two general variants:

```
keccak256(bytes("path.to.slot"))
keccak256("path.to.slot")
```

Recommendation: The following changes need to be applied:

```
diff --git a/src/vCoverageRecipient.sol b/src/vCoverageRecipient.sol
index fd11cc3..a819f0f 100644
--- a/src/vCoverageRecipient.sol
+++ b/src/vCoverageRecipient.sol
@@ -42,7 +42,7 @@ contract vCoverageRecipient is Fixable, Initializable, Implementation, IvCoverag
    /// @notice Amount of donated ETH for coverage purposes
    /// @dev keccak256(bytes("coverageRecipient.1.etherForCoverage"))
    types.Uint256 internal constant $etherForCoverage =
-    types.Uint256.wrap(0xe93b7847c80749ba2f8fc825b6270f8fe9397ebccb0d1bf9027d1701bc8bb486);
+    types.Uint256.wrap(0x78404ead9f301e938b68369206f64ac56344c119b95a4ffd13a4e2603a24feca);
    /// @notice Mapping keeping track of the donor authorizations
    /// @dev keccak256(bytes("coverageRecipient.1.donors"))
    types.Mapping internal constant $donors =
    ↪ types.Mapping.wrap(0xd52f49d8be2d7ce601d95522f8e5210aa3f0742fb69b41f8de3ce0f3de50d11f);
diff --git a/src/vOracleAggregator.sol b/src/vOracleAggregator.sol
index b8f3aec..a467a66 100644
--- a/src/vOracleAggregator.sol
+++ b/src/vOracleAggregator.sol
@@ -56,7 +56,7 @@ contract vOracleAggregator is Fixable, Initializable, Implementation, IvOracleAg
    /// @dev keccak256(bytes("oracleAggregator.1.members"))
```

```

types.Array internal constant $members =
    ↪ types.Array.wrap(0x5b2e28b4804472ea289fe5335225fbc1a7730e816c34b49b72b9a25b9a884eba);
/// @notice Array of report variants
- /// @dev keccak256(bytes("oracleAggregator.1.reportsVariants"))
+ /// @dev keccak256(bytes("oracleAggregator.1.reportVariants"))
types.Array internal constant $reportVariants =
    ↪ types.Array.wrap(0xd5a65efe21829e70e73af45217f2d820de53309c258888645bac1f3e1784b6d3);
/// @notice Array of report variants vote counts
/// @dev keccak256(bytes("oracleAggregator.1.reportVoteCounts"))
diff --git a/src/vPool.sol b/src/vPool.sol
index e0cc0ef..83d00e4 100644
--- a/src/vPool.sol
+++ b/src/vPool.sol
@@ -93,7 +93,7 @@ contract vPool is Initializable, Implementation, Fixable, IvPool {
    /// @dev keccak256(bytes("pool.1.requestedExits"))
    types.Uint256 internal constant $requestedExits =
        ↪ types.Uint256.wrap(0x9c2b631c00e01b44850d87ed83bc17dc3ac47564552a2041a5efed90136270c0);
    /// @notice The sum of covered slashed balances in the consensus layer.
-    /// @dev keccak256(bytes("pool.1.coveredBalanceSum"))
+    /// @dev keccak256(bytes("pool.1.coveredBalanceSum"))
    types.Uint256 internal constant $coveredBalanceSum =
        types.Uint256.wrap(0x9ea988a990e8bb33ba380cec278407f77e425ab7847f3f16cdf0e58a18cd237c);
    /// @notice The number of epochs per frame.
diff --git a/src/vWithdrawalRecipient.sol b/src/vWithdrawalRecipient.sol
index f072915..0806c73 100644
--- a/src/vWithdrawalRecipient.sol
+++ b/src/vWithdrawalRecipient.sol
@@ -29,7 +29,7 @@ contract vWithdrawalRecipient is Fixable, Initializable, Implementation, IvWithd
    using LAddress for types.Address;

    /// @notice Storage of the address of the vPool
-    /// @dev keccak256("withdrawalRecipient.1.vpool")
+    /// @dev keccak256("withdrawalRecipient.1.pool")
    types.Address internal constant $pool =
        ↪ types.Address.wrap(0xa10e005fb9beaf799147e65be52a7748c17540c64214d79417fe69632cb40950);

    modifier onlyPool() {

```

Also it would be great to add unit tests and potentially custom linters/detectors that would signal errors like above.

Kiln: Fixed in:

- 443b40bdbad3dc8b3a051f54c26ead013fcbdaa6
- c3dfaa218d8b5604da6a5728e4bac55e33388ecf
- 10795405f882ad40b208d6884b242bc62256e4f8 with slight inconsistency for `src/vWithdrawalRecipient.sol`

Spearbit: Fixed.

5.6.70 Use the named constants as indices in `replaceCoreHatchers(...)`

Severity: Informational

Context:

- [Nexus.sol#L196-L212](#)

Description/Recommendation: Use the named `constants` as indices in `replaceCoreHatchers(...)`.

```
uint8 internal constant FACTORY = 0;
uint8 internal constant POOL = 1;
uint8 internal constant TREASURY = 2;
uint8 internal constant WITHDRAWAL_RECIPIENT = 3;
uint8 internal constant EXEC_LAYER_RECIPIENT = 4;
uint8 internal constant COVERAGE_RECIPIENT = 5;
uint8 internal constant ORACLE_AGGREGATOR = 6;
uint8 internal constant EXIT_QUEUE = 7;
```

Kiln: Fixed in [6704d5fc4c400bc180d7d75e4252bf53eb32050b](#).

Spearbit: Fixed.

5.6.71 `Hatcher.status(...)` can be simplified

Severity: Informational

Context:

- [Hatcher.sol#L107](#)

Description: `Hatcher.status(...)`'s calculation of whether a cub is paused or not can be simplified.

Recommendation: The paused status is originally calculated as:

```
!(($globalPauseStatus.get() || $pauseStatus.get()[cub.k()].toBool()) || _isFrozen())
```

but we can apply the outer negation to simplify the expression:

```
($globalPauseStatus.get() || $pauseStatus.get()[cub.k()].toBool()) && !_isFrozen()
```

Kiln: Fixed in [PR 11](#).

Spearbit: Verified.

5.6.72 The calculation of `fixCount` in `fixes(...)` might underflow and revert

Severity: Informational

Context:

- [Hatcher.sol#L139](#)

Description: In `Hatchers` `fixes(...)` endpoint the `fixCount` is calculated as below:

```
uint256 fixCount = $fixes.toAddressA().length - currentProgress;
```

which might underflow and revert. No custom error will be thrown in this case. This can happen for example when the admin sets the `initialProgress` to a number greater than total number of global fixes.

Recommendation: For better debugging it might be best to throw a custom error or have different algorithm for calculating `fixCount`.

Kiln: Fixed in [PR 11](#).

Spearbit: Fixed.

5.6.73 Turn `Hatcher` into an abstract contract

Severity: Informational

Context:

- [Hatcher.sol#L35](#)

Description: `Hatcher` does not have a constructor, so once deployed it cannot be controlled as all the privileged accounts related to it for example the admin would be `address(0)` (or even the implementation).

Note that `PluggableHatcher` which inherits the `Hatcher` contract does have a constructor that would set these values.

Recommendation: To better signal this issue, it would be best to mark this contract as abstract.

Kiln: Fixed in [PR 11](#).

Spearbit: Verified.

5.6.74 `Hatcher`'s `InitialProgress` can be decreased which might activate stale fixes for to-be-hatched cubs

Severity: Informational

Context:

- [Hatcher.sol#L244-L256](#)

Description: The hatcher's admin can set the `InitialProgress` to a lower value which can cause old registered global fixes to be applied to to-be-hatched cubs. This is specially important to consider when updating the implementation.

Recommendation: It would be great to document this behaviour.

Kiln: A comment has been added [e76cdd61620ca9dd398d03783982994f5b5e9fb0](#).

Spearbit: Fixed.

5.6.75 Decrement storage slot hashes by one

Severity: Informational

Context: General

Description: The storage slots/hashes have known pre-images.

```
bytes32 internal constant SLOT = keccak256("path.to.slot");
```

Recommendation: It might be best to decrement them by one so that finding the pre-image would be harder.

Kiln: Fixed in [PR 60](#).

Spearbit: Fixed.

5.6.76 Rename variable \$idCounter to \$mintCounter

Severity: Informational

Context:

- [NFT.sol#L39-L42](#)

Description: The variable \$idCounter is used to count how many times a token has been minted. To make the name more descriptive, consider renaming it to \$mintCounter.

Recommendation: Rename variable \$idCounter to \$mintCounter

```
diff --git a/src/NFT.sol b/src/NFT.sol
index 6397510..aa0118b 100644
--- a/src/NFT.sol
+++ b/src/NFT.sol
@@ -36,10 +36,10 @@ abstract contract NFT is INFT {
    /// @dev keccak256(bytes("nft.1.symbol"))
    types.String internal constant $symbol =
        types.String.wrap(0xddad2df2277e0186b34991db0b7ceafa36b49b76d0a1e87f6e4d44b6b17a2080);
-    /// @notice Internal ID counter to keep track of minted tokens
-    /// @dev keccak256(bytes("nft.1.idCounter"))
-    types.Uint256 internal constant $idCounter =
-        types.Uint256.wrap(0x7846a5b82c1b046265c6a3b4fa9bcd4c12b5f063ae64a204fcd61524c68a027);
+    /// @notice Internal counter to keep track of minted tokens
+    /// @dev keccak256(bytes("nft.1.mintCounter"))
+    types.Uint256 internal constant $mintCounter =
+        types.Uint256.wrap(0x3d706fc25ad0e96a2c3fb1b58cdd70ba377f331d59f761caecaf2f3a236d99a2);
    /// @notice Internal burn counter used to keep track of the total supply
    /// @dev keccak256(bytes("nft.1.burnCounter"))
    types.Uint256 internal constant $burnCounter =
@@ -93,7 +93,7 @@ abstract contract NFT is INFT {

    /// @inheritdoc INFT
    function totalSupply() external view returns (uint256) {
-        return $idCounter.get() - $burnCounter.get();
+        return $mintCounter.get() - $burnCounter.get();
    }

    /// @inheritdoc IERC721
```

Kiln: Fixed in [PR 10](#).

Spearbit: Fixed.

5.6.77 Clear approvals after burning token

Severity: Informational

Context:

- [NFT.sol#L270-L280](#)

Description: The NFT implementation does not clear approvals upon burning. Please refer to the OpenZeppelin NFT implementation [ERC721.sol#L307-L308](#)

Recommendation: Clear approvals after burning.

```

diff --git a/src/NFT.sol b/src/NFT.sol
index 6397510..6a1b73c 100644
--- a/src/NFT.sol
+++ b/src/NFT.sol
@@ -275,6 +275,8 @@ abstract contract NFT is INFT {
    $balances.get()[from.toUint256()] -= 1;
    $owners.get()[tokenId] = 0;
    $burnCounter.set($burnCounter.get() + 1);
+   // Clear approvals from the previous owner
+   delete $tokenApprovals.get()[tokenId];

    emit Transfer(from, address(0), tokenId);
}

```

Kiln: Fixed in [PR 10](#).

Spearbit: Fixed.

5.6.78 Use `val.k()` or `val.v()` instead of `val.toUint256()` in `NFT.sol` to be consistent with the Hatcher

Severity: Informational

Context:

- [NFT.sol#L102](#)
- [NFT.sol#L261](#)
- [NFT.sol#L262](#)
- [NFT.sol#L275](#)
- [NFT.sol#L299](#)
- [NFT.sol#L300](#)
- [NFT.sol#L301](#)
- [NFT.sol#L310](#)

Description: In the NFT contract, the usage of `value.toUint256()` is inconsistent with the Hatcher contract, which uses `value.k()` and `value.v()`.

Recommendation: To maintain consistency between the NFT and Hatcher, replace all instances of `value.toUint256()` with `value.k()` if used as a key of a mapping type and `value.v()` if it's a value assignment for a mapping parameter.

Kiln: Fixed in [PR 10](#).

Spearbit: Fixed.